UNIVERSIDAD CATÓLICA BOLIVIANA “SAN PABLO” SEDE LA PAZ FACULTAD DE INGENIERÍA CARRERA DE INGENIERÍA MECATRÓNICA



UNIVERSIDAD
CATÓLICA
BOLIVIANA
LA PAZ

SIMULADOR NEUROMÓRFICO BASADO EN MEMRISTORES PARA EL ANÁLISIS Y VALIDACIÓN DE SISTEMAS NEUROMÓRFICOS

(Proyecto de Grado) presentado para la obtención del Grado de Licenciatura en Ingeniería Mecatrónica

Por: Yamil Ronald Uchani Guachalla

Tutor: Axel Daniela Campero Vega

Relator: Roberto Vidal Poma Joaniquina

La Paz - Bolivia

2026

Notación

Esta sección presenta los símbolos y acrónimos utilizados a lo largo del presente documento.

Símbolos

Símbolo	**Término**
$C2C$	Variabilidad ciclo a ciclo (*Cycle-to-Cycle*)
D	Espesor del dispositivo memristivo [nm]
$D2D$	Variabilidad dispositivo a dispositivo (*Device-to-Device*)
G	Conductancia de la celda memristiva, $G = 1/R(X)$ [S]
$\mathbf{G}$	Matriz de conductancias del crossbar, $\mathbb{R}^{N\times M}$ [S]
I_{col}	Corriente integrada por columna del crossbar [A]
I_{ideal}	Corriente ideal sin efectos parásitos [A]
R_{ON}	Resistencia en el estado de baja resistencia (LRS) [Ω]
R_{OFF}	Resistencia en el estado de alta resistencia (HRS) [Ω]
R_S	Resistencia sináptica [Ω]
R_{sense}	Resistencia de sensado del amplificador [Ω]
R_{wire}	Resistencia de línea conductora [Ω]
V_{in}	Voltaje de entrada [V]
V_{th}	Voltaje umbral de disparo [V]
V_{read}	Voltaje de lectura; no altera el estado en el protocolo simulado [V]
w	Posición de la frontera de vacancias [nm]
x	Variable de estado interna del memristor ($x = w/D$)
X	Fracción no dopada del estado, $X = 1 - x$; variable usada por el agente
Δt	Desfase temporal entre spikes, $t_{\text{post}} - t_{\text{pre}}$ [ms]
Δx	Cambio de la variable de estado x [adimensional]
ΔX	Cambio de la variable de estado X [adimensional]
η	Tasa de aprendizaje
τ_m	Constante de tiempo de membrana [s]

τ_+, τ_-	Constantes temporales de la ventana STDP [ms]

Glosario

Abreviatura	**Término**
C2C	*Cycle-to-Cycle* — Variabilidad ciclo a ciclo
D2D	*Device-to-Device* — Variabilidad dispositivo a dispositivo
DVS	*Dynamic Vision Sensor* — Sensor de visión dinámica
HRS	*High Resistance State* — Estado de alta resistencia
ISI	*Inter-Spike Interval* — Intervalo entre disparos neuronales
LIF	*Leaky Integrate-and-Fire* — Neurona de integración y disparo
LRS	*Low Resistance State* — Estado de baja resistencia
LTD	*Long-Term Depression* — Depresión a largo plazo
LTP	*Long-Term Potentiation* — Potenciación a largo plazo
MNA	*Modified Nodal Analysis* — Análisis nodal modificado
REINFORCE	Gradiente de política con baseline para aprendizaje por refuerzo
SNN	*Spiking Neural Network* — Red neuronal de pulsos
STDP	*Spike-Timing Dependent Plasticity*
TSM	*Threshold Switching Memristor*
VMM	*Vector-Matrix Multiplication* — Multiplicación matriz-vector
WTA	*Winner-Takes-All* — Ganador absoluto

Resumen

Este trabajo desarrolla y evalúa un simulador modular de dispositivos memristivos y componentes neuromórficos, junto con un agente de control que utiliza un crossbar como representación computacional de sus pesos. El sistema integra el modelo de Strukov, neuronas Leaky Integrate-and-Fire (LIF), reglas de plasticidad, variabilidad estocástica y lectura de crossbar mediante un solver nodal MNA. Con la definición residual $R^2 = 1 - \mathrm{SSE/SST}$ se obtienen $R^2 = 0{,}999657$ para el estado interno y 0,996994 para la corriente del modelo de Strukov. Para STDP, un ajuste con nueve de los 13 puntos digitalizados atribuidos a Jo et al. se evaluó en cuatro puntos reservados y obtuvo $R^2 = 0{,}94220$ y RMSE = 2,136 puntos porcentuales; el tamaño reducido del holdout limita la evidencia de generalización. La convención usada es $\Delta t = t_{\mathrm{post}} - t_{\mathrm{pre}}$, positiva para pre antes que post y LTP; la definición en prosa del artículo presenta el signo opuesto. Para LTP/LTD se simularon los pulsos de +3,2 y −2,8 V, de 300 μs, y lectura a 1 V de la Fig. 2a. El test obtuvo $R^2 = 0{,}73988$, RMSE = 8,618 nA y ERP = 31,23 %, con resultados deficientes por rama; Pearson agregado es $r = 0{,}99303$, aunque no implica concordancia de amplitud y no está definido para la predicción constante de LTP. El ajuste se hizo a la misma curva digitalizada y la evidencia es parcial. Prezioso se presenta solo como reconstrucción visual aproximada, sin métricas de ajuste ni asignación de ramas SET/RESET, porque la fuente no especifica por completo el barrido temporal. Una curva frecuencia-corriente LIF concuerda con la solución analítica del propio modelo, mientras las pruebas D2D/C2C evalúan distribuciones sintéticas, no mediciones de hardware. En un crossbar 2×2, el solver MNA difiere 10.98 % de la VMM ideal al incluir resistencias de línea y sensado. Los resultados respaldan la consistencia de varias implementaciones numéricas, pero no una validación física general. En CartPole, cinco corridas de 300 episodios con D2D/C2C habilitados obtuvieron una recompensa media de 435.31 en los últimos 200 episodios (rango: 369.66–474.31). La semilla 0 alcanzó tarde el nivel de recompensa alta, por lo que su promedio de ventana final también refleja el tiempo de aprendizaje; estos datos no establecen que una semilla sea intrínsecamente mejor que otra. Tras seleccionar un checkpoint con 50 episodios de validación por matriz, una prueba con pesos congelados en 50 episodios disjuntos obtuvo $493{,}63 \pm 0{,}93$ puntos. En Robot Sim, REINFORCE y R-STDP usan el mismo camino simulado de crossbar, dispositivo volátil de salida y neuronas LIF. La política de ambos se decodifica desde las corrientes de columna; las LIF producen spikes y participan en la dinámica WTA, pero el WTA no restringe la

acción del R-STDP actual. Su elegibilidad usa la entrada presináptica y un evento de acción centrado por la política softmax; no usa el spike LIF como evento postsináptico. La regla modula la elegibilidad con retornos futuros y actualiza el estado simulado X. Un piloto de 28 corridas comparó ambos algoritmos en CartPole, visión y obstáculos, con semillas 0 y 1 e inicializaciones $X_0 = 0{,}1$ y 0,9. Se detuvo al obtener al menos 271 exitos en la ventana móvil de 300 episodios (90.33 %). R-STDP alcanzó el criterio en 4/4 condiciones de cada tarea; REINFORCE, en 2/4 de CartPole (dos registros incompletos), 4/4 de visión y 4/4 de obstáculos. Una repetición de Car R-STDP, con evaluación cada 100 episodios, alcanzó el criterio en 2/4 condiciones para semillas 2 y 4. Son resultados exploratorios: hay dos semillas, las métricas dependen del dominio y las corridas originales de Car siguieron otra frecuencia de evaluación.

La plasticidad local R-STDP del crossbar simulado funciona como parte del enfoque híbrido que combina elegibilidad local con la señal de recompensa y asignación de crédito de REINFORCE. En cambio, la variante que usa el spike LIF real como evento post directamente en el aprendizaje de la tarea tuvo rendimiento muy bajo en pruebas exploratorias. El piloto cuantitativo corresponde al agente híbrido etiquetado R-STDP, con elegibilidad pre/acción centrada por la política; no a la variante de post LIF. La fusión actúa en la regla software de aprendizaje y recompensa; conserva el mismo modelo y camino de lectura del crossbar.

Abstract

This work develops and evaluates a modular simulator for memristive devices and neuromorphic components, together with a control agent that uses a crossbar as a computational representation of its weights. The system combines the Strukov model, Leaky Integrate-and-Fire (LIF) neurons, plasticity rules, stochastic variability, and crossbar readout through a nodal MNA solver. Evaluation combines digitized references, analytical solutions, and reproducible numerical tests. Using the residual definition $R^2 = 1 - \mathrm{SSE/SST}$, the Strukov model obtains $R^2 = 0{,}999657$ for its internal state and 0,996994 for current. An exponential STDP rule was fitted to nine of 13 digitized points attributed to Jo et al. and evaluated on four reserved points, yielding $R^2 = 0{,}94220$ and an RMSE of 2,136 percentage points; the small holdout limits the evidence for generalization. The project uses $\Delta t = t_{\mathrm{post}} - t_{\mathrm{pre}}$ (positive for pre-before-post and LTP); the paper's prose definition has the opposite sign to its digitized positive-LTP axis. For Jo et al. (2010), LTP/LTD was simulated with the Fig. 2a pulse amplitudes (+3,2 and $-2{,}8\,\mathrm{V}$), widths ($300\,\mu\mathrm{s}$), and $1\,\mathrm{V}$ reads; the held-out result was $R^2 = 0{,}73988$, RMSE = 8,618 nA, and ERP = $31{,}23\,\%$, with poor branch-wise scores and no independent-device prediction. The aggregate Pearson coefficient is $r = 0{,}99303$, but it does not imply amplitude agreement; Pearson correlation is undefined for the constant LTP predictions. The STDP comparison evaluates four reserved points. Prezioso is reported only as an approximate visual reconstruction, without fit metrics or branch attribution, because the complete sweep protocol is unavailable. An LIF frequency-current curve agrees with the analytical solution of the same model, whereas D2D/C2C tests assess synthetic distributions rather than hardware measurements. For a 2×2 crossbar, the MNA solver differs by $10.98\,\%$ from the ideal VMM when line and sensing resistances are included. These results support the numerical consistency of several implementations, but do not establish general physical validation. In CartPole, five 300-episode runs with D2D/C2C enabled achieved a mean reward of 435.31 over their last 200 episodes (range: 369.66–474.31). Seed 0 reached the high-reward regime later, so its final-window mean also reflects learning time; these results do not establish that one seed is intrinsically better than another. After selecting a checkpoint using 50 validation episodes per matrix, a frozen-weight test on 50 disjoint episodes achieved $493{,}63 \pm 0{,}93$ points. In Robot Sim, both REINFORCE and R-STDP use the same simulated crossbar, volatile output device, and LIF pathway. Both policies decode actions from column currents; LIF neurons generate spikes and participate in WTA dynamics, but

WTA does not gate the current R-STDP action selection. Its eligibility uses presynaptic input and an action event centered by the softmax policy; it does not use the LIF spike as its postsynaptic learning event. Reward-modulated eligibility uses future returns to update the simulated state X. An exploratory 28-run pilot compared both algorithms on CartPole, vision, and obstacle navigation with seeds 0 and 1 and initial states $X_0 = 0{,}1$ and 0,9. Runs stopped after at least 271 successes in a rolling window of 300 episodes (90.33met the criterion in 4/4 conditions in each task; REINFORCE met it in 2/4 CartPole conditions (two records were incomplete), 4/4 vision conditions, and 4/4 obstacle conditions. A separate Car R-STDP repeat, evaluated every 100 episodes, met the criterion in 2/4 conditions for seeds 2 and 4. These are exploratory results: there are two seeds, success metrics are domain-specific, and the original Car runs used a different evaluation frequency.

The crossbar's local reward-modulated STDP works as part of the hybrid approach, which combines local eligibility with REINFORCE's reward and credit-assignment signal. By contrast, using the real LIF spike as the postsynaptic event directly in task learning performed very poorly in exploratory runs. The quantitative pilot corresponds to the hybrid agent labelled R-STDP, using policy-centered pre/action eligibility, not the real-LIF-post variant.

Índice general

Notación III

Resumen V

Abstract VII

1. Marco Referencial **1**
1.1. Introducción . . . 1
1.2. Planteamiento del Problema . . . 2
1.2.1. Definición del Problema . . . 4
1.2.2. Pregunta de Investigación . . . 4
1.3. Objetivos . . . 4
1.3.1. Objetivo General . . . 4
1.3.2. Objetivos Específicos . . . 4
1.3.3. Criterios de Validación . . . 5
1.4. Límites y Alcances . . . 6
1.4.1. Límites . . . 6
1.4.2. Alcances . . . 7
1.5. Justificación . . . 7

2. Marco Teórico **9**
2.1. Estado del Arte . . . 9
2.1.1. Enfoque 1 . . . 9
2.1.2. Enfoque 2 . . . 10
2.1.3. Enfoque 3 . . . 11
2.1.4. Enfoque 4 . . . 11
2.1.5. Discusión . . . 12
2.2. Fundamentos Teóricos . . . 12
2.2.1. Memristor . . . 13
2.2.2. Modelo de Strukov . . . 13
2.2.3. Variabilidad Estocástica en Memristores . . . 15

2.2.4. Neurona Leaky Integrate-and-Fire . . . 16
2.2.5. Plasticidad Sináptica . . . 16
2.2.6. Arquitecturas Crossbar Memristivas . . . 18
2.2.7. Sneak Paths y Margen de Ruido . . . 20
2.2.8. Parámetros del modelo de referencia . . . 21

3. Marco Práctico **23**
3.1. Esquema general del proyecto . . . 23
3.2. Herramientas . . . 24
3.2.1. Hardware . . . 24
3.2.2. Software . . . 25
3.3. Metodología del agente neuromórfico . . . 25
3.3.1. Configuración de referencia . . . 25
3.3.2. Reglas de aprendizaje y decisión . . . 26
3.3.3. Aplicaciones evaluadas . . . 27
3.4. Validación de modelos . . . 28
3.4.1. Comparaciones cuantitativas . . . 29
3.4.2. Verificación del sistema integrado . . . 32
3.5. Resultados y discusión . . . 33
3.5.1. CartPole-v1: control con recompensa densa . . . 33
3.5.2. Visión 8×8: clasificación supervisada . . . 34
3.5.3. Navegación 10×10: control con recompensa escasa . . . 35
3.5.4. Auto 2D: entorno continuo de conducción . . . 36
3.5.5. Análisis comparativo entre tareas . . . 36
3.5.6. Piloto comparativo REINFORCE/R-STDP . . . 36
3.5.7. Limitaciones . . . 37
3.6. Análisis de costos . . . 37

4. Marco Conclusivo **39**
4.1. Conclusiones . . . 39
4.2. Respuesta a la pregunta de investigación . . . 41
4.3. Recomendaciones . . . 42
4.4. Trabajo Futuro . . . 43

A. Validación cuantitativa del simulador **45**

B. Estructura del Código Fuente y Módulos **46**
B.1. Árbol de directorios del paquete `neurolab` . . . 46
B.2. Descripción de los módulos principales . . . 48
B.3. Conjuntos de datos y salidas guardadas . . . 48
B.4. Catálogo de figuras y scripts asociados . . . 50

B.5. Resumen de la suite de pruebas unitarias (Pytest) 52
B.6. Fragmentos de Código Centrales 52
B.6.1. Modelo Matemático de Strukov (`strukov.py`) 53
B.6.2. Paso del Dispositivo (`core/memristor.py`) 53

C. Índice Maestro de Validaciones **54**
C.1. Registros del piloto comparativo REINFORCE/R-STDP 57

D. Desarrollo Detallado del Sustrato Memristivo **58**
D.0.1. Etapa 1 — Núcleo del Dispositivo Memristivo y Variabilidad 58
D.0.2. Etapa 2 — Neuronas Spiking y Sistema Híbrido 61
D.0.3. Etapa 3 — Plasticidad Sináptica 66
D.0.4. Etapa 4 — Arquitecturas Matriciales y No Idealidades 68

E. Evidencia visual de los entrenamientos **84**

Bibliografía **93**

Índice de figuras

3-1. Arquitectura general del ecosistema 24
3-2. Curvas del modelo Strukov local comparadas con las series digitalizadas atribuidas a la referencia. 28
3-3. Ajuste STDP en nueve puntos y evaluación en cuatro puntos reservados atribuidos a Jo et al. (2010). 30
3-4. Comparación LTP/LTD atribuida a Jo et al. (2010); los parámetros se ajustaron con datos de entrenamiento. 31
3-5. Distribuciones sintéticas D2D/C2C del modelo y sus referencias teóricas; no son histogramas de mediciones físicas. 32
3-6. Frecuencia de disparo LIF por Euler frente a la solución analítica del mismo modelo. 33
3-7. Recompensa de cinco corridas CartPole con D2D/C2C, suavizada para facilitar la lectura. 34
3-8. Exactitud de tres corridas de visión, suavizada con una ventana de 100 episodios. 35
3-9. Tasa de éxito de tres corridas del gridworld, suavizada con una ventana de 50 episodios. 35

D-1. Sensibilidad del dispositivo a perturbaciones gaussianas del 5 % en sus parámetros nominales. 60
D-2. Estadística D2D reportada 61
D-3. Respuesta cualitativa del sistema híbrido: Potencial de membrana $V_C(t)$ evidenciando la aceleración temporal (top), Resistencia $R_S(t)$ descendente (centro), e Intervalo Inter-Spike (ISI) modularizado (abajo). 63
D-4. Verificación de la resistencia en serie R_S proveniente de la dinámica de acoplamiento de x. 64
D-5. Validación analítica subumbral de la neurona LIF frente a la solución RC exacta en respuesta al impulso. 65
D-6. Respuesta de la neurona LIF 66
D-7. Caracterización simultánea de voltaje, corriente, resistencia y conductancia durante pulsos LTP y LTD. 68

D-8. Verificación funcional del crossbar analógico 70
D-9. Comparación cualitativa I-V con Prezioso S3a 72
D-10. Ejemplos sintéticos de conductancia 73
D-11. Respuesta I–V simulada con el modelo local inspirado en Prezioso et al. (2014) [6]. La curva se genera con parámetros configurados en el código; no incluye puntos experimentales ni constituye una validación cuantitativa del artículo. 73
D-12. Estudio de ablación comparativo (A, B, C, D) en el crossbar 24×12. . . . 76
D-13. Heatmap de la variable de estado interno X tras el entrenamiento del agente; la conductancia G se deriva de X y no se muestra en esta figura. 77
D-14. Error relativo de lectura vs. tamaño del arreglo. Crecimiento monótono desde $N = 4$ (6,5 %) hasta $N = 32$ (41 %). 78
D-15. Diagrama de dispersión MNA vs. Ideal. El error crece de forma no lineal con N. 79
D-16. Perfil de I_{col} para distintos R_{wire}. La atenuación de la corriente media aumenta con la resistencia de línea, mientras que la dispersión entre columnas se mantiene dominada por la variabilidad de $\mathbf{G}$. 80
D-17. Histograma del error relativo en VMM por tamaño de crossbar. 80
D-18. Evolución de X para celda fully-, half- y no-selected bajo 3 pulsos. 81
D-19. Media de X durante 1000 pasos en reposo para la simulación reportada. . 81
D-20. LTP/LTD y derivada discreta. La no-linealidad satura suavemente los extremos. 82
D-21. Dispersión de I_{col} y variación de $\bar{G}_{\text{mean}}$ entre 20 crossbars virtuales. 82
D-22. Costo computacional reportado del solver MNA en función de N. El ajuste de las muestras archivadas es $\mathcal{O}(N^{0,85})$. 83
D-23. I_{col} real vs. ideal al excitar únicamente la fila 0. 83

Índice de tablas

2-1. Convenciones de variables y parámetros del memristor y del crossbar. 14
2-2. Parámetros de la ventana STDP: ajuste a la serie atribuida a Jo et al. y configuración del módulo local. 18
2-3. Parámetros del dispositivo, crossbar y agente de referencia. 22

3-1. Herramientas de hardware utilizadas. 24
3-2. Herramientas de software utilizadas. 25
3-3. Parámetros de referencia del agente CartPole. 26
3-4. Configuración de las aplicaciones simuladas. 28
3-5. Métricas de validación reproducibles. R^2 es el coeficiente de determinación; ERP se normaliza por el rango de referencia. 29
3-6. Cumplimiento de los umbrales cuantitativos cuando son aplicables. 30
3-7. Cumplimiento de objetivos según el alcance de la evidencia. 31
3-8. Separación de entrenamiento, validación y test en las aplicaciones. 33
3-9. Resumen descriptivo de las corridas de presupuesto fijo. 36
3-10.Resultado del piloto de parada por éxito. Cada fracción corresponde a cuatro condiciones semilla–X_0, salvo donde se indica el alcance incompleto. 37
3-11.Estimación académica de trabajo e infraestructura. 38

B-1. Resumen de módulos del paquete `neurolab`. 48
B-2. Inventario centralizado de datos de entrada y salidas CSV citados. 49
B-3. Selección de figuras generadas 50
B-4. Pruebas automatizadas de Neuromorphic Lab y Robot Sim: 165 aprobadas (27 de octubre de 2026). 52

C-1. Índice maestro de validaciones 54

CAPÍTULO 1

Marco Referencial

1.1. Introducción

Durante las últimas décadas, el crecimiento exponencial de aplicaciones basadas en inteligencia artificial, aprendizaje automático y procesamiento masivo de datos ha impulsado una demanda cada vez mayor de recursos computacionales. Este fenómeno ha evidenciado las limitaciones presentes en las arquitecturas tradicionales basadas en el modelo de Von Neumann, donde las unidades de procesamiento y almacenamiento se encuentran físicamente separadas. Como consecuencia, una parte significativa del consumo energético total de un sistema computacional se destina al movimiento de información entre memoria y procesador, generando limitaciones de rendimiento conocidas como el cuello de botella de Von Neumann [1, 2].

La necesidad de superar estas restricciones ha motivado el desarrollo de nuevos paradigmas computacionales inspirados en el funcionamiento del sistema nervioso biológico. Entre ellos destaca la computación neuromórfica, cuyo objetivo consiste en integrar almacenamiento y procesamiento dentro de una misma estructura física para reducir el movimiento de datos y mejorar la eficiencia energética [3, 2]. Este enfoque busca reproducir características observadas en sistemas biológicos, incluyendo paralelismo masivo, aprendizaje adaptativo y tolerancia a fallos.

Uno de los dispositivos más prometedores para la implementación de sistemas neuromórficos es el memristor. Este elemento fue propuesto teóricamente por Leon Chua en 1971 como el cuarto elemento fundamental de los circuitos eléctricos, complementando al resistor, capacitor e inductor mediante una relación funcional entre flujo magnético y carga eléctrica [4]. Décadas después, Strukov et al. demostraron experimentalmente la existencia física de un dispositivo con comportamiento memristivo basado en óxido de titanio, reactivando el interés científico y tecnológico en esta área de investigación [5].

Los memristores presentan propiedades particularmente atractivas para aplicaciones neuromórficas. Entre ellas destacan su capacidad de memoria no volátil, la posibilidad de modificar gradualmente su conductancia mediante estímulos eléctricos y la capacidad de conservar es-

tados internos aun cuando la alimentación eléctrica es removida [5, 3]. Estas características permiten representar pesos sinápticos artificiales y reproducir mecanismos de aprendizaje inspirados en procesos biológicos.

Diversas investigaciones han demostrado la capacidad de los memristores para implementar fenómenos asociados a la plasticidad sináptica, incluyendo potenciación a largo plazo (Long-Term Potentiation, LTP), depresión a largo plazo (Long-Term Depression, LTD) y plasticidad dependiente del tiempo de disparo (Spike-Timing Dependent Plasticity, STDP) [6, 3]. Asimismo, los memristores pueden organizarse en arreglos tipo crossbar que permiten realizar operaciones de computación en memoria y acelerar procesos de inferencia neuronal mediante estructuras altamente paralelas [7, 8].

Sin embargo, el desarrollo de hardware neuromórfico basado en memristores continúa enfrentando desafíos importantes. Los dispositivos reales presentan fenómenos de variabilidad dispositivo a dispositivo (Device-to-Device, D2D) y variabilidad ciclo a ciclo (Cycle-to-Cycle, C2C), generando diferencias estadísticas entre dispositivos nominalmente idénticos [9, 10, 11]. Adicionalmente, el incremento en la densidad de integración de arreglos crossbar introduce corrientes parásitas conocidas como sneak paths, las cuales afectan la precisión de lectura y escritura de información [12].

Debido a estas dificultades, la simulación computacional se ha convertido en una herramienta fundamental para el análisis y validación de arquitecturas neuromórficas antes de su implementación física. Herramientas de simulación permiten estudiar el comportamiento de dispositivos memristivos, mecanismos de plasticidad y arquitecturas de interconexión reduciendo costos de desarrollo y facilitando la exploración de nuevas configuraciones experimentales [13, 14, 15].

En este contexto, el presente trabajo propone el desarrollo de un simulador neuromórfico basado en memristores capaz de integrar modelado físico del dispositivo, variabilidad estocástica, plasticidad sináptica, neuronas tipo Leaky Integrate-and-Fire y arquitecturas crossbar dentro de una misma plataforma de simulación. La propuesta busca proporcionar una herramienta que permita estudiar fenómenos neuromórficos de forma integrada y validar su comportamiento mediante comparación con resultados reportados en la literatura científica.

1.2. Planteamiento del Problema

La computación neuromórfica se ha consolidado como una de las líneas de investigación más prometedoras para el desarrollo de sistemas capaces de ejecutar tareas de aprendizaje, reconocimiento de patrones y procesamiento cognitivo con una eficiencia energética superior a la ofrecida por las arquitecturas computacionales tradicionales [3, 2]. Este interés ha impulsado el desarrollo de dispositivos electrónicos emergentes capaces de emular el comportamiento de neuronas y sinapsis biológicas, siendo los memristores una de las tecnologías más estudiadas debido a sus propiedades de memoria no volátil y modificación gradual de conductancia

[4, 5].

A pesar de los avances logrados durante los últimos años, el desarrollo de hardware neuromórfico basado en memristores continúa enfrentando importantes desafíos científicos y tecnológicos. Uno de los principales problemas radica en la complejidad inherente de los dispositivos memristivos reales. A diferencia de los modelos ideales utilizados en etapas iniciales de investigación, los dispositivos físicos presentan fenómenos de variabilidad dispositivo a dispositivo y variabilidad ciclo a ciclo, provocando diferencias significativas en parámetros como voltajes de conmutación, velocidades de transición resistiva y niveles de conductancia alcanzados durante procesos de aprendizaje [9, 10, 11].

Estas variaciones afectan directamente el desempeño de sistemas neuromórficos construidos a partir de grandes cantidades de memristores. En aplicaciones prácticas, pequeñas diferencias en el comportamiento individual de los dispositivos pueden generar modificaciones acumulativas en los pesos sinápticos de una red neuronal, alterando procesos de aprendizaje y clasificación [16, 3]. Como consecuencia, resulta insuficiente analizar únicamente modelos ideales del memristor, siendo necesario incorporar mecanismos que permitan representar comportamientos estocásticos observados experimentalmente.

Otro desafío importante está relacionado con la integración de los memristores dentro de arquitecturas neuromórficas completas. Diversos trabajos han demostrado la viabilidad de utilizar memristores como sinapsis artificiales y elementos de almacenamiento analógico [6, 17]; sin embargo, la evaluación conjunta de dispositivos memristivos, neuronas artificiales y mecanismos de aprendizaje continúa siendo un problema complejo. Con frecuencia, las investigaciones se enfocan únicamente en uno de estos componentes, dificultando el análisis integral del comportamiento del sistema.

De manera similar, los arreglos crossbar memristivos representan una de las arquitecturas más prometedoras para implementar computación en memoria y acelerar operaciones neuronales mediante paralelismo masivo [7, 8]. No obstante, el incremento de densidad en estas estructuras introduce fenómenos no deseados, entre ellos las corrientes parásitas conocidas como sneak paths. Estas corrientes generan rutas alternativas para el flujo de información, degradando la precisión de lectura y escritura y limitando la escalabilidad de los sistemas [12].

La validación experimental de estos fenómenos requiere equipamiento especializado, procesos de fabricación avanzados y recursos económicos considerables. Además, la construcción de prototipos físicos limita la capacidad de explorar rápidamente diferentes configuraciones arquitectónicas y escenarios de operación. Esta situación dificulta el estudio sistemático de la interacción entre comportamiento físico del memristor, plasticidad sináptica, actividad neuronal y arquitecturas de interconexión.

Por otra parte, existen herramientas especializadas orientadas al modelado de memristores, simulación de redes neuronales o análisis de arreglos crossbar [13, 14, 15]. Sin embargo, muchas de estas soluciones se encuentran enfocadas en problemas específicos y no integran simultáneamente el comportamiento físico del dispositivo, la variabilidad estocástica,

los mecanismos de plasticidad sináptica y los efectos arquitectónicos presentes en sistemas neuromórficos de gran escala.
En consecuencia, surge la necesidad de desarrollar una herramienta de simulación que permita analizar de manera integrada los diferentes niveles que conforman un sistema neuromórfico basado en memristores. Una plataforma de estas características facilitaría la evaluación de modelos memristivos, la implementación de neuronas artificiales, el estudio de mecanismos de aprendizaje y el análisis de arquitecturas crossbar bajo condiciones realistas de operación, reduciendo costos experimentales y favoreciendo la investigación en computación inspirada en el cerebro.

1.2.1. Definición del Problema

Actualmente, la investigación en hardware neuromórfico basado en memristores enfrenta dificultades asociadas a la complejidad de modelar simultáneamente el comportamiento físico de los dispositivos, la variabilidad inherente presente en procesos de conmutación resistiva, los mecanismos de plasticidad sináptica y las arquitecturas de interconexión utilizadas en sistemas neuromórficos. La ausencia de herramientas de simulación que integren estos elementos dentro de una misma plataforma limita la capacidad de evaluar y validar diseños neuromórficos antes de su implementación física.

1.2.2. Pregunta de Investigación

¿Con qué nivel de concordancia un simulador neuromórfico basado en memristores puede reproducir el comportamiento de dispositivos memristivos, neuronas LIF, mecanismos de plasticidad sináptica (STDP) y una arquitectura crossbar simplificada, en comparación con los resultados reportados en la literatura científica, considerando además la incorporación de variabilidad estocástica (D2D y C2C)?

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar un simulador neuromórfico basado en memristores que integre el modelado físico del dispositivo, variabilidad estocástica, plasticidad sináptica y una arquitectura crossbar simplificada, con el fin de analizar y validar cuantitativamente su concordancia con resultados reportados en la literatura científica.

1.3.2. Objetivos Específicos

1. Implementar y validar el modelo memristivo de Strukov, incorporando variabilidad estocástica dispositivo a dispositivo (D2D) y ciclo a ciclo (C2C).

2. Caracterizar el comportamiento dinámico del memristor mediante análisis de histéresis y evolución de estados resistivos bajo diferentes condiciones de excitación.

3. Implementar una neurona tipo Leaky Integrate-and-Fire (LIF) como unidad básica de procesamiento neuronal.

4. Desarrollar una sinapsis memristiva con plasticidad dependiente del tiempo de disparo (STDP) y analizar la evolución del peso sináptico.

5. Integrar los componentes memristor, sinapsis STDP y neurona LIF en una arquitectura crossbar simplificada y analizar su comportamiento colectivo.

6. Validar cuantitativamente los modelos implementados mediante comparación con resultados reportados en la literatura científica, utilizando métricas de error numérico y coincidencia funcional.

1.3.3. Criterios de Validación

La validación de los modelos implementados se realizará mediante dos enfoques complementarios:
Validación cuantitativa: Se compararán las curvas y valores numéricos obtenidos por el simulador con los resultados reportados en la literatura científica, utilizando las siguientes métricas:

- **Error relativo porcentual (ERP):** para comparar valores puntuales (voltajes de conmutación, niveles de conductancia, umbrales neuronales).

- **Error cuadrático medio (RMSE):** para comparar curvas completas (histéresis I-V, evolución temporal de estados, curvas de aprendizaje STDP).

- **Coeficiente de determinación ($\mathbf{R^2}$):** para evaluar la calidad del ajuste entre las curvas simuladas y las de referencia.

Para una referencia y_i y una predicción alineada $\hat{y}_i$, el error cuadrático medio es $\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$ y conserva las unidades de la magnitud comparada. En este trabajo, el error relativo porcentual (ERP) de una curva se calcula como $100\ \mathrm{máx}_i\,|y_i - \hat{y}_i| / (\mathrm{máx}_i\, y_i - \mathrm{mín}_i\, y_i)$; se normaliza por el rango de la referencia y no es el error porcentual punto a punto. Cuando se comparan parámetros escalares, el ERP se calcula respecto al valor de referencia correspondiente.
Validación cualitativa: Se evaluará la coincidencia funcional de los fenómenos reproducidos, verificando que el simulador capture correctamente:

- La forma de la curva de histéresis del memristor (figura en forma de lazo).

- La dependencia exponencial de la plasticidad sináptica respecto al intervalo de tiempo entre pulsos (ventana STDP).
- El comportamiento de integración y disparo de la neurona LIF (umbral, período refractario, frecuencia de disparo vs. corriente inyectada).
- La respuesta colectiva de la arquitectura crossbar ante patrones de entrada conocidos.

Se considerará que un modelo está **validado** cuando cumpla simultáneamente: ERP $< 10\,\%$ en parámetros clave, $R^2 > 0.90$ en comparación de curvas, y coincidencia cualitativa completa con el fenómeno reportado.

1.4. Límites y Alcances

1.4.1. Límites

Toda investigación requiere establecer límites que definan claramente el alcance de los resultados obtenidos y permitan delimitar el problema de estudio. En este trabajo, los límites establecidos responden a la naturaleza computacional del proyecto y a la necesidad de concentrar los esfuerzos en el análisis de sistemas neuromórficos basados en memristores.
La presente investigación se limita al desarrollo de un simulador computacional orientado al análisis de dispositivos memristivos y arquitecturas neuromórficas. Por tanto, no contempla la fabricación física de memristores ni la implementación experimental de circuitos electrónicos sobre hardware real.
El comportamiento de los dispositivos memristivos será representado mediante modelos matemáticos obtenidos de la literatura científica, incorporando fenómenos de variabilidad y comportamiento estocástico reportados experimentalmente. En consecuencia, los resultados obtenidos corresponderán a simulaciones numéricas y no a mediciones directas realizadas sobre dispositivos físicos.
La investigación se enfocará principalmente en el estudio de neuronas tipo Leaky Integrate-and-Fire (LIF), por lo que otros modelos neuronales más complejos, tales como Hodgkin-Huxley, FitzHugh-Nagumo o Izhikevich, no serán considerados dentro del alcance del presente trabajo.
Los mecanismos de aprendizaje implementados estarán limitados a procesos de plasticidad sináptica basados en STDP, utilizando memristores como elementos de almacenamiento de peso sináptico. No se abordarán otros esquemas de aprendizaje supervisado o no supervisado de mayor complejidad computacional.
Asimismo, el análisis arquitectónico estará centrado en arreglos crossbar memristivos simplificados. No se abordará la implementación de arquitecturas neuromórficas especializadas de gran escala ni sistemas comerciales de computación neuromórfica.

Finalmente, la validación de los modelos implementados se realizará mediante comparación con resultados reportados en la literatura científica, sin considerar procesos de caracterización experimental sobre dispositivos fabricados específicamente para esta investigación.

1.4.2. Alcances

El simulador desarrollado permitirá representar el comportamiento dinámico de dispositivos memristivos basados en el modelo de Strukov, incorporando fenómenos de variabilidad dispositivo a dispositivo y ciclo a ciclo observados en dispositivos reales.
La plataforma permitirá analizar características fundamentales del memristor, incluyendo curvas de histéresis corriente-voltaje, evolución temporal de estados resistivos, modificación gradual de conductancia y efectos asociados a procesos de conmutación resistiva.
Dentro del ámbito neuromórfico, el simulador incorporará neuronas tipo Leaky Integrate-and-Fire para representar mecanismos básicos de integración temporal y generación de pulsos neuronales. Estas neuronas podrán interactuar con sinapsis memristivas para reproducir comportamientos inspirados en procesos biológicos de aprendizaje.
Asimismo, se implementará un mecanismo de plasticidad sináptica basado en STDP, permitiendo estudiar la evolución de pesos sinápticos bajo diferentes patrones de estimulación neuronal.
La investigación también incluirá la implementación de arreglos crossbar memristivos simplificados orientados al análisis de arquitecturas de computación en memoria. Estas estructuras permitirán evaluar el comportamiento colectivo de múltiples dispositivos memristivos y estudiar fenómenos asociados al procesamiento paralelo de información.
Finalmente, los modelos implementados serán validados mediante comparación con resultados reportados en la literatura científica, incluyendo trabajos relacionados con modelado memristivo, plasticidad sináptica, redes neuromórficas y arreglos crossbar, proporcionando evidencia cuantitativa y cualitativa de la capacidad del simulador para reproducir fenómenos observados en investigaciones previas.

1.5. Justificación

La computación neuromórfica se ha consolidado como una de las principales alternativas para superar las limitaciones energéticas y de escalabilidad presentes en las arquitecturas computacionales tradicionales. Dentro de este contexto, los memristores han despertado un creciente interés debido a su capacidad para almacenar información de forma no volátil y modificar gradualmente su conductancia, características que permiten representar el comportamiento de sinapsis biológicas [3, 5].
A pesar de los avances reportados en la literatura, el desarrollo experimental de sistemas neuromórficos basados en memristores continúa enfrentando desafíos asociados a la fabricación de dispositivos, la variabilidad inherente de los procesos de conmutación resistiva y la

complejidad de validar arquitecturas de gran escala [11, 9, 10]. Como consecuencia, las herramientas de simulación desempeñan un papel fundamental al permitir el análisis de estos fenómenos antes de la implementación física de los sistemas.

Desde una perspectiva científica, el presente trabajo contribuye al estudio de la relación existente entre el comportamiento físico de los memristores y los mecanismos de aprendizaje inspirados biológicamente. La integración de modelos memristivos, plasticidad sináptica, neuronas artificiales y arquitecturas crossbar dentro de una misma plataforma permite analizar fenómenos neuromórficos de manera integral y facilita la reproducción de resultados reportados en investigaciones previas [6, 12].

Desde el punto de vista tecnológico, el simulador propuesto proporciona una herramienta capaz de evaluar configuraciones neuromórficas sin necesidad de recurrir inicialmente a procesos de fabricación especializados, reduciendo costos de desarrollo y facilitando la exploración de nuevas arquitecturas de computación en memoria [8, 18].

Finalmente, desde una perspectiva académica, la investigación integra conocimientos de electrónica, modelado matemático, sistemas dinámicos e inteligencia artificial, contribuyendo a la formación en un área de creciente relevancia para el desarrollo de tecnologías inspiradas en el cerebro.

CAPÍTULO 2

Marco Teórico

El presente capítulo expone los principales antecedentes científicos y tecnológicos relacionados con el desarrollo de sistemas neuromórficos basados en memristores. Se analizan diferentes enfoques reportados en la literatura científica, incluyendo el modelado físico de dispositivos memristivos, la incorporación de fenómenos de variabilidad estocástica, la implementación de redes neuromórficas basadas en memristores y el estudio de arquitecturas crossbar. Posteriormente, se presentan los fundamentos teóricos necesarios para comprender el funcionamiento de los modelos y algoritmos implementados en esta investigación.

2.1. Estado del Arte

Durante las últimas décadas, el desarrollo de dispositivos memristivos ha impulsado nuevas líneas de investigación orientadas a la construcción de sistemas computacionales inspirados en el cerebro biológico. La posibilidad de integrar almacenamiento y procesamiento dentro de una misma estructura física ha convertido a los memristores en una de las tecnologías más prometedoras para la computación neuromórfica. Como consecuencia, numerosos trabajos han abordado distintos aspectos relacionados con modelado físico, variabilidad, aprendizaje sináptico y arquitecturas de computación en memoria.
A continuación, se presentan los principales enfoques identificados en la literatura científica que sustentan el desarrollo de la presente investigación.

2.1.1. Enfoque 1

Uno de los hitos más importantes en el área de dispositivos electrónicos emergentes fue la formulación teórica del memristor realizada por Leon Chua en 1971. En dicho trabajo se propuso la existencia de un cuarto elemento fundamental de circuito capaz de establecer una relación directa entre la carga eléctrica y el flujo magnético, complementando al resistor, capacitor e inductor tradicionales [4]. Aunque inicialmente se trató de una propuesta teórica,

este concepto abrió una nueva línea de investigación orientada al desarrollo de dispositivos con capacidad de memoria integrada.
Décadas después, Strukov et al. demostraron experimentalmente la existencia física de un dispositivo con comportamiento memristivo basado en una estructura de dióxido de titanio (TiO_2) [5]. Este trabajo constituye uno de los pilares fundamentales de la investigación moderna en memristores, ya que proporciona un modelo matemático capaz de describir la evolución temporal del estado interno del dispositivo mediante una variable de estado asociada al desplazamiento de vacancias de oxígeno.
El modelo propuesto por Strukov describe un dispositivo compuesto por dos regiones con diferentes niveles de conductividad. La frontera entre ambas regiones varía en función de la corriente aplicada, modificando el valor instantáneo de resistencia del dispositivo. Como resultado, el memristor presenta una característica de histéresis en el plano corriente-voltaje, permitiendo almacenar información sin necesidad de alimentación continua [5].
La simplicidad matemática del modelo de Strukov ha favorecido su utilización en simulaciones y estudios académicos. Sin embargo, investigaciones posteriores han demostrado que los dispositivos reales presentan comportamientos más complejos que aquellos descritos por el modelo ideal. Trabajos recientes han incorporado efectos físicos adicionales, restricciones de frontera y mecanismos no lineales para mejorar la representación de dispositivos memristivos fabricados experimentalmente [11, 19].
A pesar de sus limitaciones, el modelo de Strukov continúa siendo una referencia fundamental debido a su capacidad para capturar los principios esenciales del comportamiento memristivo y proporcionar una base sólida para el desarrollo de modelos más avanzados. Por esta razón, numerosos simuladores y plataformas de investigación utilizan este modelo como punto de partida para la implementación de sistemas neuromórficos basados en memristores.

2.1.2. Enfoque 2

Aunque los modelos ideales permiten comprender los principios fundamentales de operación de un memristor, diversos estudios experimentales han demostrado que los dispositivos físicos presentan fenómenos de variabilidad que afectan significativamente su comportamiento eléctrico. Estas variaciones pueden manifestarse tanto entre dispositivos fabricados bajo condiciones nominalmente idénticas como entre diferentes ciclos de operación de un mismo dispositivo [9, 10].
La variabilidad dispositivo a dispositivo, conocida como Device-to-Device (D2D), se encuentra asociada a diferencias estructurales introducidas durante los procesos de fabricación. Por otra parte, la variabilidad ciclo a ciclo, denominada Cycle-to-Cycle (C2C), se relaciona con fluctuaciones presentes durante procesos repetitivos de escritura y borrado. Ambos fenómenos generan dispersión en parámetros como voltajes de conmutación, resistencia de encendido y resistencia de apagado [10].
Con el objetivo de representar estas características, diversos autores han propuesto modelos

estocásticos capaces de incorporar incertidumbre dentro del comportamiento memristivo. Estos modelos incluyen distribuciones probabilísticas para parámetros físicos y mecanismos de transición resistiva, permitiendo reproducir comportamientos observados experimentalmente que no pueden ser capturados mediante modelos deterministas convencionales [9, 16].
La incorporación de variabilidad constituye un aspecto particularmente importante para aplicaciones neuromórficas, ya que las diferencias entre dispositivos pueden afectar procesos de aprendizaje, almacenamiento de información y estabilidad de redes neuronales. Como consecuencia, los modelos modernos de memristores tienden a integrar mecanismos estocásticos que permitan aproximar con mayor precisión el comportamiento observado en dispositivos reales [11, 10].

2.1.3. Enfoque 3

La utilización de memristores dentro de sistemas neuromórficos ha dado lugar a una importante línea de investigación orientada a la implementación de sinapsis artificiales capaces de reproducir mecanismos de aprendizaje inspirados en procesos biológicos. Dentro de este contexto, uno de los trabajos más influyentes corresponde a la investigación desarrollada por Prezioso et al., quienes demostraron la operación de una red neuromórfica integrada basada en memristores de óxido metálico [6].
El trabajo de Prezioso evidenció la posibilidad de emplear memristores como elementos de almacenamiento de pesos sinápticos dentro de redes neuronales artificiales. Mediante la modificación gradual de la conductancia de los dispositivos, los autores lograron implementar procesos de aprendizaje capaces de clasificar patrones de entrada sin necesidad de entrenamiento externo complejo [6].
Paralelamente, diversos estudios han demostrado que los memristores pueden reproducir fenómenos de plasticidad sináptica observados en sistemas biológicos. Entre ellos destacan la potenciación a largo plazo (LTP), la depresión a largo plazo (LTD) y la plasticidad dependiente del tiempo de disparo (STDP), mecanismos considerados fundamentales para procesos de aprendizaje y adaptación neuronal [3].
Estos avances han consolidado a los memristores como una de las tecnologías más prometedoras para el desarrollo de hardware neuromórfico, permitiendo implementar sinapsis artificiales compactas, energéticamente eficientes y capaces de almacenar información de forma no volátil [17].

2.1.4. Enfoque 4

El crecimiento de las redes neuromórficas basadas en memristores ha impulsado el desarrollo de arquitecturas capaces de conectar grandes cantidades de dispositivos dentro de una misma estructura física. Entre las alternativas más estudiadas destacan los arreglos tipo crossbar, los cuales permiten organizar memristores en forma matricial para realizar operaciones de

computación en memoria mediante mecanismos altamente paralelos [7, 8].
Las arquitecturas crossbar presentan ventajas importantes en términos de densidad de integración, eficiencia energética y velocidad de procesamiento. Debido a que cada memristor puede representar un peso sináptico individual, estas estructuras resultan especialmente adecuadas para implementar redes neuronales artificiales y sistemas de inferencia neuromórfica [8].
Sin embargo, el incremento del tamaño de los arreglos crossbar introduce fenómenos no ideales que afectan su funcionamiento, tales como corrientes parásitas y degradación del margen de ruido, los cuales han sido ampliamente estudiados en la literatura como limitaciones de escalabilidad [12]. En el contexto de la presente investigación, se abordará el análisis de arreglos crossbar simplificados, dejando el estudio de estos fenómenos de escala masiva como una extensión natural para trabajos futuros.

2.1.5. Discusión

Los enfoques analizados evidencian la evolución progresiva de la investigación en sistemas neuromórficos basados en memristores. El modelo de Strukov proporciona una base matemática sólida para describir el comportamiento fundamental del dispositivo, mientras que los estudios de variabilidad permiten aproximar con mayor precisión las características observadas en dispositivos reales. Por otra parte, las investigaciones orientadas a redes neuromórficas demuestran la capacidad de los memristores para implementar mecanismos de aprendizaje inspirados biológicamente, y los estudios sobre arquitecturas crossbar muestran su potencial para desarrollar sistemas de computación en memoria de alta densidad.
Sin embargo, la mayoría de los trabajos revisados abordan estos aspectos de forma independiente. En consecuencia, existe la necesidad de contar con herramientas capaces de integrar modelado memristivo, variabilidad estocástica, plasticidad sináptica y análisis arquitectónico dentro de una misma plataforma. Esta necesidad constituye la principal motivación para el desarrollo del simulador neuromórfico propuesto en la presente investigación.

2.2. Fundamentos Teóricos

La presente investigación integra conceptos provenientes de la electrónica, la computación neuromórfica y el modelado matemático de dispositivos memristivos. En esta sección se presentan los fundamentos teóricos necesarios para comprender el funcionamiento de los modelos implementados dentro del simulador propuesto. Se abordan los principios de operación del memristor, los mecanismos de variabilidad presentes en dispositivos reales, los modelos neuronales utilizados en sistemas neuromórficos y las arquitecturas de interconexión empleadas para la implementación de redes neuronales basadas en memristores.

2.2.1. Memristor

El memristor fue propuesto por Leon Chua en 1971 como el cuarto elemento fundamental de los circuitos eléctricos, complementando al resistor, capacitor e inductor [4]. Su principal característica consiste en establecer una relación funcional entre la carga eléctrica acumulada y el flujo magnético asociado al dispositivo.
A diferencia de un resistor convencional, cuya resistencia permanece constante para un determinado valor de temperatura, el memristor posee una resistencia dependiente de la historia de excitaciones eléctricas previamente aplicadas. Como consecuencia, el dispositivo es capaz de almacenar información incluso después de retirar la alimentación eléctrica, propiedad conocida como memoria no volátil.
Matemáticamente, la relación fundamental propuesta por Chua puede expresarse como:

$$d\phi = M(q)dq \tag{2-1}$$

donde ϕ representa el flujo magnético, q corresponde a la carga eléctrica acumulada y $M(q)$ define la memristancia del dispositivo.
Esta característica permite que el memristor conserve información acerca de estados previos, convirtiéndolo en un candidato ideal para representar sinapsis artificiales dentro de sistemas neuromórficos [4, 5].

2.2.2. Modelo de Strukov

La primera demostración experimental de un dispositivo memristivo fue realizada por Strukov et al. en 2008 mediante una estructura basada en dióxido de titanio (TiO_2) [5]. El dispositivo se encuentra formado por dos regiones con diferente conductividad: una región dopada con alta conductancia y una región no dopada con baja conductancia.
La resistencia total del dispositivo puede expresarse mediante:

$$R(w) = R_{ON}\frac{w}{D} + R_{OFF}\left(1 - \frac{w}{D}\right) \tag{2-2}$$

Definiendo la fracción dopada normalizada $x = w/D \in [0, 1]$, la expresión equivalente es $R(x) = R_{ON}x + R_{OFF}(1 - x)$. En esta tesis se usa también la variable complementaria $X = 1 - x$, que representa la fracción no dopada:

$$R(X) = R_{ON} + X(R_{OFF} - R_{ON}).$$

Ambas expresiones son equivalentes. En el simulador del agente se emplea X porque la matriz de estado del crossbar se representa directamente con esta variable y la conductancia se obtiene como $G = 1/R(X)$. Todas las medias y desviaciones de X reportadas en el Capítulo 3 corresponden a esta variable de estado; no son estadísticas de G.
La Tabla **2-1** resume los símbolos y su interpretación.

Tabla **2-1**: Convenciones de variables y parámetros del memristor y del crossbar.

Símbolo	**Definición**	**Unidades**	**Rango**	**Efecto**
w	Ancho de la región dopada	m	[0,D]	Aumenta al crecer la región dopada
D	Espesor total de la capa activa	m	$D > 0$	Normaliza la posición de la frontera
x	Fracción dopada, $x = w/D$	–	[0,1]	Si x aumenta, R disminuye
X	Fracción no dopada, $X = 1 - x$	–	[0,1]	Si X aumenta, R aumenta
R	Resistencia del dispositivo	Ω	$[R_{ON}, R_{OFF}]$	–
G	Conductancia, $G = 1/R$	S	$[1/R_{OFF}, 1/R_{ON}]$	–
w_s	Peso sináptico normalizado	–	[0,1]	Si w_s aumenta, G aumenta

En la notación de la tabla:

- R_{ON} representa la resistencia mínima.
- R_{OFF} representa la resistencia máxima.
- D corresponde al espesor total del dispositivo.
- w define la posición de la frontera entre las regiones dopada y no dopada.

La evolución temporal ideal de la variable de estado se encuentra dada por:

$$\frac{dx}{dt} = \frac{1}{D}\frac{dw}{dt} = \mu_v \frac{R_{ON}}{D^2} i(t) \tag{2-3}$$

donde μ_v representa la movilidad efectiva de vacancias de oxígeno e $i(t)$ corresponde a la corriente que atraviesa el dispositivo. En la configuración Strukov–Biolek del simulador, esta dinámica se multiplica además por la función de ventana $f_B(x, i) = 1 - (x - \text{stp}(-i))^{2p}$, con $p = 5$, para atenuar la evolución al acercarse a los límites del estado, donde $\text{stp}(-i) = 1$ si $i < 0$ y 0 en caso contrario. El estado inicial del archivo de configuración es $x_0 = 0{,}1$. Si el agente deriva su estado complementario de ese valor sin una sobrescritura, resulta $X_0 = 1 - x_0 = 0{,}9$. Sin embargo, la configuración de CartPole analizada pasa explícitamente `initial_X=0.1`; por tanto, en esas corridas el valor explícito prevalece y equivale a $x_0 = 0{,}9$. Esta distinción entre el parámetro del dispositivo y el estado inicial explícito del agente se mantiene en la tabla de parámetros del Capítulo 3.

Este modelo permite reproducir la característica de histéresis observada experimentalmente en memristores y constituye la base de numerosos simuladores utilizados en investigación neuromórfica [5].

2.2.3. Variabilidad Estocástica en Memristores

Aunque el modelo de Strukov permite describir adecuadamente los principios fundamentales de operación de un memristor, diversos estudios experimentales han demostrado que los dispositivos físicos presentan comportamientos que difieren de las predicciones obtenidas mediante modelos ideales [11, 9, 10]. Estas diferencias se encuentran asociadas a procesos de fabricación, defectos estructurales, fluctuaciones térmicas y mecanismos microscópicos involucrados en la formación y ruptura de filamentos conductivos.
Uno de los fenómenos más importantes observados en dispositivos memristivos reales corresponde a la variabilidad estocástica. Este comportamiento se manifiesta mediante diferencias en los parámetros eléctricos de dispositivos nominalmente idénticos y mediante fluctuaciones observadas durante ciclos repetidos de operación.
La variabilidad suele clasificarse en dos categorías principales:

- Variabilidad dispositivo a dispositivo (Device-to-Device, D2D).
- Variabilidad ciclo a ciclo (Cycle-to-Cycle, C2C).

La variabilidad D2D se encuentra asociada a diferencias introducidas durante los procesos de fabricación. Factores como el espesor de las capas activas, la distribución de defectos cristalinos y la concentración de vacancias pueden generar diferencias significativas entre dispositivos fabricados bajo condiciones aparentemente idénticas [10].
Por otra parte, la variabilidad C2C se produce cuando un mismo dispositivo presenta respuestas diferentes durante ciclos consecutivos de escritura y borrado. Este fenómeno está relacionado con la naturaleza probabilística de los procesos de migración iónica y formación de filamentos conductivos [9].
Como consecuencia, parámetros como la resistencia de encendido (R_{ON}), la resistencia de apagado (R_{OFF}), los voltajes de conmutación y la conductancia alcanzada después de cada pulso pueden presentar distribuciones estadísticas en lugar de valores únicos deterministas.
La incorporación de mecanismos estocásticos dentro de los modelos memristivos resulta particularmente importante en aplicaciones neuromórficas. A diferencia de sistemas digitales convencionales, donde la variabilidad suele considerarse una fuente de error, diversos estudios han demostrado que ciertos niveles de aleatoriedad pueden contribuir positivamente a procesos de aprendizaje y adaptación inspirados biológicamente [16].
Por esta razón, los simuladores modernos de sistemas neuromórficos incorporan modelos probabilísticos capaces de representar fenómenos de variabilidad observados experimentalmente, permitiendo aproximar con mayor precisión el comportamiento de dispositivos físicos reales [11, 9, 10].

2.2.4. Neurona Leaky Integrate-and-Fire

Las neuronas constituyen la unidad básica de procesamiento dentro del sistema nervioso biológico. Su función principal consiste en integrar señales provenientes de múltiples sinapsis y generar impulsos eléctricos cuando la actividad acumulada supera un determinado umbral. Dentro de los diferentes modelos neuronales existentes, la neurona Leaky Integrate-and-Fire (LIF) representa una de las aproximaciones más utilizadas debido a su simplicidad matemática y capacidad para reproducir los mecanismos fundamentales de integración temporal observados en neuronas biológicas. Su formulación constituye una aproximación clásica en el modelado de neuronas de disparo [20].
El modelo LIF considera que el potencial de membrana evoluciona de acuerdo con la ecuación:

$$\tau_m \frac{dV_m}{dt} = -(V_m - V_{rest}) + RI(t) \tag{2-4}$$

donde:

- V_m representa el potencial de membrana.
- V_{rest} corresponde al potencial de reposo.
- τ_m es la constante de tiempo de la membrana.
- R representa la resistencia equivalente.
- $I(t)$ corresponde a la corriente de entrada.

La neurona integra continuamente la corriente recibida desde sus entradas. Cuando el potencial de membrana alcanza un valor umbral previamente definido, la neurona genera un pulso de salida o *spike*.
Posteriormente, el potencial de membrana es reiniciado a un valor de reposo, iniciando nuevamente el proceso de integración.
La simplicidad computacional del modelo LIF ha favorecido su utilización en sistemas neuromórficos y simuladores de redes neuronales de gran escala. Asimismo, su comportamiento resulta adecuado para estudiar mecanismos de plasticidad sináptica basados en el tiempo relativo de ocurrencia de eventos neuronales.

2.2.5. Plasticidad Sináptica

La capacidad de aprendizaje del sistema nervioso biológico se encuentra estrechamente relacionada con la plasticidad sináptica, proceso mediante el cual la intensidad de conexión entre neuronas puede modificarse en respuesta a diferentes patrones de actividad neuronal. Este mecanismo permite que las redes neuronales adapten su comportamiento a partir

de experiencias previas, constituyendo una de las bases fundamentales del aprendizaje y la memoria.

En sistemas neuromórficos, los memristores resultan particularmente adecuados para representar sinapsis artificiales debido a su capacidad para modificar y conservar estados de conductancia de manera no volátil. Como consecuencia, numerosos trabajos han explorado el uso de dispositivos memristivos para implementar mecanismos de plasticidad inspirados en procesos biológicos [3, 6].

En la literatura se documentan diversos fenómenos de plasticidad, entre los cuales destacan la Potenciación a Largo Plazo (LTP) y la Depresión a Largo Plazo (LTD), que describen cambios progresivos en la eficacia sináptica tras estímulos repetitivos. Estos fenómenos constituyen la base conceptual de múltiples mecanismos de aprendizaje en sistemas neuromórficos.

No obstante, uno de los mecanismos de aprendizaje más importantes y biológicamente relevantes en neurociencia computacional corresponde a la Plasticidad Dependiente del Tiempo de Disparo (Spike-Timing Dependent Plasticity, STDP). Este mecanismo establece que la modificación de una conexión sináptica depende de la diferencia temporal precisa entre los disparos de una neurona presináptica y una neurona postsináptica.

Si la neurona presináptica dispara antes que la neurona postsináptica, la conexión sináptica tiende a fortalecerse. Por el contrario, si la neurona postsináptica dispara antes que la neurona presináptica, la conexión tiende a debilitarse.

Una ventana STDP exponencial ideal puede expresarse como referencia conceptual, siguiendo los estudios de plasticidad dependiente del tiempo y dispositivos memristivos [21, 22]:

$$\Delta w = \begin{cases} A_+ e^{-\Delta t/\tau_+}, & \Delta t > 0 \\ -A_- e^{\Delta t/\tau_-}, & \Delta t < 0 \end{cases} \tag{2-5}$$

donde:

- Δw representa la variación del peso sináptico.
- Δt corresponde a la diferencia temporal entre disparos.
- A_+ y A_- definen las amplitudes de potenciación y depresión.
- τ_+ y τ_- representan constantes temporales del proceso.

Con $w_0 = 0$, la regla STDP implementada en `neurolab.synapses.stdp` normaliza el cambio porcentual de conductancia:

$$\Delta G_{ij} = G_{ij} \frac{p_{\text{máx}}}{100} f(\Delta t), \qquad f(\Delta t) = \begin{cases} e^{-\Delta t/\tau_+}, & \Delta t > 0, \\ -e^{\Delta t/\tau_-}, & \Delta t < 0, \\ 0, & \Delta t = 0, \end{cases} \tag{2-6}$$

El valor $p_{\text{máx}} = 0{,}5\,\%$ por evento corresponde al parámetro `max_percent_change` de `STDPRule`. Esta normalización preserva la forma exponencial y limita el cambio respecto de la conductancia actual; la forma ideal en Δw anterior se conserva como referencia conceptual.
Los valores digitalizados atribuidos a Jo et al. en `jo2010_stdp.csv` presentan asimetría: los cambios observados incluyen $-18{,}48\,\%$ a $\Delta t = -8{,}64$ ms y $+12{,}63\,\%$ a $\Delta t = 9{,}01$ ms. Un ajuste exponencial de mínimos cuadrados por separado a los puntos de cada signo da los parámetros de referencia de la Tabla **2-2**. Este ajuste a puntos digitalizados no se interpreta como una medición adicional ni como una parametrización que el agente utilice.

Tabla **2-2**: Parámetros de la ventana STDP: ajuste a la serie atribuida a Jo et al. y configuración del módulo local.

Parámetro	**Ajuste a serie Jo et al.**	**Módulo `STDPRule`**
A_+	$16{,}12\,\%$	0,05
$\lvert A_-\rvert$ (magnitud)	$25{,}58\,\%$	$A_- = -0{,}025$
τ_+	40,50 ms	17 ms
τ_-	23,55 ms	34 ms
$p_{\text{máx}}$	–	$0{,}5\,\%$ por evento

El ajuste se realizó sobre la magnitud de cada rama, con $|\Delta w| = |A_\pm| \exp(-|\Delta t|/\tau_\pm)$; por ello $|A_-| > |A_+|$ en la serie digitalizada. Los parámetros del módulo son valores normalizados de configuración (adimensionales), donde A_- se guarda con signo negativo, y no se sustituyen por el ajuste de referencia. En `STDPRule`, A_+ y A_- normalizan la fracción aplicada a la conductancia; con $w_0 = 0$ se cancelan en esa operación. Por tanto, el coeficiente porcentual máximo es $p_{\text{máx}}$ para ambas ramas y la asimetría de la respuesta de conductancia procede de las constantes temporales, no de los valores configurados de $A_\pm$. El ajuste no lineal de mínimos cuadrados presentó RMSE de 1,35 puntos porcentuales para la rama de potenciación y 1,04 puntos porcentuales para la rama de depresión. Debido a su capacidad para modificar gradualmente la conductancia en función de estímulos eléctricos, los memristores constituyen una plataforma adecuada para implementar mecanismos STDP [3, 6]. El módulo STDP se conserva para el análisis de plasticidad del sustrato; la regla REINFORCE con baseline del agente se describe por separado en el Capítulo 3.

2.2.6. Arquitecturas Crossbar Memristivas

Uno de los principales desafíos en la implementación de sistemas neuromórficos consiste en conectar un gran número de neuronas y sinapsis de forma eficiente. Para abordar este problema, los memristores suelen organizarse en estructuras matriciales conocidas como arreglos

crossbar, las cuales permiten implementar redes neuronales de alta densidad mediante una disposición regular de filas y columnas conductoras [7, 8].
Un arreglo crossbar está formado por un conjunto de líneas horizontales y verticales que se intersectan entre sí. En cada punto de intersección se ubica un dispositivo memristivo cuya conductancia representa el peso sináptico asociado a una conexión neuronal. Esta organización permite almacenar grandes cantidades de información dentro de un área física reducida y facilita la implementación de operaciones de computación en memoria.
La principal ventaja de las arquitecturas crossbar radica en su capacidad para realizar operaciones de multiplicación matriz-vector directamente sobre el hardware. Esta característica resulta especialmente relevante para redes neuronales artificiales, ya que gran parte de los procesos de inferencia y aprendizaje pueden expresarse mediante este tipo de operaciones matemáticas.
Considerando un vector de entrada representado por tensiones aplicadas a las filas del arreglo, la corriente obtenida en cada columna puede expresarse mediante la Ley de Ohm y la Ley de Corrientes de Kirchhoff como:

$$I_j = \sum_{i=1}^{N} G_{ij} V_i \tag{2-7}$$

Con la convención de este trabajo, $\mathbf{G} \in \mathbb{R}^{N \times M}$ tiene una fila por entrada y una columna por salida, mientras que $\mathbf{V} \in \mathbb{R}^N$ contiene los voltajes aplicados a las filas. Por tanto, el vector de corrientes de columna es $\mathbf{I} = \mathbf{G}^\mathsf{T}\mathbf{V} \in \mathbb{R}^M$.
donde:

- I_j representa la corriente de salida en la columna j.
- V_i corresponde a la tensión aplicada en la fila i.
- G_{ij} representa la conductancia del memristor ubicado en la intersección (i, j).

Esta ecuación muestra que el arreglo crossbar realiza de manera natural una multiplicación matriz-vector, donde la matriz de conductancias actúa como conjunto de pesos sinápticos de la red neuronal.
Gracias a esta propiedad, los arreglos crossbar han sido ampliamente estudiados como una alternativa para implementar computación en memoria, reduciendo significativamente el movimiento de datos entre memoria y procesador característico de las arquitecturas tradicionales [8]. Asimismo, su elevada densidad de integración los convierte en una de las estructuras más prometedoras para el desarrollo de hardware neuromórfico escalable.
En el contexto de esta investigación, se analizan arreglos crossbar simplificados para evaluar el comportamiento colectivo de múltiples dispositivos memristivos y su integración con neuronas LIF. El protocolo de lectura del agente resuelve mediante MNA la red eléctrica incluida en su topología y considera resistencias de línea y de sensado, como se detalla en

la siguiente sección. Esto no equivale a simular todas las configuraciones de *sneak paths*, en particular la prueba clásica con líneas no seleccionadas flotantes. El análisis de escalabilidad a arreglos de alta densidad, selectores y margen de ruido se considera una extensión para trabajos futuros [12].

2.2.7. Sneak Paths y Margen de Ruido

A medida que aumenta el tamaño de los arreglos crossbar memristivos, aparecen fenómenos eléctricos que no están presentes en estructuras de pequeña escala. Uno de los más importantes corresponde a las corrientes parásitas conocidas como *Sneak Paths*, las cuales constituyen uno de los principales obstáculos para la implementación de sistemas memristivos de alta densidad [12].

Un Sneak Path aparece cuando la corriente eléctrica encuentra rutas alternativas distintas de la trayectoria prevista durante operaciones de lectura o escritura. Debido a la naturaleza altamente interconectada de los arreglos crossbar, la corriente puede propagarse a través de múltiples dispositivos memristivos, generando contribuciones no deseadas que alteran el valor medido en la celda objetivo.

La presencia de estas corrientes produce errores en la estimación de estados resistivos y puede afectar significativamente la precisión de operaciones realizadas sobre el arreglo. Este problema se vuelve especialmente crítico cuando el número de filas y columnas aumenta, ya que la cantidad de rutas alternativas disponibles crece rápidamente con el tamaño de la matriz [12].

Otro parámetro importante asociado al desempeño de arreglos crossbar corresponde al margen de ruido (*Noise Margin*). Este parámetro permite cuantificar la capacidad del sistema para distinguir correctamente entre diferentes estados lógicos o resistivos en presencia de perturbaciones eléctricas.

Una reducción excesiva del margen de ruido incrementa la probabilidad de errores durante operaciones de lectura y limita la escalabilidad de la arquitectura. Como consecuencia, numerosos estudios han propuesto técnicas orientadas a minimizar los efectos de Sneak Paths mediante modificaciones arquitectónicas, dispositivos selectores o estrategias de polarización especializadas [12].

El análisis de *sneak paths* y margen de ruido resulta fundamental para evaluar la viabilidad de arquitecturas neuromórficas basadas en memristores. En este trabajo, la lectura del crossbar del agente se resuelve mediante análisis nodal modificado (MNA), con resistencias de línea $R_{\text{wire}} = 1\,\text{m}\Omega$ y de sensado $R_{\text{sense}} = 100\,\Omega$. El solver representa las corrientes y caídas de tensión que resultan de esa red modelada; el Experimento 10 evalúa el acoplamiento entre columnas bajo la excitación indicada. No se implementó la configuración clásica de lectura con una celda seleccionada y las demás líneas flotantes, ni la dinámica de programación/escritura con rutas parásitas. Esos casos y la caracterización física de hardware quedan fuera del alcance.

2.2.8. Parámetros del modelo de referencia

La Tabla **2-3** reúne los valores de la configuración Strukov–Biolek y del agente CartPole usada como referencia. Los valores de lectura y aprendizaje corresponden al protocolo del agente; los parámetros eléctricos y LIF se convierten a unidades SI desde los archivos JSON.

Tabla **2-3**: Parámetros del dispositivo, crossbar y agente de referencia.

Símbolo	**Valor**	**Unidad**	**Fuente**	**Uso**
R_{ON}	100	Ω	`strukov_ideal.json`	Strukov y agente
R_{OFF}	16 000	Ω	`strukov_ideal.json`	Strukov y agente
D	10	nm	`strukov_ideal.json`	Modelo Strukov
μ_v	10^{-14}	$\mathrm{m}^2/(\mathrm{V\,s})$	`strukov_ideal.json`	Modelo Strukov
x_0	0,1	adimensional	`strukov_ideal.json`	Estado físico inicial; $X_0 = 0{,}1$ en el agente
X_0	0,1	adimensional	`run_robot.py`	Estado inicial del agente; equivale a $x_0 = 0{,}9$
p (Biolek)	5	adimensional	`strukov_ideal.json`	Ventana de frontera
V_{th} (Strukov)	0,1	V	`StrukovConfig`, `StrukovMathModel`	Umbral predeterminado del modelo; no medido fisicamente
V_{read}	0,05	V	`run_robot.py`	Rail de lectura; ruta MNA sin actualizacion de X
R_{wire}	10^{-3}	Ω	`crossbar_brain.py`	Solver MNA del agente
R_{sense}	100	Ω	`crossbar_brain.py`	Solver MNA del agente
C_m	100	nF	`lif_config.json`	Neurona LIF
R_{series}	100	kΩ	`lif_config.json`	Neurona LIF
R_{leak}	1	MΩ	`lif_config.json`	Neurona LIF
$V_{\mathrm{th,LIF}}$	1,0	V	`lif_config.json`	Umbral LIF
t_{ref}	2×10^{-3}	s	`lif_config.json`, `lab_bridge.py`	LIF (el JSON almacena 2 ms y el puente lo convierte a segundos)
η	0,005	adimensional	`run_robot.py`	Tasa de aprendizaje REINFORCE
T_{init}	1,0 o 0,05	adimensional	`run_robot.py`	Temperatura inicial seleccionable
$T_{\mathrm{mín}}$	0,05	adimensional	`run_robot.py`	Temperatura mínima
T_{decay}	0,995	adimensional	`run_robot.py`	Decaimiento de temperatura por episodio
W_b	200	episodios	`run_robot.py`	Ventana del baseline móvil

CAPÍTULO 3

Marco Práctico

El presente capítulo describe la implementación y validación del simulador neuromórfico desarrollado. Primero se presentan la arquitectura general del sistema y las herramientas empleadas; después, la metodología seguida, desde el modelado del dispositivo memristivo hasta su integración en un agente de control autónomo. Finalmente, se exponen los resultados de las simulaciones y se estima el costo de reproducirlas.
Las Etapas 1–4 del desarrollo del sustrato (modelo memristivo y variabilidad, neurona LIF, plasticidad sináptica y crossbar) se documentan en ese orden en el Apéndice D. Este capítulo resume su integración en el agente y presenta las aplicaciones evaluadas, sin asignarles números de etapa adicionales.

3.1. Esquema general del proyecto

La solución desarrollada se estructuró como un ecosistema computacional compuesto por dos subsistemas acoplados. El primero, denominado *Neuromorphic Lab*, constituye el motor de simulación encargado de modelar dispositivos memristivos, neuronas tipo Leaky Integrate-and-Fire y arreglos crossbar con efectos no ideales. El segundo, denominado *Robot Sim*, implementa un agente de control autónomo que utiliza el arreglo crossbar simulado como sustrato de pesos de su política.
La Figura **3-1** presenta la organización general de ambos subsistemas y su relación unidireccional: el simulador robótico consume los modelos físicos del laboratorio a través de una capa de abstracción (*lab_bridge*), pero el laboratorio no conoce la existencia del agente.



Figura **3-1**: Arquitectura general del ecosistema. El laboratorio (*arriba*) provee los modelos físicos; el simulador robótico (*abajo*) los integra en un agente de control.

El flujo de información del agente sigue la secuencia *sensores* → *codificador* → *crossbar* → *decodificador* → *actuadores*, con el aprendizaje aplicado al final de cada episodio mediante una regla de plasticidad modulada por recompensa. El detalle de cada componente se desarrolla en la Sección 3.3.

3.2. Herramientas

El proyecto se desarrolló con herramientas de código abierto, sin dependencias de software propietario. La Tabla **3-1** y la Tabla **3-2** listan los componentes principales utilizados.

3.2.1. Hardware

Tabla **3-1**: Herramientas de hardware utilizadas.

Componente	**Uso**
Laptop de desarrollo	Ejecución de simulaciones y scripts Python (CPU x86-64, 8 GB RAM, SSD)

3.2.2. Software

Tabla **3-2**: Herramientas de software utilizadas.

Herramienta	**Uso**
Python 3.10+	Lenguaje principal del simulador
NumPy	Álgebra lineal y matrices del crossbar
Matplotlib	Generación de figuras y visualización
PySide6	Interfaces gráficas del laboratorio neuromórfico
Tkinter	Diálogos de configuración de las aplicaciones
Git	Control de versiones

3.3. Metodología del agente neuromórfico

El agente recibe el estado del entorno, lo codifica en entradas del crossbar, lee sus corrientes de columna y convierte la salida neuronal en una acción. El arreglo constituye el sustrato de pesos; el entorno y la recompensa se simulan en software. El desarrollo detallado de los dispositivos, la LIF, la plasticidad y el solver se documenta en el Apéndice D.

3.3.1. Configuración de referencia

CartPole usa un crossbar de 24×12, lectura a 0,05 V y una neurona LIF por salida. El estado normalizado X se convierte en resistencia mediante $R(X) = R_{\text{on}} + X(R_{\text{off}} - R_{\text{on}})$; por ello, X es un estado interno del modelo y no una conductancia. La configuración nominal es $R_{\text{on}} = 100\,\Omega$, $R_{\text{off}} = 16\,\text{k}\Omega$, $X_0 = 0{,}1$ y límites $0 \leq X \leq 1$. La tabla resume los parámetros eléctricos y temporales usados como referencia.

Tabla **3-3**: Parámetros de referencia del agente CartPole.

Parámetro	Configuración	Valor efectivo
R_{on}	$100\,\Omega$	$100\,\Omega$
R_{off}	$16\,\text{k}\Omega$	$16\,000\,\Omega$
Estado inicial	$X_0 = 0{,}1$	$x_0 = 0{,}9$ bajo $X = 1 - x$
V_{read}	$0{,}05\,\text{V}$	$0{,}05\,\text{V}$
V_{th} LIF	$1{,}0\,\text{V}$	$1{,}0\,\text{V}$
t_{ref}	$2\,\text{ms}$	$2 \times 10^{-3}\,\text{s}$
dt	–	$10^{-4}\,\text{s}$

En CartPole, las observaciones se codifican en filas activas; el solver nodal MNA calcula las corrientes considerando la conductancia del crossbar y las resistencias configuradas de línea y sensado. Las neuronas LIF integran las corrientes, generan spikes al alcanzar el umbral y se reinician según el período refractario. La decisión de acción se obtiene de la actividad de salida. Estos pasos describen el simulador, no un circuito fabricado.
La lectura del agente calcula $G(X)$ y resuelve la red MNA sin integrar la dinamica no volatil del estado durante ese paso; el metodo de lectura no llama a la actualizacion de las celdas, por lo que en esta ruta simulada $\Delta X = 0$ por construccion. Esto cuantifica el cambio de estado impuesto por el software en cada lectura, no una perturbacion electrica medida en un dispositivo fisico. El modelo Strukov del laboratorio si incorpora un umbral de deriva ionica de $0{,}1\,\text{V}$ como valor predeterminado de `StrukovConfig`; ese umbral se aplica al integrar la dinamica de la celda y no es una medicion experimental del umbral del dispositivo. El dispositivo volatil de salida, en cambio, si se actualiza en cada paso y modula la corriente que recibe la LIF.

3.3.2. Reglas de aprendizaje y decisión

En REINFORCE, el agente conserva una elegibilidad por conexión durante el episodio y la actualiza al terminarlo según el retorno y un baseline. La contribución instantánea es

$$e_{ij,t} = \text{pre}_{i,t}\left(\mathbf{1}_{j=\text{col}_t} - \pi_{j,t}\right), \qquad E_{ij,t} = \gamma E_{ij,t-1} + e_{ij,t},$$

donde $\pi_{j,t}$ es la probabilidad softmax de la columna j. La softmax convierte puntuaciones en probabilidades que suman uno: una temperatura alta reparte más la probabilidad y una baja la concentra. El decoder muestrea según esa distribución y la regla de aprendizaje ajusta el crossbar con el resultado del episodio. La implementación se describe en `robot_sim/neurobot/agent.py` y `robot_sim/neurobot/crossbar_brain.py`.

La regla local STDP del laboratorio usa spikes binarios pre/post y trazas exponenciales p_i y q_j:

$$\tau_{\text{pre}}\dot{p}_i = -p_i + s_i^{\text{pre}}, \qquad \tau_{\text{post}}\dot{q}_j = -q_j + s_j^{\text{post}}, \qquad \Delta G_{ij}^{\text{STDP}} = \eta\left(A_+ p_i s_j^{\text{post}} - A_- s_i^{\text{pre}} q_j\right).$$

En la regla R-STDP del módulo, la elegibilidad decae y acumula ese cambio:

$$E_{ij,t} = e^{-dt/\tau_e} E_{ij,t-1} + \Delta G_{ij,t}^{\text{STDP}}, \qquad \Delta G_{ij,t}^{\text{R-STDP}} = R_t E_{ij,t}.$$

El módulo correspondiente es `neurolab/crossbar/plasticity.py`.
Las corridas del agente etiquetadas R-STDP en el Apéndice E no implementan esa regla estricta con spikes LIF como evento post. En los entornos de control, el runner construye un evento de acción centrado por la política, $z_{j,t} = \mathbf{1}_{j=a_t} - \pi_{j,t}$, y una elegibilidad por decisión $e_{ij,t} = a_+ \,\text{pre}_{i,t}\,\text{máx}(z_{j,t}, 0) + a_- \,\text{pre}_{i,t}\,\text{mín}(z_{j,t}, 0)$. Los spikes LIF y los ganadores WTA se registran, pero no sustituyen a $z_{j,t}$ en esta actualización; la tarea de visión usa una elegibilidad derivada del gradiente de corriente. Para cada paso se calcula el retorno descontado $G_t = r_t + \gamma G_{t+1}$, la ventaja $A_t = G_t - b_t$ respecto al baseline temporal previo y su normalización $\widehat{A}_t = A_t / s_A$, donde $s_A = \text{máx}(\text{std}(A), 10^{-8})$ (se usa $s_A = 1$ para un solo paso). La elegibilidad modulada del episodio es

$$E_{ij}^R = \frac{1}{\sqrt{T}} \sum_{t=0}^{T-1} e_{ij,t}\widehat{A}_t, \qquad \Delta X = -\eta E^R.$$

El cambio se escala para cumplir el límite configurado de $\text{máx}_{ij}\,|\Delta X_{ij}|$ y se anula en episodios que alcanzan el umbral de congelamiento. La actualización opera directamente sobre el estado abstracto X y las variabilidades D2D/C2C modifican el incremento aplicado. Para $E^R > 0$, $\Delta X < 0$ reduce $R(X)$ y aumenta $G(X)$; el signo opuesto produce el efecto contrario. Esta comprobación algebraica confirma la dirección del mapeo en el código, pero no demuestra aprendizaje estable ni programación por pulsos físicos. Por ello, en esta tesis las corridas del runner se describen como el piloto híbrido pre/acción modulado por retorno (conservando R-STDP como nombre del modo del software), no como evidencia de R-STDP canónica pre/post. La variante de tarea que sí empleó spikes LIF reales como evento post tuvo bajo rendimiento exploratorio. Los resultados y su alcance se informan en la Sección 3.5.6; la topología y el modelo eléctrico del crossbar se mantienen entre reglas de aprendizaje.

3.3.3. Aplicaciones evaluadas

La misma organización se adaptó a cuatro tareas. Se conservaron la lectura del crossbar y la actualización por recompensa; las dimensiones y el significado de la salida dependen del entorno. Los registros por condición se incluyen en el Apéndice E.

Tabla **3-4**: Configuración de las aplicaciones simuladas.

Tarea	Crossbar	Entrada y salida	Métrica
CartPole	24×12	Estado continuo; dos acciones	Recompensa y supervivencia
Visión	64×8	Patrón binario 8×8; ocho clases	Exactitud
Obstáculos	16×4	Sensores del gridworld; cuatro movimientos	Llegada a meta
Auto 2D	24×8	Rayos, dirección y distancia; cuatro controles	Llegada a meta

3.4. Validación de modelos

La validación se organizó en tres niveles: comprobaciones internas de código y unidades, comparación de series simuladas con puntos digitalizados de publicaciones y pruebas funcionales de las aplicaciones. Solo se presentan métricas cuando existe correspondencia punto a punto y protocolo definido. Una coincidencia visual, una distribución sintética o una traza producida por el mismo modelo no equivalen a una medición de hardware. El inventario de fuentes y scripts está en el apéndice de datos; los protocolos completos se conservan en el Apéndice D.

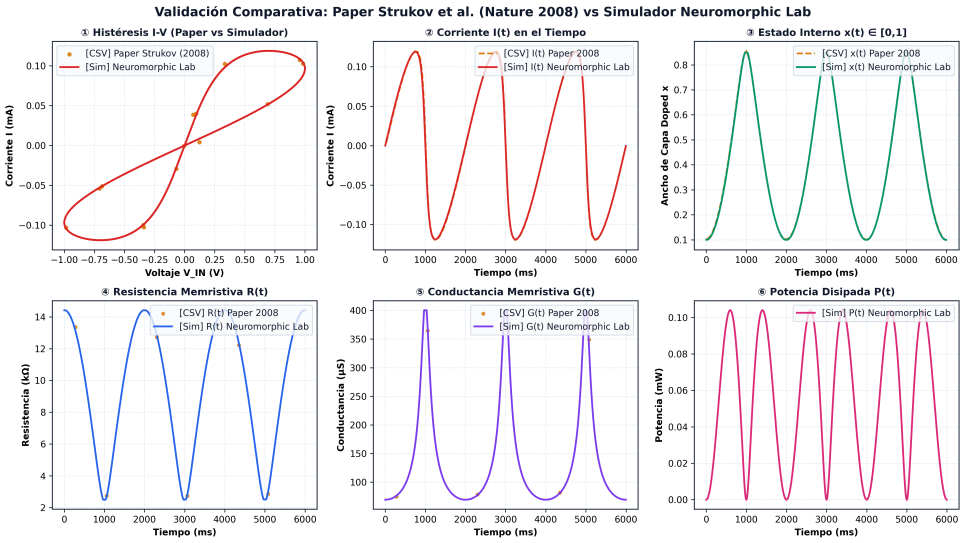


Figura **3-2**: Curvas del modelo Strukov local comparadas con las series digitalizadas atribuidas a la referencia.

3.4.1. Comparaciones cuantitativas

Para Strukov se compararon estado, corriente, resistencia y conductancia con series digitalizadas atribuidas al artículo de 2008. Prezioso es una comparación visual, pues los puntos no contienen protocolo temporal completo. La ventana STDP usa nueve puntos de ajuste y cuatro reservados; LTP/LTD usa 18 puntos de test. La LIF $f - I$ compara Euler con la solución analítica del mismo modelo, no con mediciones experimentales. La tabla resume las métricas reproducibles.

Tabla **3-5**: Métricas de validación reproducibles. R^2 es el coeficiente de determinación; ERP se normaliza por el rango de referencia.

Comparación	n	R^2	MAE	RMSE	ERP (%)
Strukov, estado $x(t)$	107	0.999657	0.003676	0.004491	1.27
Strukov, corriente $I(t)$	136	0.996994	0.002586 mA	0.004126 mA	6.22
Strukov, resistencia $R(t)$	87	0.998319	0.135431 kΩ	0.162264 kΩ	3.54
Strukov, conductancia $G(t)$	87	0.983726	7.55759 μS	15.64415 μS	19.04
Prezioso, $I(V)$ visual	179	–	–	–	–
STDP, holdout	4	0.94220	1.90404 pp	2.13626 pp	14.87
LTP/LTD, test	18	0.73988	8.3491 nA	8.6181 nA	31.23
LIF $f - I$, Euler vs analítica	14	0.999964	0.086431 Hz	0.151393 Hz	0.55

STDP y LTP/LTD proceden de conjuntos digitalizados pequeños; sus parámetros ajustados son efectivos y no caracterizan independientemente un dispositivo. En LTP/LTD, el resultado agregado oculta un ajuste deficiente por rama. Prezioso carece de etiquetas temporales y de rama suficientes para una métrica. La prueba D2D/C2C verifica propiedades estadísticas de muestras sintéticas, no una población experimental. Las Figuras **3-3** y **3-4** permiten inspeccionar los datos atribuidos a Jo et al.; las particiones y limitaciones se amplían en el apéndice técnico.

Tabla **3-6**: Cumplimiento de los umbrales cuantitativos cuando son aplicables.

Prueba	ERP $< 10\,\%$	$R^2 > 0{,}90$	Conclusión
Strukov: x, I, R	Sí	Sí	Cumplen en las series comparadas.
Strukov: G	No	Sí	ERP $= 19{,}04\,\%$; no cumple todos los umbrales.
STDP, holdout ($n = 4$)	No	Sí	Parcial; ERP $= 14{,}87\,\%$ y muestra pequeña.
LTP/LTD ($n = 18$)	No	No	No cumple; $R^2 = 0{,}73988$, ERP $= 31{,}23\,\%$.
Prezioso $I(V)$	No calculable	No calculable	Comparación visual; faltan puntos con protocolo suficiente.
LIF $f - I$	Sí*	Sí*	Verificación interna frente a solución analítica, no validación experimental independiente.

*Métricas de integración para el mismo modelo analítico; no se interpretan como concordancia con hardware publicado.

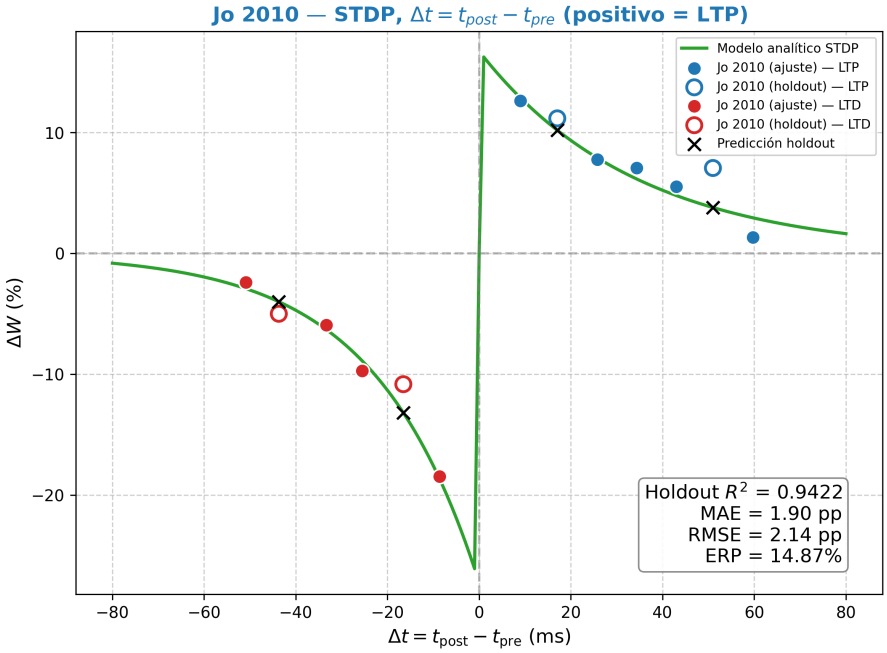


Figura **3-3**: Ajuste STDP en nueve puntos y evaluación en cuatro puntos reservados atribuidos a Jo et al. (2010).

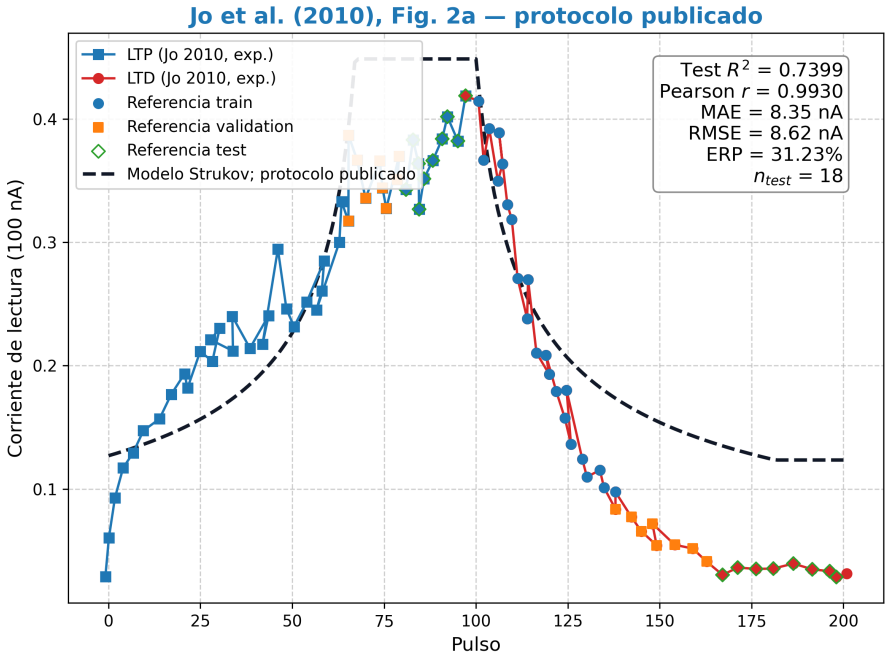


Figura **3-4**: Comparación LTP/LTD atribuida a Jo et al. (2010); los parámetros se ajustaron con datos de entrenamiento.

Tabla **3-7**: Cumplimiento de objetivos según el alcance de la evidencia.

Objetivo	Evidencia resumida	Estado
Modelos memristivos y variabilidad	Comparaciones digitalizadas y pruebas sintéticas; alcance acotado.	Parcial
Neurona LIF	Disparo, reinicio, refractario y curva $f - I$ frente a solución analítica.	Verificado internamente
Plasticidad local	STDP con cuatro puntos reservados; conjunto pequeño.	Parcial
Crossbar	Solver MNA y comprobación numérica; sin medición física.	Verificado en simulación
Aprendizaje de tareas	REINFORCE de referencia y piloto R-STDP con resultados variables.	Parcial

3.4.2. Verificación del sistema integrado

Se verificó la conversión de unidades, un caso manual de crossbar 2×2 y la invariancia de pesos durante evaluaciones congeladas. La respuesta LIF se contrastó con la solución analítica frecuencia–corriente; la prueba temporal comprueba umbral, reinicio y refractario en el propio modelo. El Apéndice técnico distingue estas verificaciones de una validación física.

La prueba manual usa $G = [[0{,}001, 0{,}0005], [0{,}00033333, 0{,}00025]]\,\mathrm{S}$ y $V = [0{,}05, 0]^{\mathsf{T}}\,\mathrm{V}$. La multiplicación ideal produce $[50, 25]$ μA; el solver MNA produce $[44{,}1176, 23{,}2558]$ μA. La diferencia corresponde a las resistencias del protocolo del solver, no a la identidad matricial ideal.

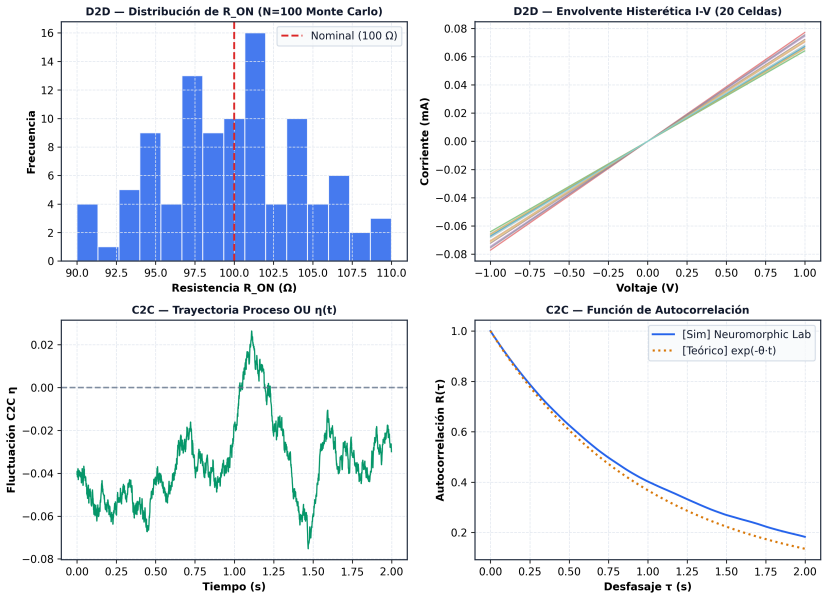


Figura **3-5**: Distribuciones sintéticas D2D/C2C del modelo y sus referencias teóricas; no son histogramas de mediciones físicas.

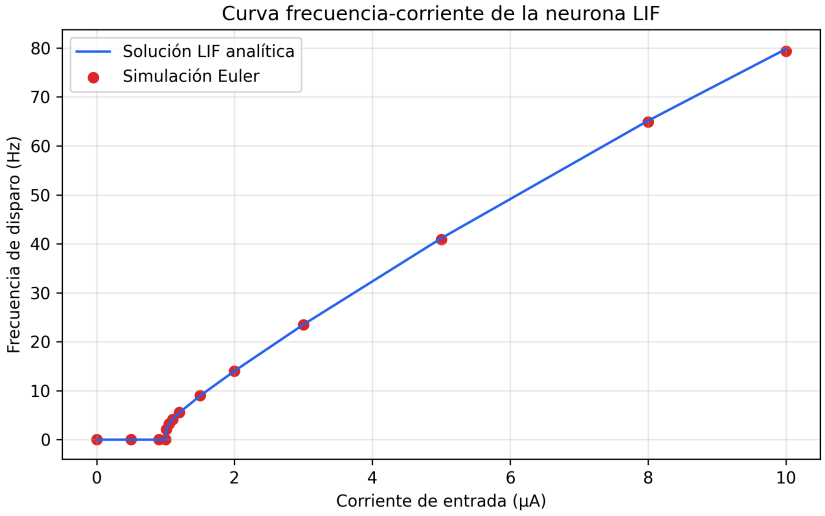


Figura **3-6**: Frecuencia de disparo LIF por Euler frente a la solución analítica del mismo modelo.

3.5. Resultados y discusión

Las métricas de cada aplicación son específicas y no se interpretan como comparación controlada cuando cambian la arquitectura, la recompensa o el presupuesto de entrenamiento. Los registros y matrices por condición están en el Apéndice E.

Tabla **3-8**: Separación de entrenamiento, validación y test en las aplicaciones.

Tarea	Entrenamiento	Validación y selección	Test independiente
CartPole	Cinco semillas, 300 episodios cada una.	50 episodios por checkpoint; se selecciona semilla 4 (493,77).	50 episodios, semillas 20000–20049; $493{,}63 \pm 0{,}93$.
Visión	Tres semillas, 5000 episodios sobre ocho plantillas.	160 variantes por checkpoint con inversión de píxel $p = 0{,}05$; selección: semilla 2 (42,5 %).	400 variantes nuevas; exactitud 42,5 %.
Obstáculos	Semillas 0–2; 746, 800 y 800 episodios.	20 mapas por checkpoint; selección: semilla 2 (25 %).	50 mapas nuevos; éxito 4 % (2/50).
Auto 2D	Una semilla; horizonte reportado de 3000 episodios y snapshots cada 200.	20 escenarios por snapshot; checkpoint reportado con 15 %.	50 escenarios; éxito reportado 14 % (7/50); faltan logs para recalcular.

3.5.1. CartPole-v1: control con recompensa densa

Se entrenó el agente 24×12 durante 300 episodios en cinco semillas, con D2D/C2C y supresión de actualización en cada episodio cuya recompensa alcanzó 430. La recompensa media de los últimos 200 episodios fue 369.66, 421.44, 474.31, 462.79 y 448.36 para las semillas 0–4 (media 435,31, desviación estándar 37,28). Cuatro corridas alcanzaron una recompensa

episódica de 480. El umbral omite una actualización de pesos de ese episodio, pero no congela permanentemente el agente.
Para evaluar dentro de la tarea, se seleccionó en validación el checkpoint de la semilla 4 y se evaluó sin aprendizaje en 50 episodios de test independientes. Obtuvo $493{,}63 \pm 0{,}93$ puntos (rango 490.43–494.94); X no cambió y el decoder fue greedy. En la comparación emparejada ideal–D2D/C2C, la diferencia media fue $+99{,}46 \pm 226{,}90$ puntos, pero solo tres de cinco pares favorecieron la condición variable; el resultado es descriptivo y no prueba una ventaja general. La curva de cinco semillas está en la Figura **3-7**.

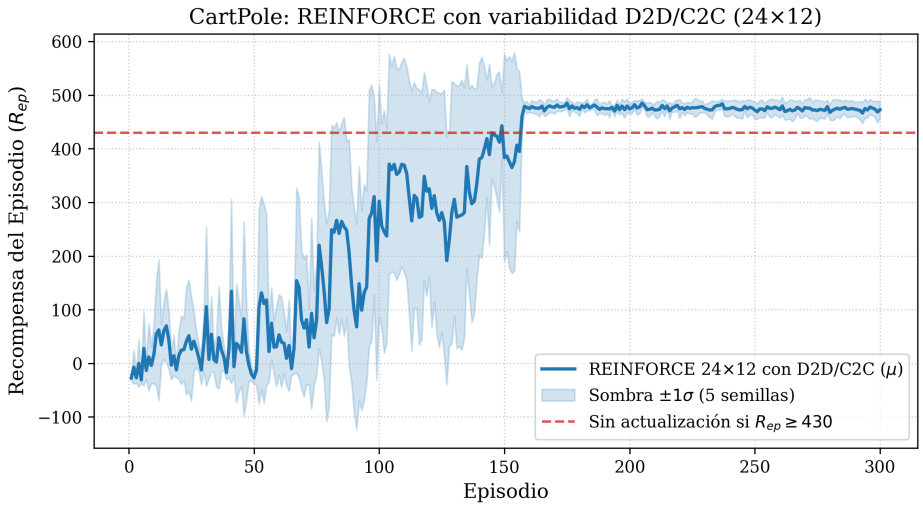


Figura **3-7**: Recompensa de cinco corridas CartPole con D2D/C2C, suavizada para facilitar la lectura.

3.5.2. Visión 8×8: clasificación supervisada

La tarea clasificó ocho patrones binarios mediante un crossbar 64×8. En tres corridas de 5000 episodios, el acierto de los últimos 200 episodios fue 100 %. Ese dato corresponde a las ocho plantillas del entrenamiento. Para medir robustez se reservaron variantes con inversión de píxeles; tras selección por validación, la semilla 2 obtuvo 42.5 % sobre 400 variantes de test, frente a 12.5 % aleatorio. El test mide ruido sintético sobre las plantillas, no clases ni imágenes externas.

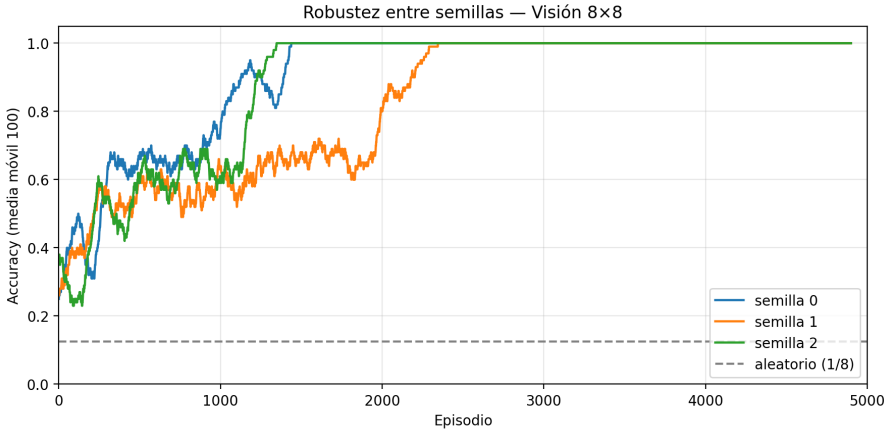


Figura **3-8**: Exactitud de tres corridas de visión, suavizada con una ventana de 100 episodios.

3.5.3. Navegación 10×10: control con recompensa escasa

El gridworld contiene 15 obstáculos y cuatro movimientos, con recompensa por llegar a la meta y penalizaciones por paso y colisión. Las tasas de éxito de los últimos 200 episodios fueron 100 %, 49.5 % y 82 % (media no ponderada 77.17 %). En 50 mapas nuevos, el checkpoint seleccionado en validación alcanzó la meta en 2 mapas (4 %). El buen resultado en el mapa de entrenamiento no se transfirió a mapas distintos en esta prueba. Los registros de esa evaluación no están incluidos como JSON recalculable y el 4 % se identifica como resultado reportado.

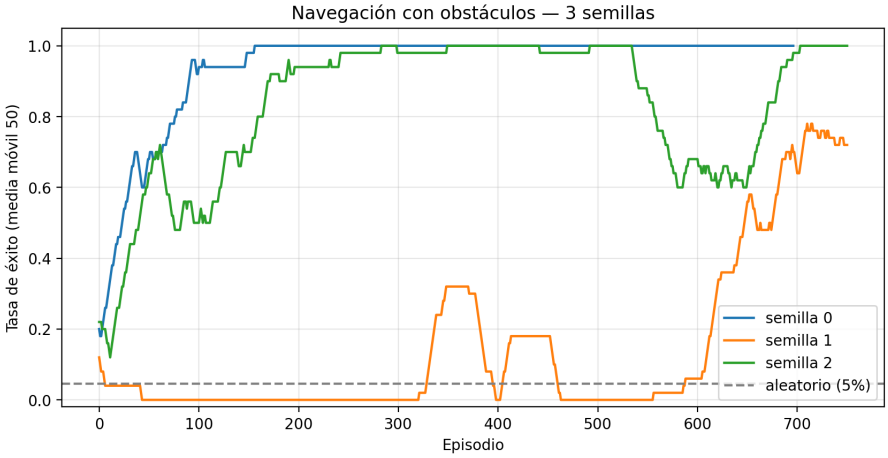


Figura **3-9**: Tasa de éxito de tres corridas del gridworld, suavizada con una ventana de 50 episodios.

3.5.4. Auto 2D: entorno continuo de conducción

El entorno combina ocho sensores de distancia, dirección y cuatro controles, con crossbar 24×8. El registro disponible indica 5.0 % de éxito en los últimos 200 episodios; una evaluación reportada obtuvo 14 % (7/50) en test. Solo existe una semilla y faltan los archivos necesarios para reconstruir la curva y el checkpoint evaluado. Se considera un resultado piloto, no evidencia de robustez entre semillas.

3.5.5. Análisis comparativo entre tareas

La Tabla **3-9** resume los últimos 200 episodios de los registros de presupuesto fijo. Recompensa, exactitud y tasa de llegada son magnitudes diferentes; además, las tareas usan matrices y protocolos distintos. La comparación es descriptiva y no estima un efecto causal de la arquitectura ni de la regla de aprendizaje.

Tabla **3-9**: Resumen descriptivo de las corridas de presupuesto fijo.

Tarea	Episodios	Semillas	Métrica final	$\bar{X}$	σ_X
CartPole	300	5	$\bar{R}_{200} = 435{,}31$	0.1379	0.0717
Visión 8×8	5000	3	$\mathrm{Acc}_{200} = 100\,\%$	$\approx 0{,}10$	0.022–0.030
Navegación	746–800	3	$S_{200} = 77{,}17\,\%$	0.221–0.345	0.190–0.288
Auto 2D	1000	1	$S_{200} = 5{,}0\,\%$	0.319	0.274

3.5.6. Piloto comparativo REINFORCE/R-STDP

El lote de parada por éxito evaluó CartPole, visión y obstáculos con ambos algoritmos, semillas 0 y 1 e inicializaciones $X_0 = 0{,}1$ y $X_0 = 0{,}9$. Car se ejecutó solo con R-STDP. El criterio fue al menos 271 éxitos dentro de los últimos 300 episodios (90.33 %), con máximo de 10 000 episodios; no exige 300 éxitos consecutivos. Cada tarea conserva su tamaño de crossbar y definición de éxito. La tabla resume los registros del piloto.

Tarea	Regla	Criterio	Episodios / alcance
CartPole	REINFORCE	2/4	349 y 1249; dos registros incompletos a 559 y 1132.
CartPole	R-STDP	4/4	307–1199 episodios.
Visión	REINFORCE	4/4	586–1357 episodios.
Visión	R-STDP	4/4	470–2951 episodios.
Obstáculos	REINFORCE	4/4	300 episodios en las cuatro condiciones.
Obstáculos	R-STDP	4/4	300–336 episodios.
Car, lote principal	R-STDP	1/4	Una condición a 3775; tres a 10 000 sin criterio.
Car, repetición corregida	R-STDP	2/4	Semilla 2: 750 y 1548; semilla 4: ambas a 10 000 sin criterio.

Tabla **3-10**: Resultado del piloto de parada por éxito. Cada fracción corresponde a cuatro condiciones semilla–X_0, salvo donde se indica el alcance incompleto.

R-STDP designa el modo implementado en el runner: elegibilidad local modulada por retorno. La regla STDP local del crossbar sí produce cambios; el piloto muestra que su integración con la tarea todavía es inestable entre condiciones y dominios. No se extrapola a hardware. La discusión del evento post, el baseline y la estabilidad está en el Apéndice D.

3.5.7. Limitaciones

Los resultados son de simulación y no se dispone de un prototipo físico. Las comparaciones con artículos usan datos digitalizados, a veces con pocos puntos o protocolos incompletos; las pruebas sintéticas no sustituyen mediciones. Las aplicaciones no comparten una métrica ni un protocolo idénticos. Visión se evaluó con variantes de plantillas, navegación mostró baja transferencia a mapas nuevos y Auto 2D cuenta con una semilla y archivos incompletos. El piloto R-STDP alcanza el criterio en algunas condiciones, pero no permite concluir robustez general. Estas limitaciones acotan las afirmaciones del trabajo y motivan más semillas, registros íntegros y protocolos previamente fijados.

3.6. Análisis de costos

El costo es una estimación académica de reproducir simulaciones en una computadora disponible; no es un registro de gastos ni de horas efectivas. Se imputan 260 horas a Bs. 20/h y asignaciones de equipo y servicios. No se incluyen computadora nueva ni hardware memristivo físico.

Tabla **3-11**: Estimación académica de trabajo e infraestructura.

Concepto	Base estimada	Importe
Análisis y diseño	30 h × Bs. 20/h	Bs. 600
Desarrollo e implementación	160 h × Bs. 20/h	Bs. 3 200
Pruebas y validación	35 h × Bs. 20/h	Bs. 700
Documentación y tesis	35 h × Bs. 20/h	Bs. 700
Trabajo total	260 h × Bs. 20/h	Bs. 5 200
Uso de equipo disponible	Estimación	Bs. 400
Electricidad e internet	Estimación conjunta	Bs. 200
Total académico referencial		Bs. 5 800

Se consolidaron las categorías de costo en una tabla para evitar repetir los mismos importes en varios resúmenes. Las tarifas son supuestos de planificación, no comprobantes ni consumos medidos.

CAPÍTULO 4

Marco Conclusivo

El presente capítulo resume los resultados obtenidos a lo largo del desarrollo de la plataforma de simulación neuromórfica, formula recomendaciones orientadas a futuras iteraciones del proyecto y delinea las líneas de trabajo futuro que se derivan de las limitaciones identificadas.

4.1. Conclusiones

El trabajo desarrolló `neurolab`, un simulador modular de memristores, neuronas LIF, plasticidad y crossbar, y lo conectó con el agente Robot Sim. La evidencia se limita a simulaciones, verificaciones internas, soluciones analíticas y curvas digitalizadas atribuidas a publicaciones. No constituye una validación general de hardware físico. El estado de cumplimiento de los objetivos, con el alcance de cada evidencia, se resume en la Tabla **3-7**.

1. **Modelo memristivo y variabilidad:** El modelo de Strukov alcanza $R^2 = 0{,}999657$ para $x(t)$ y $R^2 = 0{,}996994$ para $I(t)$ bajo la alineación y escalado especificados en el Capítulo 3; para resistencia y conductancia se obtienen $R^2 = 0{,}998319$ y $R^2 = 0{,}983726$, respectivamente. D2D/C2C están habilitados en las corridas actuales de Robot Sim mediante modificadores por celda. Las pruebas estadísticas reproducen propiedades de distribuciones sintéticas, no variabilidad medida en dispositivos.

2. **Histéresis y estado:** Se implementaron ciclos I–V y evolución acotada de estado. Para Prezioso, se presenta una reconstrucción visual aproximada basada en la Fig. 1(b). El artículo disponible no especifica completamente el barrido temporal y la digitalización no conserva tiempo ni etiquetas de rama; no se calculan métricas de ajuste ni se demuestra la validación I–V por rama. El objetivo se considera parcialmente cumplido.

3. **Neurona LIF:** La implementación integra, dispara, reinicia y respeta el período refractario. La curva $f - I$ se contrastó con la solución analítica del mismo modelo ($R^2 = 0{,}999964$, MAE = 0,086 Hz, RMSE = 0,151 Hz), no con datos experimentales independientes.

4. **Plasticidad:** Se implementó una regla STDP en el módulo de sinapsis; al ajustar nueve puntos y evaluar cuatro reservados, la ventana obtiene $R^2 = 0{,}94220$ y RMSE $= 2{,}136$ puntos porcentuales. El holdout es pequeño. La curva digitalizada de Jo asigna signo positivo a LTP, pero la definición temporal en prosa del artículo usa el signo contrario; el proyecto declara y prueba la convención $\Delta t = t_{\text{post}} - t_{\text{pre}}$. Para LTP/LTD se emplean los pulsos publicados de $+3{,}2$ y $-2{,}8\,\text{V}$, de $300\,\mu\text{s}$, y lectura a $1\,\text{V}$. En los 18 puntos de test se obtuvo $R^2 = 0{,}73988$, Pearson $r = 0{,}99303$, RMSE $= 8{,}618$ nA y ERP $= 31{,}23\,\%$; las métricas por rama son deficientes ($R^2 = -8{,}54$ para LTP y $R^2 = -829{,}99$ para LTD). Los parámetros efectivos se ajustan a la misma curva digitalizada, por lo que no son una caracterización independiente del dispositivo. La evidencia es parcial. Las corridas de Robot Sim resumidas en este documento usan REINFORCE con baseline. El software entregado ofrece también R-STDP en los cuatro runners y sus pruebas funcionales verifican partes de su mecanismo. La carpeta de evidencias incluye un piloto exploratorio de 28 corridas en CartPole, visión y obstáculos (dos semillas y dos valores de X_0), además de cuatro corridas R-STDP de Car bajo un protocolo inicial. Una repetición separada de Car R-STDP con evaluación cada 100 episodios usó semillas 2 y 4. La muestra sigue siendo pequeña, Car REINFORCE no está emparejado en estos lotes y el protocolo Car inicial difiere; no se afirma superioridad estadística ni se generaliza a otras condiciones.

5. **Crossbar e integración:** El solver MNA incluye resistencias de línea y sensado en lectura. En el caso 2×2 la diferencia relativa de norma respecto a la VMM ideal es $10{,}98\,\%$; es una comprobación numérica, no experimental. Robot Sim usa una matriz abstracta X cuyos valores determinan $R(X)$ y $G(X)$. En ambos modos, el memristor volátil de salida modula la corriente que llega a la LIF. En REINFORCE, la acción se decodifica desde las corrientes de columna; en el R-STDP actual también se decodifica desde esa política softmax y WTA no la restringe. La elegibilidad R-STDP usa eventos pre del codificador y la acción decodificada centrada por la softmax; los spikes LIF se registran, pero no actúan como evento post en el piloto cuantitativo del runner. Este piloto usa elegibilidad pre/acción centrada por la política y modulada por retornos descontados; no es la regla canónica de pares pre/post. Se distinguen tres resultados: la STDP local del laboratorio usa spikes pre/post y trazas temporales; una variante de tarea con spikes LIF reales como evento post obtuvo bajo rendimiento exploratorio; y las corridas de la carpeta de evidencias, etiquetadas R-STDP por el modo del software, corresponden al híbrido pre/acción. Estas corridas no demuestran que la R-STDP canónica aprenda las tareas. Las reglas cambian en software, sin alterar la topología ni el modelo eléctrico del crossbar. El bajo desempeño de la variante directa está observado, pero las pruebas no aíslan una causa única.

6. **Validación cuantitativa:** Las métricas se calcularon con procedimientos de alineación y limitaciones documentados sobre curvas digitalizadas atribuidas a publicaciones.

La evidencia es heterogénea: Prezioso cuenta con una reconstrucción visual aproximada sin métricas de ajuste; STDP se evalúa en cuatro puntos reservados y LTP/LTD muestra errores elevados, en particular por rama. Los parámetros LTP/LTD se estiman sobre la misma curva digitalizada. Ninguna comparación representa validación frente a mediciones experimentales propias.

4.2. Respuesta a la pregunta de investigación

La concordancia depende de la magnitud y del protocolo. Con $R^2 = 1 - \mathrm{SSE/SST}$, el modelo local de Strukov muestra alta concordancia de forma con las series digitalizadas usadas ($R^2 = 0{,}999657$ para el estado, 0,996994 para la corriente, 0,998319 para la resistencia y 0,983726 para la conductancia). Los ERP de conductancia Strukov (19,04 %) y de la curva STDP (14,87 %) son errores máximos normalizados por el rango de sus respectivas series; el umbral ERP $< 10\,\%$ del Capítulo 1 se aplica a parámetros puntuales, mientras la aceptación de curvas se evalúa con R^2 y coincidencia cualitativa. Por ello, ambos ERP no contradicen por sí solos el criterio de curva: Strukov cumple $R^2 > 0{,}90$, y STDP obtiene $R^2 = 0{,}94220$ en un holdout pequeño. Para Prezioso se dispone de una reconstrucción visual aproximada de la Fig. 1(b), no de métricas de ajuste: el barrido temporal no está completamente especificado y los datos digitalizados no permiten separar las ramas SET/RESET. La histéresis se mantiene como evidencia parcial. En STDP, el holdout de cuatro puntos tiene RMSE = 2,136 puntos porcentuales; su tamaño no permite una afirmación amplia de generalización. Además, la curva digitalizada asigna $\Delta t > 0$ a LTP, mientras que la definición temporal en prosa de Jo et al. presenta el signo opuesto. El proyecto usa explícitamente $\Delta t = t_{\mathrm{post}} - t_{\mathrm{pre}}$ y prueba que $\Delta t > 0$ potencia y $\Delta t < 0$ deprime. En LTP/LTD, se obtuvo en 18 puntos de test $R^2_{\mathrm{residual}} = 0{,}73988$, MAE = 8,349 nA, RMSE = 8,618 nA y ERP = 31,23 %, aplicando los pulsos y la lectura indicados en la Fig. 2a. Pearson del test agregado es $r = 0{,}99303$ ($r^2 = 0{,}98610$), pero este coeficiente mide covariación y no sustituye al error residual: la predicción de LTP es constante (Pearson no definido) y LTD obtiene $r = -0{,}22170$. Las métricas desagregadas son débiles ($R^2 = -8{,}54$ para LTP y $R^2 = -829{,}99$ para LTD); el resultado agregado no basta para demostrar concordancia por rama. Los parámetros efectivos se ajustaron a la misma curva digitalizada, por lo que no acreditan amplitud ni predicción independientes. En LIF, Euler concuerda con la solución analítica del propio modelo ($R^2 = 0{,}999964$). En conjunto, la evidencia cuantitativa es heterogénea: alta en Strukov y LIF, limitada en STDP por el tamaño del holdout y no satisfactoria por rama en LTP/LTD. La variabilidad D2D/C2C se probó estadísticamente con muestras sintéticas y se habilitó en el crossbar del agente, sin comparación con medidas de hardware. El MNA se comprobó en un caso pequeño, cuya diferencia relativa de norma respecto a la VMM ideal fue 10,98 %; no es una comparación con hardware. En consecuencia, la evidencia disponible no permite afirmar concordancia experimental general para dispositivos, redes SNN o crossbars físicos. Entre las limitaciones

de integración están el uso de X como estado del modelo simulado y no como dispositivo físico, la LIF como monitor/WTA en REINFORCE y la ausencia de programación con *sneak paths*. Los resultados principales de Auto 2D de esta tesis usan una semilla; la carpeta de evidencias agrega un piloto independiente con tres semillas emparejadas por condición y evaluación final congelada. El piloto comparativo posterior ya cubre CartPole, visión y obstáculos, pero solo con dos semillas; Car cuenta con una repetición R-STDP separada y no incluye una condición REINFORCE emparejada. Falta ampliar la muestra y uniformar el protocolo Car antes de estimar diferencias entre reglas. En el agente, $X_0 = 0{,}1$ (equivalente a $x_0 = 0{,}9$) es una elección de diseño que difiere de la convención de estado inicial del dispositivo aislado; el piloto también explora $X_0 = 0{,}9$, pero no sustituye un estudio de sensibilidad con mayor tamaño muestral ni permite extrapolar a otras inicializaciones.
Otras limitaciones del alcance son: las referencias de literatura se introducen como curvas digitalizadas y no como mediciones propias; el holdout STDP comprende solo cuatro puntos y los parámetros efectivos de LTP/LTD se estiman sobre la misma curva de referencia, con desempeño deficiente por rama; la reconstrucción Prezioso es visual y no identifica el protocolo temporal del artículo; en REINFORCE, la LIF actúa como monitor/WTA y no como decisor principal; la matriz X representa el estado de celdas simuladas, no un crossbar físico; las actualizaciones del agente cambian directamente el estado abstracto X, con límites y variabilidad configurada, sin simular una forma de onda de programación ni pulsos discretos por actualización; los valores de γ y la actualización de elegibilidad difieren entre tareas; el solver MNA incluye resistencias de línea y el acoplamiento de la topología de lectura simulada, pero no la configuración clásica de *sneak paths* con líneas flotantes ni la programación parásita; y los resultados principales de Auto 2D cuentan con una sola semilla. El piloto separado de la carpeta de evidencias amplía esa tarea a tres semillas por condición, pero sigue siendo exploratorio. Por tanto, los resultados de las tareas no constituyen una comparación general entre algoritmos ni permiten generalizar el rendimiento más allá de las configuraciones evaluadas.

4.3. Recomendaciones

A partir de las lecciones aprendidas durante el desarrollo y de las limitaciones observadas en la validación, se formulan las siguientes recomendaciones:

- **No idealidades de crossbar:** la lectura incluye resistencias de línea y el acoplamiento de la topología simulada, pero no la configuración clásica de *sneak paths* con líneas flotantes ni la escritura parásita. Se recomienda modelar y validar ambos casos antes de extrapolar a arreglos de mayor densidad.

- **Acoplamiento térmico:** complementar el ruido estocástico macroscópico actual con un modelo de calentamiento de Joule dependiente de la disipación temporal a escala nanométrica.

- **Robustez del solver:** el esquema de integración Euler Forward requiere pasos temporales restrictivos ($dt \leq 10^{-3}$ s) en los umbrales ultra no lineales del dispositivo; se recomienda evaluar integradores adaptativos (p. ej. Runge-Kutta) para ampliar la ventana numérica de simulación.

- **Reproducibilidad continua:** mantener la suite de validación como herramienta de referencia en `neuromorphic_lab/validaciones/`, ejecutando automáticamente los scripts (p. ej. `python 01_strukov_2008.py`) bajo Python 3.10+ para revalidar el ecosistema en cada modificación, sin intervención manual del operador.

- **Validación de escritura y selección:** contrastar con datos experimentales los pulsos de escritura y los cambios de X y G del modo R-STDP; evaluar si la LIF debe participar en la selección de acciones del modo REINFORCE. Mantener explícita la distinción entre X, conductancia y pesos normalizados.
  item
  textbfCobertura estadística: ampliar el piloto REINFORCE/R-STDP desde dos semillas a una muestra previamente dimensionada; emparejar Car bajo el mismo intervalo de evaluación y reportar resultados por condición. Mantener separados los resultados incompletos y las corridas que alcanzan el presupuesto máximo sin éxito. Documentar además las fuentes y el proceso de digitalización de cada CSV atribuido a la literatura.

4.4. Trabajo Futuro

El simulador se evaluó en cuatro tareas: control con recompensa densa (CartPole-v1), clasificación supervisada (Visión 8×8), navegación con recompensa escasa (Gridworld 10×10) y control en un entorno 2D de estado continuo (Auto 2D). La extensión a otros entornos de control continuo y a tareas con acciones continuas se plantea como trabajo futuro. También se consideran la unificación de la regla de elegibilidad entre tareas, la integración de la LIF en el lazo de decisión y la modelización de *sneak paths* durante escritura.
Las líneas de desarrollo identificadas sobre la base de los resultados obtenidos comprenden:

- **Crossbars de alta densidad:** extender el solver MNA para arreglos de escala industrial (por ejemplo 128×128) con selectores (*memristor switches*) y modelado completo de sneak paths y margen de ruido.

- **Redes neuronales pulsantes completas:** combinar la neurona LIF implementada con las sinapsis STDP en redes SNN capaces de tareas de reconocimiento de patrones de mayor complejidad (p. ej. MNIST o sensores DVS).

- **Comparación de reglas de aprendizaje:** completar y documentar una comparación controlada de REINFORCE y R-STDP en las cuatro tareas de Robot Sim, con semillas

emparejadas, presupuesto de entrenamiento equivalente, checkpoints seleccionados solo con validación y pruebas finales con pesos congelados. Publicar los registros originales y las métricas por semilla antes de concluir que una regla supera a la otra.

- **Co-simulación con hardware:** exportar las matrices conductivas entrenadas a flujos de diseño de circuitos (p. ej. SPICE o frameworks de computación en memoria) para acercar la simulación a la implementación física.
- **Benchmarking inter-simulador:** comparar las métricas de Neuromorphic Lab con herramientas de referencia del área para contextualizar su precisión dentro del estado del arte.

APÉNDICE A

Validación cuantitativa del simulador

Este apéndice orienta al lector hacia la evidencia vigente, sin repetir figuras o tablas del cuerpo principal. Las métricas reproducibles y el alcance de cada comparación se presentan en la Sección 3.4; los protocolos y limitaciones se desarrollan en el Apéndice D. El inventario de scripts, fuentes y archivos se consulta en el Apéndice B y en la Tabla **C-1**. Los resultados históricos que no pudieron reproducirse durante la revisión no se presentan como evidencia vigente.

APÉNDICE B

Estructura del Código Fuente y Módulos

En este apéndice se resume la organización actual del paquete `neurolab` y la cobertura de las pruebas de Neuromorphic Lab y Robot Sim. El árbol muestra rutas principales y omite archivos de respaldo y módulos auxiliares.

B.1. Árbol de directorios del paquete `neurolab`

La estructura vigente dentro de `neuromorphic_lab/neurolab/` incluye los siguientes módulos principales:

```
neurolab/
|-- __init__.py
|-- core/
|   |-- base_device.py       # Interfaces de modelos y modificadores
|   |-- config.py
|   |-- memristor.py
|   +-- validation_metrics.py
|-- devices/
|   |-- strukov.py           # Modelo lineal Strukov 2008
|   |-- yakopcic.py          # Modelo asimétrico Yakopcic 2011
|   |-- models/strukov.py
|   +-- realism/             # c2c, d2d, noise, volatile, window
|-- neurons/
|   |-- lif.py               # Neurona Leaky Integrate-and-Fire
|   +-- base.py
|-- synapses/
|   |-- memristive_synapse.py# Conexión sináptica basada en memristor
|   +-- stdp.py
|-- circuits/
```

```
|   +-- hybrid.py
|-- crossbar/
|   |-- core.py              # Clase Crossbar unificada (M x N)
|   |-- nodal_solver.py
|   |-- line_resistance.py
|   +-- validation.py
|-- gui/
|   |-- main_window.py       # Ventana principal PySide6
|   |-- base_view.py         # Canvas base genérico BaseCrossbarCanvas
|   |-- crossbar_view.py     # Panel multitab Crossbar
|   |-- crossbar_2x2_view.py # Vista interactiva 2x2
|   |-- crossbar_4x4_view.py # Vista interactiva 4x4
|   +-- widgets/             # Componentes Matplotlib y sliders
|-- io/
|   +-- validation_loader.py
+-- robot/
|   |-- env_wrapper.py
|   |-- rl_loop.py
|   +-- crossbar_brain.py
+-- validation/
    |-- statistics.py
    |-- prezioso2015_s3.py
    +-- jo2010_stdp.py
```

B.2. Descripción de los módulos principales

Tabla **B-1**: Resumen de módulos del paquete `neurolab`.

Módulo	Responsabilidad Principal
`neurolab.core`	Define la configuración, el dispositivo memristivo base y las métricas de validación.
`neurolab.devices`	Implementa los modelos memristivos (Strukov, Yakopcic, Memristor Volátil).
`neurolab.neurons`	Implementa la neurona Leaky Integrate-and-Fire (LIF) y su configuración.
`neurolab.synapses`	Implementa la sinapsis memristiva y las reglas LTP/LTD y STDP. Los runners ofrecen REINFORCE y R-STDP: el primero actualiza X con elegibilidad de política; el segundo usa elegibilidad pre/acción modulada por retornos descontados. En ambos, el memristor volátil modula la corriente de entrada a las LIF. El R-STDP actual no usa el spike LIF como evento post ni aplica pulsos físicos. El piloto de comparación está en el apéndice de datos; no permite concluir superioridad estadística.
`neurolab.circuits`	Ensambla el circuito híbrido memristor-neurona.
`neurolab.crossbar`	Gestiona matrices, solver nodal, resistencias de línea y validación.
`neurolab.gui`	Proporciona la interfaz gráfica PySide6 con renderizado dinámico interactivo.
`neurolab.io`	Gestiona la carga de perfiles y datos de validación.
`neurolab.validation`	Agrupa validaciones estadísticas y comparaciones de referencia. Las métricas generales están en `neurolab.core.validation_metrics`.

B.3. Conjuntos de datos y salidas guardadas

La Tabla **B-2** reúne archivos de entrada y resultados guardados citados en los Apéndices A y D. Se indican sus rutas y estado de disponibilidad; la procedencia científica se mantiene según

la atribución registrada en el proyecto. Los archivos de salida de simulación se distinguen expresamente de las referencias.

Tabla **B-2**: Inventario centralizado de datos de entrada y salidas CSV citados.

Archivo y ubicación	**Contenido o uso**	**Estado en el proyecto**
`02_SOFTWARE/neuromorphic_lab/data_validation/strukov_2008/CvsT.csv`, `.../VvsT.csv` y `.../WDvsT.csv`	Series digitalizadas de corriente, voltaje y estado atribuidas a Strukov (2008)	Incluidas; son datos de referencia, no salidas del simulador
`02_SOFTWARE/neuromorphic_lab/data_validation/prezioso/VvsC_Prezioso.csv`	Ciclo I–V asociado a Prezioso (2014)	Presente
`02_SOFTWARE/neuromorphic_lab/data_validation/jo2010/jo2010_stdp.csv` y `.../jo2010_ltp_ltd.csv`	Series de referencia STDP y LTP/LTD	Presentes; asociadas a Jo et al. (2010)
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/validacion_wang2025.csv`	Resumen asociado a la validación Wang 2025; no contiene las señales originales	Incluido; es un resumen, no una serie temporal de referencia
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/validacion_strukov.csv`	Trazas simuladas t, V, I, x, R, G y P	Presente; salida del modelo, no CSV de referencia
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/validacion_d2d_c2c.csv`	Resumen de comprobaciones estadísticas sintéticas D2D/C2C	Presente; no contiene mediciones físicas
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/lif_fi_curve.csv`	Frecuencia LIF simulada y solución analítica para 14 corrientes	Generado por `26_lif_fi_curve.py`; no es referencia experimental

Continúa en la siguiente página...

Tabla **B-2**: Inventario de datos (continuación)

Archivo y ubicación	Contenido o uso	Estado en el proyecto
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/validacion_estadistica_completa.csv`	Resultados estadísticos archivados de variabilidad D2D/C2C	Presente; fuente de los valores archivados señalados en C-1
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/prezioso_s3a_modelo_ejemplo.csv` y variantes `s3b/s3c`	Curvas, matriz e histograma sintéticos para D-21/D-22	Presentes; datos generados por el modelo local, no mediciones publicadas
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/lif_behavior_qualitative.csv`	Traza temporal LIF que acompaña la figura de comportamiento	Incluida; salida simulada, no datos experimentales de Wang
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/validacion_iv_ltp_ltd.csv`	Datos guardados de la simulación de histéresis I–V	Presente; asociado a V21 y al script `19_iv_histeresis.py`
`02_SOFTWARE/neuromorphic_lab/validaciones/datos/lif_rc_validation.csv`	Trazas de la prueba numérica LIF subumbral frente a la solución RC analítica	Presente; CSV archivado asociado a V22, separado de la comparación cualitativa con Wang

B.4. Catálogo de figuras y scripts asociados

El catálogo es una guía de localización de 26 entradas que agrupan 32 archivos gráficos representativos. No sustituye las figuras y explicaciones técnicas del Apéndice D ni constituye un inventario exhaustivo de todos los recursos del repositorio.

Tabla **B-3**: Selección de figuras y scripts asociados; 26 entradas agrupan 32 archivos gráficos.

#	Archivo de figura	Script asociado	Contenido representado
1	`fig_01_strukov.png`	`01_strukov_2008.py`	Seis paneles de Strukov

Continúa en la siguiente página...

Tabla **B-3**: Catálogo de figuras y scripts asociados (continuación)

#	Archivo de figura	Script asociado	Contenido representado
2	fig_02_prezioso.png	02_prezioso_2014.py	Curva del modelo Yakopcic
3	fig_03_lif.png	03_lif_analitica.py	Respuesta subumbral LIF
4	fig_04_convergencia.png	04_convergencia.py	Convergencia temporal
5	fig_05_d2d_c2c.png	05_d2d_c2c.py	Variabilidad D2D y C2C
6	fig_06_hfo2_isi_legible.pdf	06_hfo2_isi.py	Acoplamiento HfO_2–LIF
7	fig_07_r_x.png	07_r_x_consistencia.py	Relación $R(x)$
8	fig_08_rs_x.png	08_rs_de_x.py	Resistencia R_S frente a $x(t)$
9	fig_09_dt_extremos.png	09_dt_extremos.py	Estabilidad temporal
10	fig_10_triangular.png	10_triangular.py	Respuesta a excitación triangular
11	fig_11_ruido.png	11_ruido.py	Sensibilidad paramétrica al ruido
12	fig_12_estadistica.png	12_estadistica.py	Estadística D2D/C2C archivada
13	fig_13_lif_behavior.pdf	25_lif_behavior.py	Integración y disparo LIF cualitativos
14	fig_14_ltp_ltd.png	14_ltp_ltd.py	Pulsos LTP/LTD
15	fig_15_stdp.png	15_stdp.py	Ventana STDP
16	fig_16_ciclo.png	16_ltp_ltd_ciclo.py	Ciclos LTP/LTD
17	fig_17_stdp_spikes.png	17_stdp_spikes.py	Solapamiento de pulsos STDP
18	fig_18_stdp_temporal.png	18_stdp_temporal.py	Evolución temporal de pesos
19	fig_19_iv_hysteresis.png	19_iv_histeresis.py	Histéresis I–V
20	fig_20_vi_ltp_ltd.png	20_vi_ltp_ltd.py	Variables V-I-R-G
21a-d	fig_21a-d_crossbar_*.png	21_crossbar_1x1.py	Señales del crossbar 1×1
22	fig_22_prezioso_v2.png	22_prezioso_v2.py	Respuesta I–V simulada
23a-c	23a--23c_prezioso_s3*_modelo_ejemplo.pdf	23_prezioso_s3.py	Curvas cualitativas y matriz sintética

Continúa en la siguiente página...

Tabla **B-3**: Catálogo de figuras y scripts asociados (continuación)

#	Archivo de figura	Script asociado	Contenido representado
24	`auto2d_trayectoria_seed0.png`	`generate_auto2d_trajectory.py`	Ejemplo de trayectoria exitosa guardada en Auto 2D
25	`lif_fi_curve.png`	`26_lif_fi_curve.py`	Curva LIF frecuencia–corriente y solución analítica
26	`fig_14b_ltp_ltd_jo2010.png`; `fig_15b_stdp_jo2010.png`	`24_jo2010_white.py`	Ajustes LTP/LTD y STDP de Jo et al.

B.5. Resumen de la suite de pruebas unitarias (Pytest)

El paquete declara Python 3.10 como versión mínima. Las pruebas de ambos subsistemas se ejecutaron el 27 de octubre de 2026 bajo Python 3.14, desde la carpeta `02_SOFTWARE` de la entrega, mediante el comando:

```
python-B-mpytest-q-pno:cacheprovider--basetemp.pytest-tmpneuromorphic_
lab/testsrobot_sim/tests
```

Se ejecutaron 165 pruebas y las 165 pasaron. La prueba adicional *de invariancia del estado durante la lectura* realiza 1000 lecturas consecutivas y comprueba que el estado X y la conductancia derivada G permanezcan idénticos antes y después de la lectura. Esta ejecución verifica las pruebas automatizadas de ambos subsistemas, no reproduce por sí sola todas las métricas científicas transcritas en los apéndices.

Tabla **B-4**: Pruebas automatizadas de Neuromorphic Lab y Robot Sim: 165 aprobadas (27 de octubre de 2026).

Resultado	Número de pruebas
Ejecutadas y aprobadas	119

B.6. Fragmentos de Código Centrales

A continuación se exponen fragmentos representativos del núcleo matemático del simulador `neurolab`, ilustrando la aplicación de tipado estático, modularidad y las leyes físicas descritas en el marco teórico.

B.6.1. Modelo Matemático de Strukov (`strukov.py`)

Implementación de la ecuación diferencial gobernante (dx/dt) encapsulada como un módulo matemático independiente.

```
class StrukovMathModel(BaseMathModel):
    """
    Modelo Matemático Determinista de Strukov et al. (Nature 2008).
    Ecuación diferencial de deriva iónica normalizada:
        dx/dt = (mu_v * R_on / D^2) * I(t)
    """
    def compute_dxdt(
        self, state: float, voltage: float, current: float,
        electrical: ElectricalConfig,
        model_config: Optional[StrukovConfig] = None
    ) -> float:
        cfg = model_config if model_config is not None else StrukovConfig()
        v_th = float(getattr(cfg, "V_th", getattr(cfg, "v_th", 0.1)))
        if v_th > 0 and abs(voltage) <= v_th:
            return 0.0
        return float((cfg.mu_v * electrical.r_on / (cfg.D ** 2)) * current)
```

B.6.2. Paso del Dispositivo (`core/memristor.py`)

La API vigente obtiene la corriente antes de actualizar el estado y devuelve esa corriente. La integración y los modificadores se aplican en `update`.

```
def step(self, voltage: float, dt: float) -> float:
    i = self.current(voltage)
    self.update(voltage, dt)
    return i
```

APÉNDICE C

Índice Maestro de Validaciones

Este apéndice es el índice central de 28 verificaciones y métricas reportadas. Cada fila relaciona el identificador con el origen, el script, el resultado consignado y la evidencia principal en los Apéndices A o D. Las referencias a figuras y tablas sirven para localizar el desarrollo; no implican ejecuciones independientes ni cambian el alcance descrito junto a cada evidencia.

Tabla **C-1**: Índice de V01–V28 con scripts, valores recalculados o reportados y evidencia principal. El alcance indicado en cada fila distingue resultados reproducidos de históricos.

ID	Origen	Script asociado	Métrica	Valor consignado	Evidencia
V01	Strukov 2008	`01_strukov_2008.py`	R^2 y ERP por rango	Estado/corriente R^2 = 0,999657/0,996994; ERP = 1,27/6,22 %	`validacion_strukov_metricas.csv`; Tabla **3-5**
V02	Strukov 2008	`07_r_x_consistencia.py`	Error relativo $R(x)$	0,00 % reportado	Fig. **??**
V03	Strukov 2008	`04_convergencia.py`	Orden de convergencia p	0,98 reportado	Fig. **??**
V04	Strukov 2008	`09_dt_extremos.py`	Umbral máximo de dt	10^{-3} s reportado	Fig. **??**
V05	Strukov 2008	`10_triangular.py`	Rango de $x(t)$	$[0{,}0, 0{,}38]$ reportado	Fig. **??**

Continúa en la siguiente página...

Tabla **C-1**: Índice maestro de validaciones (continuación)

ID	Origen	Script asociado	Métrica	Valor consignado	Evidencia
V06	Prezioso 2014	`02_prezioso_2014.py`	Histéresis I–V	Visual aprox.; sin métricas ni ramas identificadas	Fig. **??**
V07	Modelo local inspirado en Prezioso 2014	`22_prezioso_v2.py`	Salida I–V simulada; sin ajuste a datos experimentales	Sin ajuste cuantitativo	Fig. **D-11**
V08	Comparación cualitativa Prezioso 2015 S3a	`23_prezioso_s3.py`	Forma I–V sin puntos digitalizados	Cualitativa	Fig. **D-9**
V09	Modelo local	`23_prezioso_s3.py`	Matriz virtual 4×4	Sintética; semilla 42	Fig. **D-10**
V10	Modelo local	`23_prezioso_s3.py`	Histograma de matriz virtual	Derivado de V09	Fig. **D-10**
V11	Modelo local HfO_2–LIF	`06_hfo2_isi.py`	ISI dentro de dos ciclos simulados	Resultado local reportado	Fig. **D-3**
V12	Identidad interna del modelo	`08_rs_de_x.py`	R_S frente a $x(t)$	Verificación algebraica reportada	Fig. **D-4**
V13	Wang 2025; referencia codificada	`13_wang_2025.py`	Parámetros de referencia	Resumen guardado; no es serie temporal	Tabla **B-2**
V14	Jo 2010, atribuido	`24_jo2010_white.py`	STDP; holdout (R^2, RMSE)	0,94220; 2,13626 pp; $n = 4$; signo declarado	Tabla **3-5**
V15	Jo 2010	`17_stdp_spikes.py`	Solapamiento de pulsos	$\Delta V_{\text{pico}} = 450\,\text{mV}$ reportado	Fig. **??**
V16	Jo 2010	`18_stdp_temporal.py`	Acumulación temporal	$R^2 = 0{,}99$ reportado	Fig. **??**

Continúa en la siguiente página...

Tabla **C-1**: Índice maestro de validaciones (continuación)

ID	Origen	Script asociado	Métrica	Valor consignado	Evidencia
V17	Jo 2010	`14_ltp_ltd.py`	Potenciación $\Delta G > 0$	Signo reportado	Fig. **??**
V18	Jo 2010	`14_ltp_ltd.py`	Depresión $\Delta G < 0$	Signo reportado	Fig. **??**
V19	Jo 2010	`16_ltp_ltd_ciclo.py`	Error de retorno (100 ciclos)	0,84 % reportado	Fig. **??**
V20	Jo 2010	`20_vi_ltp_ltd.py`	Coherencia V-I-R-G	Error = 0,00 % reportado	Fig. **D-7**
V21	Jo 2010	`19_iv_histeresis.py`	Lazo pinzado en origen	$I(0) = 0{,}0$ nA reportado	CSV en Tabla **B-2**
V22	Solución exacta	`03_lif_analitica.py`	R^2 subumbral LIF	0.9999815 (archivado)	Fig. **3-6**
V23	Variabilidad D2D	`05_d2d_c2c.py`	Test KS p-valor D2D	0,8959 reproducido	Fig. **3-5**
V24	Variabilidad C2C sintética	`05_d2d_c2c.py`	Error relativo de varianza OU	3,62 %; prueba sintética	Fig. **3-5**
V25	Ruido paramétrico	`11_ruido.py`	CV de I_{pp}	115 % reportado	Fig. **D-1**
V26	D2D/C2C	`12_estadistica.py`	KS p-valor	ON: 0.7092, OFF: 0.8351 x_0: 0.6187, C2C: 0.0000	Fig. **D-2**
V27	Crossbar 1×1	`21_crossbar_1x1.py`	Operación VMM ($I = G^{\mathsf{T}} V$)	Error = 0,00 % reportado	Fig. **D-8**
V28	Neurona LIF	`26_lif_fi_curve.py`	Curva $f - I$ frente a solución analítica	14 corrientes; mismo modelo	Fig. **3-6**

C.1. Registros del piloto comparativo REINFORCE/R-STDP

Los resultados de aprendizaje por tarea se conservan fuera de la suite de validaciones de dispositivos enumerada en la Tabla **C-1**. El lote comparativo contiene 28 corridas: 24 condiciones para CartPole, visión y obstáculos (dos algoritmos, semillas 0 y 1, y $X_0 = 0{,}1$ y 0,9) y cuatro condiciones Car R-STDP. Cada carpeta contiene el resultado JSON, la curva de éxito, la imagen del crossbar y matrices inicial y final. El protocolo y el resumen están en `../../03_EVIDENCIAS/ENTRENAMIENTOS/COMPARATIVA_SEEDS_0_1_X0/`.

El criterio fue superar el 90 % de éxitos en una ventana móvil de 300 episodios, equivalente a un mínimo de 271 éxitos; el presupuesto máximo fue 10,000 episodios. La definición binaria de éxito varía por dominio y se describe en cada JSON. Los dos registros CartPole REINFORCE incompletos no se clasifican como fallos al máximo. Las corridas Car del primer lote usaron evaluación cada 200 episodios; no deben combinarse como comparación directa con la repetición corregida, que usó cada 100.

La repetición corregida incluye cuatro corridas Car R-STDP con semillas 2 y 4 y $X_0 = 0{,}1$ y 0,9. La semilla 2 alcanzó el criterio en los episodios 750 y 1548; las dos condiciones de la semilla 4 llegaron a 10,000 episodios sin alcanzarlo. Sus registros y resumen están en `../../03_EVIDENCIAS/ENTRENAMIENTOS/CAR_RSTDP_EVAL100_SEEDS_2_4_X0/`. Estos pilotos documentan ejecuciones bajo configuraciones concretas; no constituyen validación en hardware ni una estimación estadística concluyente de superioridad entre algoritmos.

APÉNDICE D

Desarrollo Detallado del Sustrato Memristivo

Este apéndice documenta en orden las Etapas 1–4 del desarrollo del sustrato memristivo que sirvió como base para las cuatro aplicaciones presentadas en el Marco Práctico: modelo memristivo y variabilidad, neurona LIF, plasticidad sináptica y crossbar. El Capítulo 3 resume las Etapas 2 y 4 en el contexto de integración, y continúa con las Etapas 5–7 de aplicación; por tanto, la numeración del apéndice y del capítulo es la misma. La evidencia varía por etapa: incluye verificaciones analíticas del código, simulaciones locales, comparaciones cualitativas con figuras publicadas y algunas comparaciones cuantitativas reportadas. Estas categorías no son equivalentes; una figura sintética o una comparación visual no constituye una validación experimental cuantitativa. Los valores numéricos de experimentos no reejecutados se identifican como resultados reportados y no como resultados reproducidos en esta revisión.

D.0.1. Etapa 1 — Núcleo del Dispositivo Memristivo y Variabilidad

Esta sección describe la implementación y simulación de modelos memristivos y de variabilidad. Incluye comparaciones cuantitativas con datos digitalizados reportadas previamente; salvo el estadístico D2D especificado abajo, sus métricas no se reprodujeron durante esta revisión y no deben interpretarse como una validación independiente ya verificada.

Etapa 1.1: Modelo Base y Curvas Características

Implementación Cualitativa y Fundamentos Físicos El núcleo de la simulación recae en el modelo físico del memristor. Para modelar el comportamiento ideal de dispositivos de dióxido de titanio (TiO_2), se implementó el modelo fenomenológico de Strukov et al. (2008) [5]. El dispositivo se concibe como dos resistencias variables en serie, dependientes de la anchura del filamento conductor $w(t)$ respecto a la longitud total D.
La dinámica del estado interno $x(t) = w(t)/D \in [0, 1]$ está gobernada por la movilidad iónica μ_v. Cuando se necesita expresar el estado en la convención usada por la matriz del agente se define $X = 1 - x$, la fracción no dopada; por tanto, X no es una conductancia. La resistencia

y la conductancia se obtienen como $R(X) = R_{\mathrm{ON}} + X(R_{\mathrm{OFF}} - R_{\mathrm{ON}})$ y $G(X) = 1/R(X)$, respectivamente:

$$\frac{dx}{dt} = \frac{\mu_v R_{\mathrm{ON}}}{D^2} i(t) \cdot f_w(x) \tag{D-1}$$

donde $f_w(x)$ es una función ventana (por ejemplo, Joglekar [23] o Biolek [24]) que modela los efectos de frontera en los extremos $x = 0$ y $x = 1$. La resistencia total del dispositivo en cada instante es la combinación lineal convexa de sus límites resistivos:

$$R(x) = R_{\mathrm{ON}} x + R_{\mathrm{OFF}}(1 - x) \tag{D-2}$$

La respuesta cualitativa del modelo de Strukov exhibe el clásico lazo de histéresis I-V pinzado en el origen. Al aplicar voltajes periódicos, la conductancia oscila suavemente acotada por los límites R_{ON} y R_{OFF}. El estado interno $x(t)$ se restringe estrictamente al rango sin saturaciones espurias, comprobable ante señales triangulares continuas (Figura **??**).

La figura comparativa de Strukov y las métricas recalculadas se presentan en la Figura **3-2** y la Tabla **3-5** del Capítulo 3; aquí se conserva el análisis complementario sin repetir el panel.

Propósito y Relevancia: El panel permite inspeccionar conjuntamente la simulación y las series digitalizadas disponibles. La atribución y el escalado de referencia limitan la interpretación; no es una medición de hardware.

El informe previo también consigna un orden de convergencia $p = 0{,}98$ y un límite $dt \leq 1\,\mathrm{ms}$. Ese análisis no se reprodujo en esta revisión.

La figura de distribuciones D2D/C2C se presenta en la Figura **3-5** del Capítulo 3; aquí se mantienen la descripción del protocolo y sus limitaciones.

Propósito y Relevancia: La figura reúne muestras y curvas simuladas para ilustrar variabilidad D2D y C2C; no demuestra ajuste a mediciones de dispositivos fabricados ni valida su uso en chips industriales.

Finalmente, se evaluó la sensibilidad paramétrica con 50 instancias deterministas y 50 instancias con perturbaciones gaussianas relativas del 5 % en R_{ON}, R_{OFF} y x_0 (Figura **D-1**). En las instancias perturbadas, el coeficiente de variación de la corriente pico a pico supera el 115 %. Este experimento caracteriza la propagación de variaciones paramétricas; no evalúa ruido térmico transitorio inyectado en el integrador.

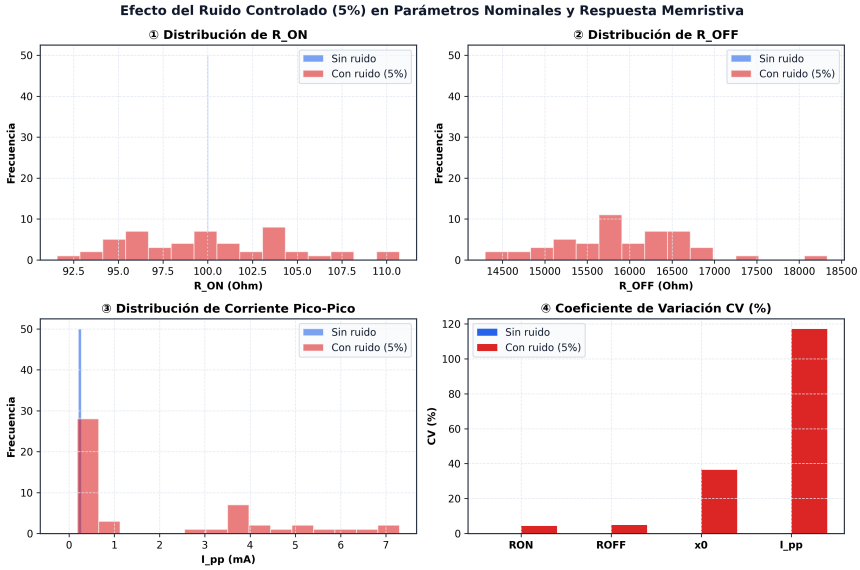


Figura **D-1**: Sensibilidad del dispositivo a perturbaciones gaussianas del 5 % en sus parámetros nominales.

Propósito y Relevancia: La figura compara las distribuciones de R_{ON}, R_{OFF} y corriente pico a pico, además del coeficiente de variación, entre dispositivos nominales y dispositivos con perturbaciones paramétricas. No representa una respuesta frecuencial ni permite concluir que el estado del memristor actúe como filtro pasa-bajos.

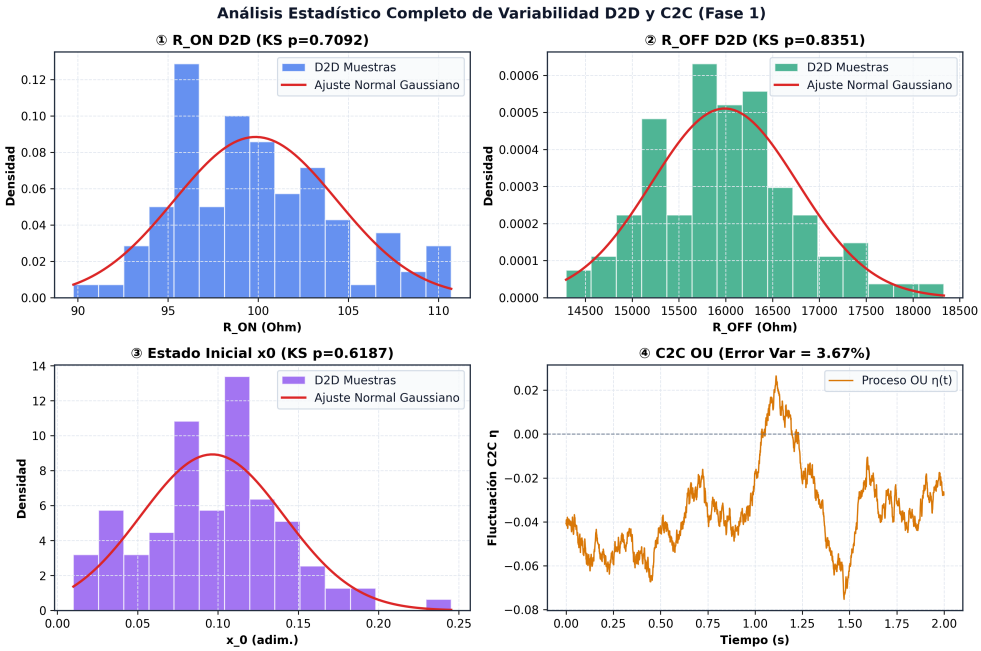


Figura **D-2**: Distribuciones D2D y pruebas KS de una corrida archivada distinta del cálculo reproducido con `05_d2d_c2c.py`.

Los valores p del panel, incluido 0,7092 para R_{ON}, proceden de la corrida archivada en `validacion_estadistica_completa.csv`, generada por `12_estadistica.py`. Esa rutina usa una muestra distinta y ajusta media y desviación muestrales; no se revalidó la calibración del KS tras estimar esos parámetros con la misma muestra. El valor 0,8959 informado arriba proviene de `05_d2d_c2c.py`, con los valores nominales para normalizar y contrastar contra $\mathcal{N}(0,1)$. Son análisis diferentes, por lo que sus p-valores no son intercambiables; el análisis archivado no se reprodujo en esta revisión.

Propósito y Relevancia: El panel reúne resúmenes estadísticos de muestras simuladas. Estos histogramas describen la población generada por el modelo; no demuestran que esa población reproduzca una distribución experimental ni validan arreglos físicos.

D.0.2. Etapa 2 — Neuronas Spiking y Sistema Híbrido

En esta segunda fase, el proyecto integra matemáticamente el modelo de neurona biológica Leaky Integrate-and-Fire (LIF) junto con el dispositivo memristivo, creando el primer bloque de construcción analógico (Memristor–Neurona) capaz de emular codificación temporal y procesamiento neuromórfico sensorial.

Etapa 2.1: Neurona LIF y Acoplamiento Híbrido

Implementación Cualitativa y Teoría Física El circuito híbrido simulado conecta un modelo de memristor en serie entre un estímulo de entrada (voltaje sensorial o pre-sináptico) y una neurona LIF. El potencial de membrana se representa como el voltaje de un capacitor $V_C(t)$ sujeto a una corriente de fuga paralela R_{leak}; no se trata de un circuito físico medido. La ecuación diferencial estocástica que rige el potencial de membrana está dada por:

$$C_m \frac{dV_C}{dt} = -\frac{V_C(t) - V_{\text{rest}}}{R_{\text{leak}}} + I_{\text{inj}}(t) + \sigma_n \cdot \eta(t) \tag{D-3}$$

El acoplamiento se logra inyectando la corriente transmembrana $I_{\text{inj}}(t)$, cuyo valor es dictaminado por la ley de Ohm en la rama de entrada a través de la conductancia del memristor dependiente del estado:

$$I_{\text{inj}}(t) = \frac{V_{\text{in}}(t) - V_C(t)}{R_S(x(t))} \tag{D-4}$$

Cuando $V_C(t) \geq V_{\text{th}}$, la neurona genera matemáticamente un evento discreto (*spike*), y resetea instantáneamente su potencial a V_{reset}, entrando en un período refractario absoluto. Para representar un *sensor volátil*, se emplea un modelo computacional inspirado en dióxido de hafnio (HfO_2). Incluye un término fenomenológico de relajación del estado en ausencia de estímulo:

$$\frac{dx}{dt} = f(V_M, x) - \frac{x}{\tau_{\text{relax}}} \left(1 - H(V_M - V_{\text{hold}})\right) \tag{D-5}$$

Cualitativamente (Figura **D-3**), en la simulación el estímulo de 1,5 V cambia el estado y la resistencia del modelo memristivo, modificando la corriente de entrada y los intervalos entre disparos. La figura ilustra adaptación temporal dentro de este modelo; no demuestra transducción biológica ni una respuesta de un dispositivo fabricado.

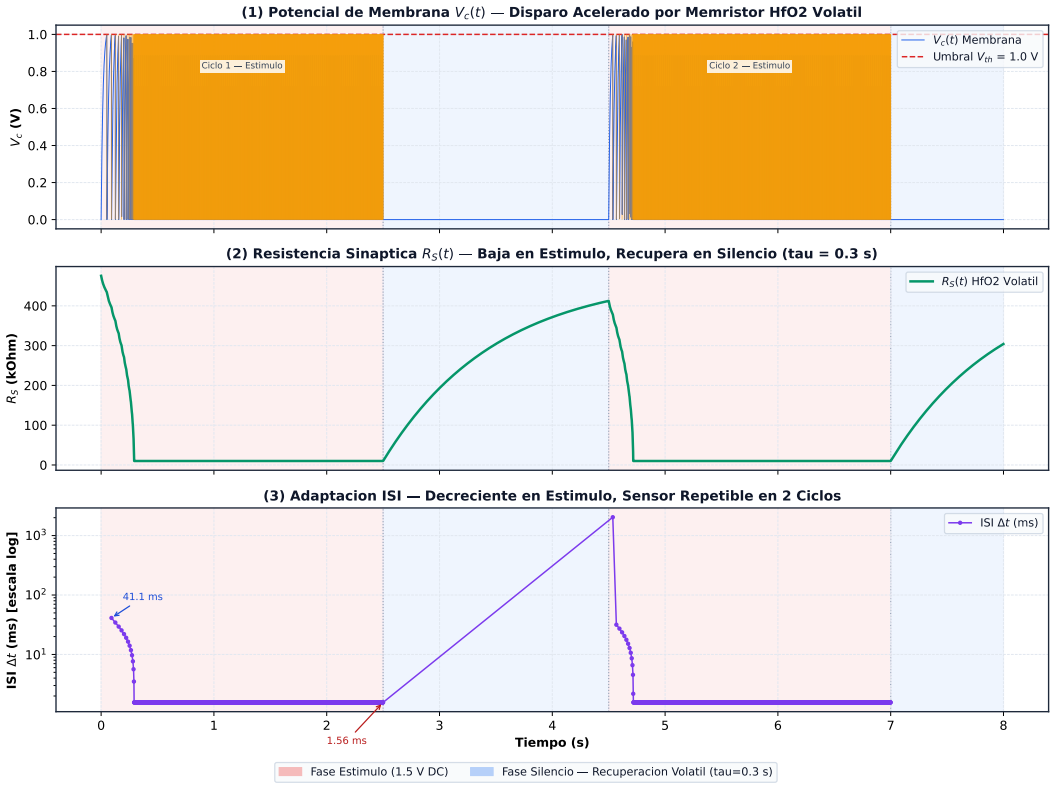


Figura **D-3**: Respuesta cualitativa del sistema híbrido: Potencial de membrana $V_C(t)$ evidenciando la aceleración temporal (top), Resistencia $R_S(t)$ descendente (centro), e Intervalo Inter-Spike (ISI) modularizado (abajo).

Propósito y Relevancia: La figura ilustra el acoplamiento implementado entre el modelo memristivo y la neurona LIF. No constituye una validación de un sensor físico ni de un receptor nociceptivo.

Validación Cuantitativa Computacional La integridad matemática de esta fase fue comprobada contra soluciones exactas y papers de referencia del estado del arte.

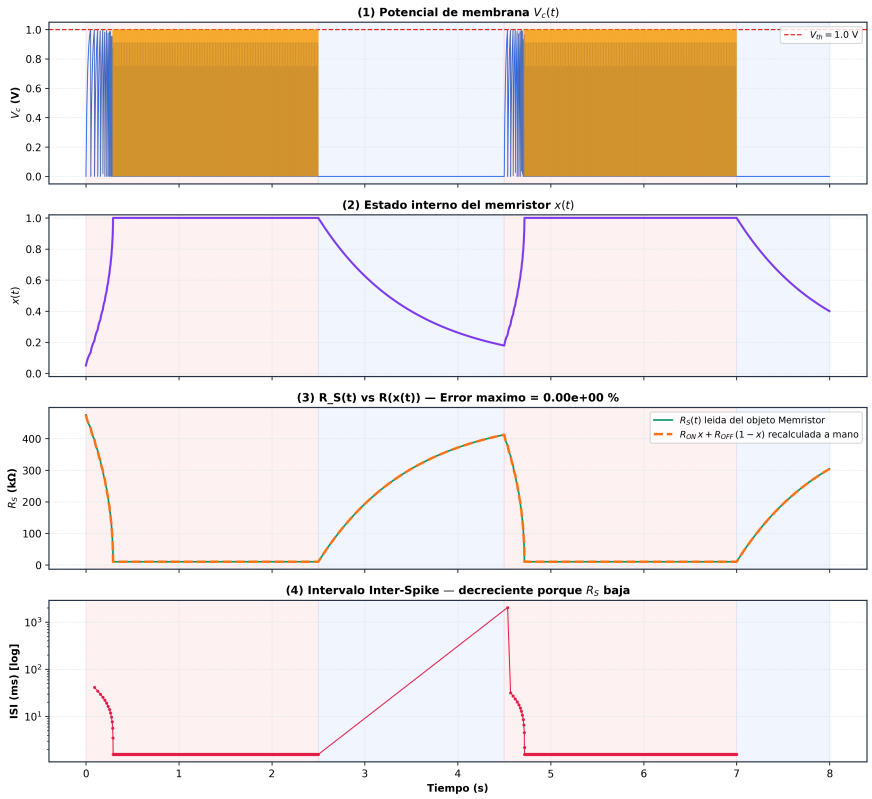


Figura **D-4**: Verificación de la resistencia en serie R_S proveniente de la dinámica de acoplamiento de x.

Propósito y Relevancia: Este gráfico comprueba, para la ejecución y las variables mostradas, la consistencia entre R_S y $x(t)$ según la relación implementada. No prueba por sí solo ausencia general de pérdida de eventos, retrasos ni errores en todas las ejecuciones.
Solución Analítica LIF Subumbral: Se sometió el módulo integrador numérico de la neurona LIF a una validación estricta frente a su solución analítica exacta de circuito RC subumbral:

$$V_m(t) = V_\infty + (V_{m,0} - V_\infty)e^{-t/\tau_{eq}} \tag{D-6}$$

La comparación cuantitativa entre la simulación discreta y la solución RC analítica es distinta de la comparación cualitativa con Wang descrita más adelante. En el CSV archivado `lif_rc_validation.csv`, el cálculo de R^2 sobre las 1000 muestras disponibles es 0,9999815, que redondeado a cuatro decimales da 1,0000. Este resultado describe solo esa prueba subumbral y no se atribuye a la comparación cualitativa ni demuestra exactitud general del integrador para otros pasos o condiciones.

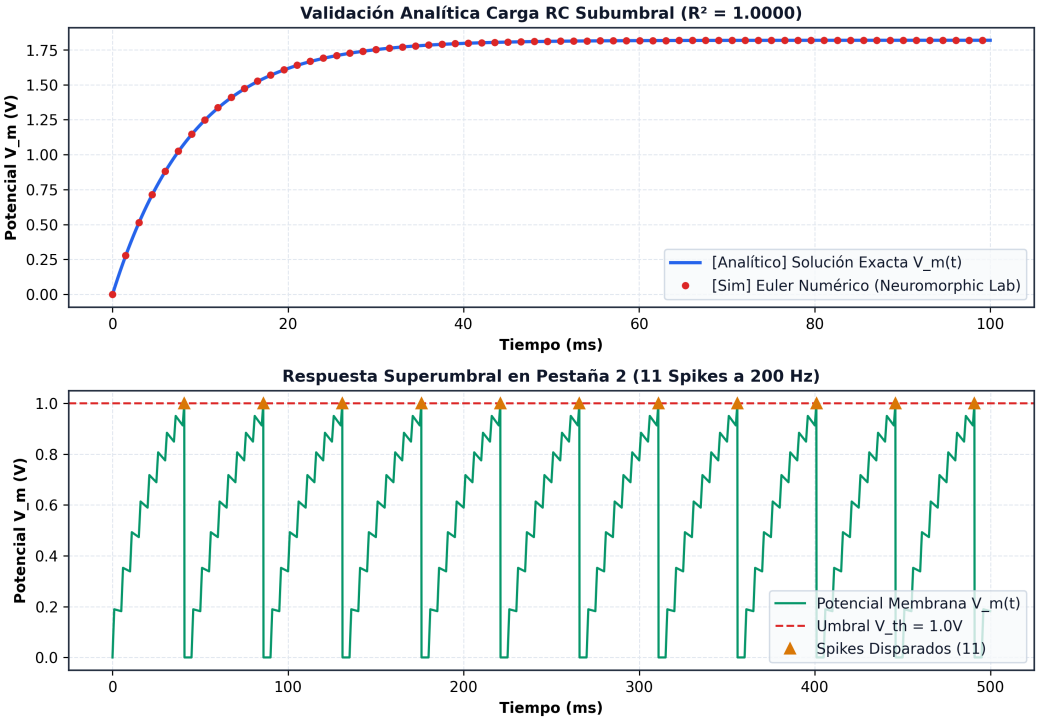


Figura **D-5**: Validación analítica subumbral de la neurona LIF frente a la solución RC exacta en respuesta al impulso.

Propósito y Relevancia: La comparación permite evaluar la respuesta subumbral para las condiciones ensayadas. El valor archivado no basta para descartar sesgos algorítmicos en otras configuraciones ni para afirmar que el núcleo esté libre de errores.

Comportamiento cualitativo de integración y disparo LIF: La neurona LIF del simulador comparte el comportamiento funcional descrito por Wang et al. [3]: integra los estímulos, alcanza el umbral y genera eventos de disparo con reinicio. La Figura **D-6** muestra esta secuencia en el simulador `Neuromorphic Lab`, con la configuración guardada en `02_SOFTWARE/neuromorphic_lab/configs/lif_config.json`: $C_m = 100\,\text{nF}$, $R_S = 100\,\text{k}\Omega$, $R_{\text{leak}} = 1\,\text{M}\Omega$, umbral de 1 V y período refractario de 2 ms. La entrada simulada es un tren unipolar de 2 V a 200 Hz, con ciclo útil del 20 %, durante 0,5 s. Con paso de integración de 1 μs, esta ejecución produce 11 eventos de disparo.

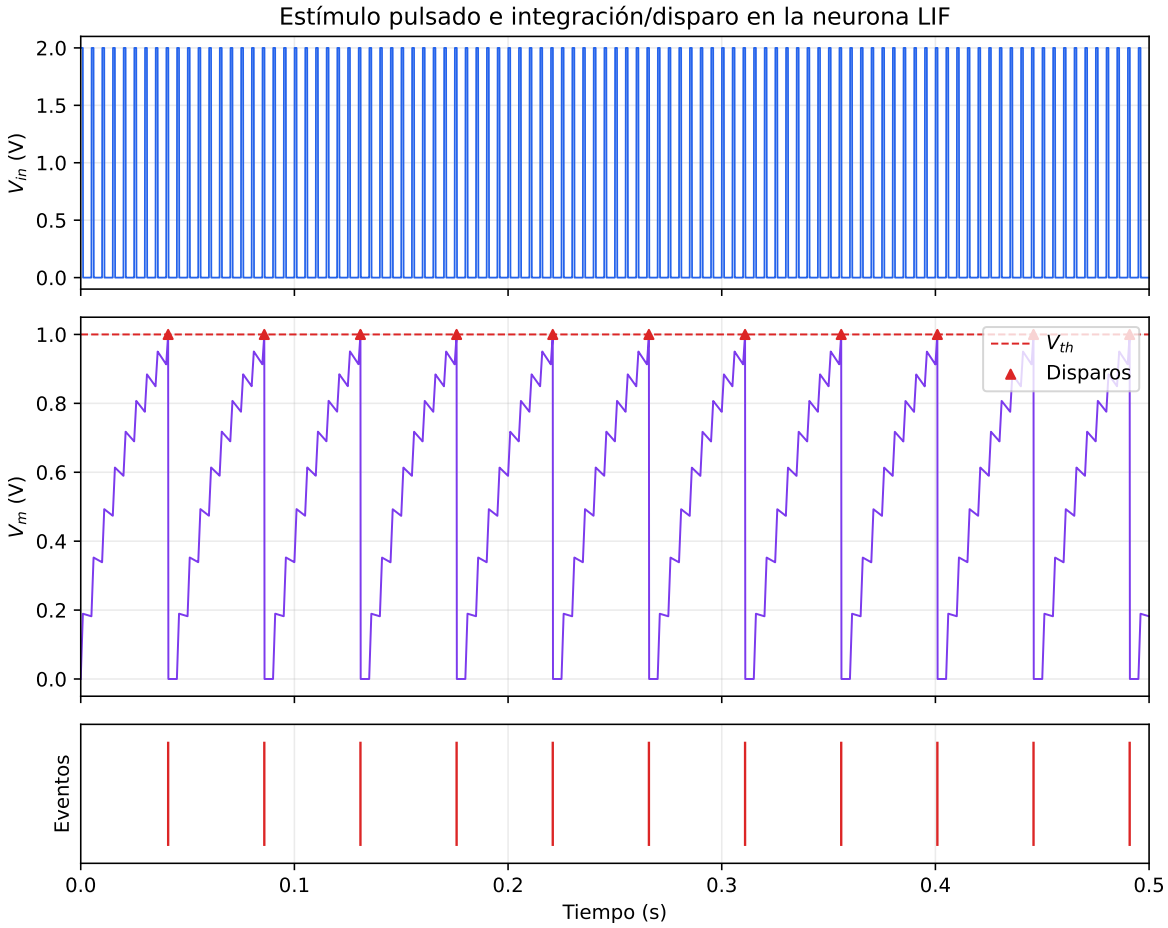


Figura **D-6**: Respuesta de la neurona LIF del simulador: estímulo pulsado, integración del potencial de membrana y eventos de disparo, en concordancia funcional con la dinámica descrita por Wang et al. [3].

Propósito y Relevancia: La figura documenta que el simulador LIF implementa la secuencia de integración y disparo. El panel inferior es un raster de eventos, no una forma de onda de voltaje de salida ni una medición física.

D.0.3. Etapa 3 — Plasticidad Sináptica

Esta fase describe simulaciones de plasticidad LTP/LTD y STDP con modelos memristivos. Las comparaciones con Jo et al. (2010) se recalcularon con el protocolo y las particiones descritos abajo, usando puntos digitalizados de las figuras publicadas. No son mediciones propias ni certifican por sí solas una respuesta física.

Etapa 3.1: LTP/LTD y STDP

Implementación Cualitativa (LTP/LTD) Para estudiar la respuesta acumulativa de la conductancia sináptica (representando el peso lógico W_{ij}), se aplicaron secuencias controladas de trenes de pulsos de voltaje idénticos. Matemáticamente, cada pulso induce un minúsculo diferencial dx positivo o negativo sobre el estado del filamento según su polaridad. Los pulsos positivos inducen la Potenciación a Largo Plazo (LTP), aumentando la conductancia de forma progresiva. Los pulsos negativos provocan la Depresión a Largo Plazo (LTD), borrando la historia almacenada. Como se observa en la Figura **??**, la evolución temporal de la conductancia se da de forma escalonada, exhibiendo una no-linealidad asintótica que satura suavemente en los bordes impuestos $G_{\max}$ y $G_{\min}$.

El ajuste STDP y sus puntos reservados se muestran en la Figura **3-3** del Capítulo 3. Se conserva aquí la interpretación ampliada de la partición y sus límites.

Propósito y Relevancia: La figura documenta la asimetría del conjunto digitalizado y el error de predicción en los puntos reservados. No certifica la precisión de un cambio de conductancia físico ni la validez general del componente.

Curva LTP/LTD atribuida a Jo et al. El CSV `jo2010_ltp_ltd.csv` contiene 87 puntos atribuidos por el validador a la Fig. 2a; no son 200 registros experimentales brutos. Se simularon 100 pulsos SET de +3,2 V y 100 RESET de −2,8 V, ambos de 300 μs, con lecturas a 1 V. De los 87 puntos, dos quedan fuera del intervalo de 0–200 pulsos; los 85 restantes se separan, dentro de cada rama, en bloques contiguos 60/20/20: 50 de entrenamiento, 17 de validación y 18 de test. Se ajustan R_{on}, R_{off}, el coeficiente de deriva efectivo y x_0 solo con entrenamiento; validación selecciona entre tres inicializaciones. No se aplica un reescalado min–max ni una calibración afín posterior. En test se obtuvo $R^2_{\text{residual}} = 0{,}73988$, MAE = 8,3491 nA, RMSE = 8,6181 nA y ERP = 31,23 %. Por rama, el resultado es deficiente: $R^2 = -8{,}54$ en LTP ($n = 10$) y $R^2 = -829{,}99$ en LTD ($n = 8$). Pearson agregado es $r = 0{,}99303$ ($r^2 = 0{,}98610$), pero este valor mide covariación y no sustituye el R^2_{residual}: la predicción de LTP es constante, por lo que su correlación no está definida; en LTD se obtiene $r = -0{,}22170$ ($r^2 = 0{,}04915$). Los parámetros efectivos se ajustan a la misma curva digitalizada y no son una caracterización independiente del dispositivo; la evidencia es parcial y no demuestra predicción independiente ni validación completa del artículo.

La comparación LTP/LTD y sus puntos reservados se muestran en la Figura **3-4** del Capítulo 3; se conserva aquí el análisis cuantitativo por rama.

Propósito y Relevancia: La figura muestra la forma de la curva bajo la normalización declarada. No demuestra fidelidad física general ni identifica un mecanismo microscópico concreto.

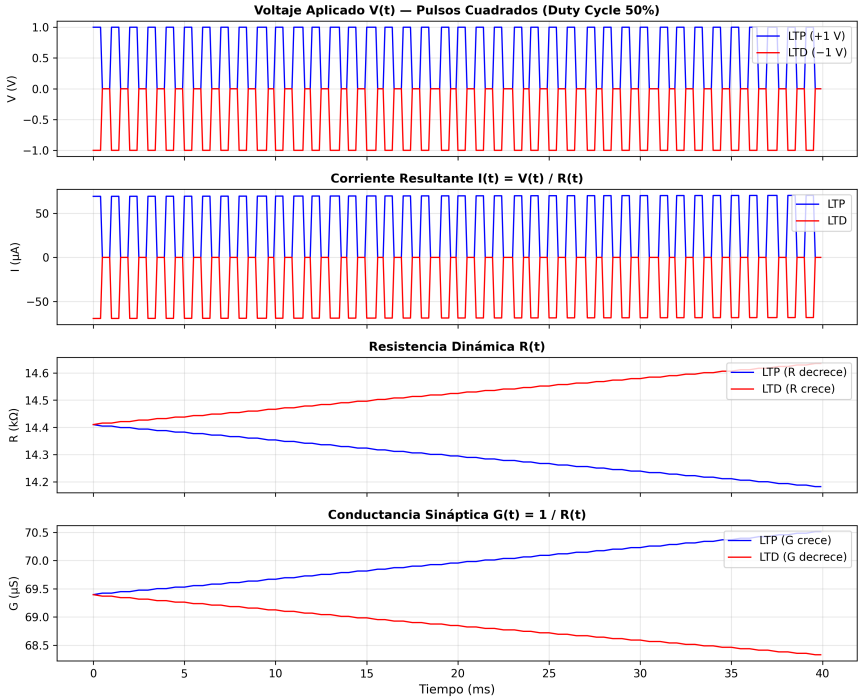


Figura **D-7**: Caracterización simultánea de voltaje, corriente, resistencia y conductancia durante pulsos LTP y LTD.

Propósito y Relevancia: La figura muestra variables registradas por la simulación durante los pulsos. No constituye una observación microscópica ni una prueba general de conservación para otras configuraciones.

D.0.4. Etapa 4 — Arquitecturas Matriciales y No Idealidades

La Etapa 4 estudia implementaciones a nivel de red mediante un arreglo cruzado bidimensional (crossbar). Su aplicación al agente de control se desarrolla en la subseccion siguiente. Los resultados corresponden a modelos y entornos de simulación, no a mediciones de un crossbar fabricado.

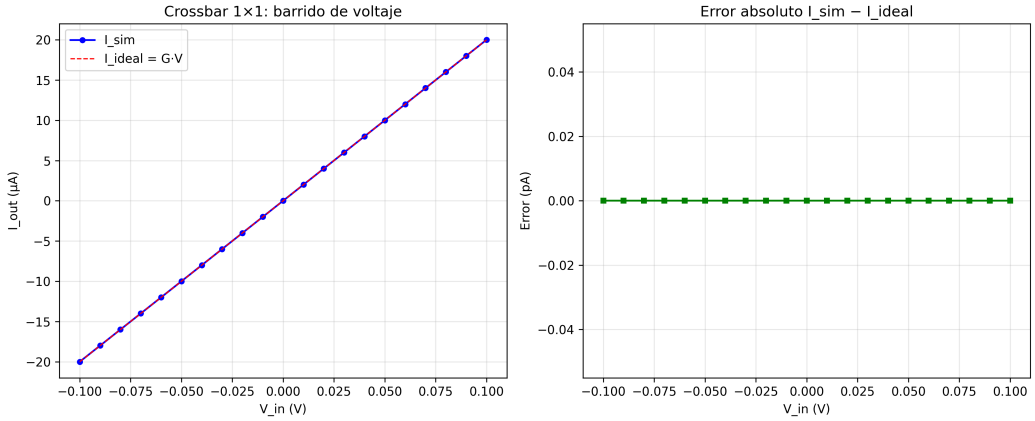
Etapa 4.1: Matriz Crossbar e In-Memory

Una matriz crossbar se representa por $\mathbf{G} \in \mathbb{R}^{N\times M}$, con N filas de entrada y M columnas de salida; cada intersección aloja un dispositivo memristivo simulado.
En el modelo ideal, al aplicar un vector de voltajes de entrada $\mathbf{V} = [V_1, V_2, \ldots, V_N]^\mathsf{T} \in \mathbb{R}^N$ a las filas, las conductancias en paralelo realizan una multiplicación matriz-vector. La corriente I_j de cada columna sigue la ley de Ohm y la ley de corrientes de Kirchhoff:

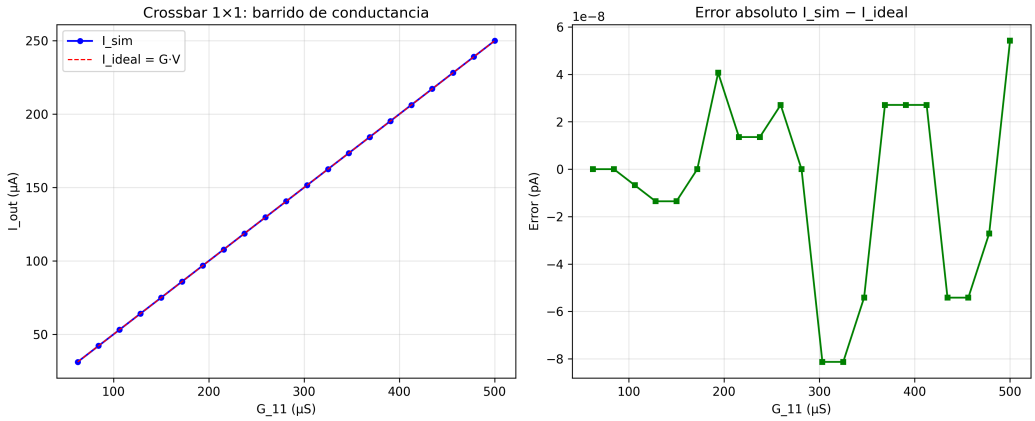
$$I_j(t) = \sum_{i=1}^{N} V_i(t) \cdot G_{ij}(t) \quad \Longrightarrow \quad \mathbf{I} = \mathbf{G}^{\mathsf{T}}\mathbf{V}, \qquad \mathbf{I} \in \mathbb{R}^M \tag{D-7}$$

En un arreglo analógico ideal, las contribuciones de las celdas se procesarían en paralelo. Esta descripción algebraica no demuestra latencia física constante ni complejidad computacional $\mathcal{O}(1)$ para el solver numérico: el costo de resolver el modelo MNA se trata por separado en el Experimento 9.

Implementación Cualitativa (Operación Analógica 1×1) Se implementó y se visualizó estructuralmente el sistema base modular probando una celda atómica unitaria que simula la interacción macro-escalar.
Como se ilustra en las salidas de simulación (Figuras **D-8**a-d), al suministrar la onda cuadrada de voltaje al modelo, la dinámica de la celda memristiva de Yakopcic cambia su estado. La línea de columna calcula $\mathbf{I} = \mathbf{G}^{\mathsf{T}}\mathbf{V}$ en el modelo, con las filas de $\mathbf{G}$ asociadas a las entradas y las columnas a las salidas. Estas figuras ilustran la operación implementada; no son mediciones de hardware ni prueban por sí solas su comportamiento físico.

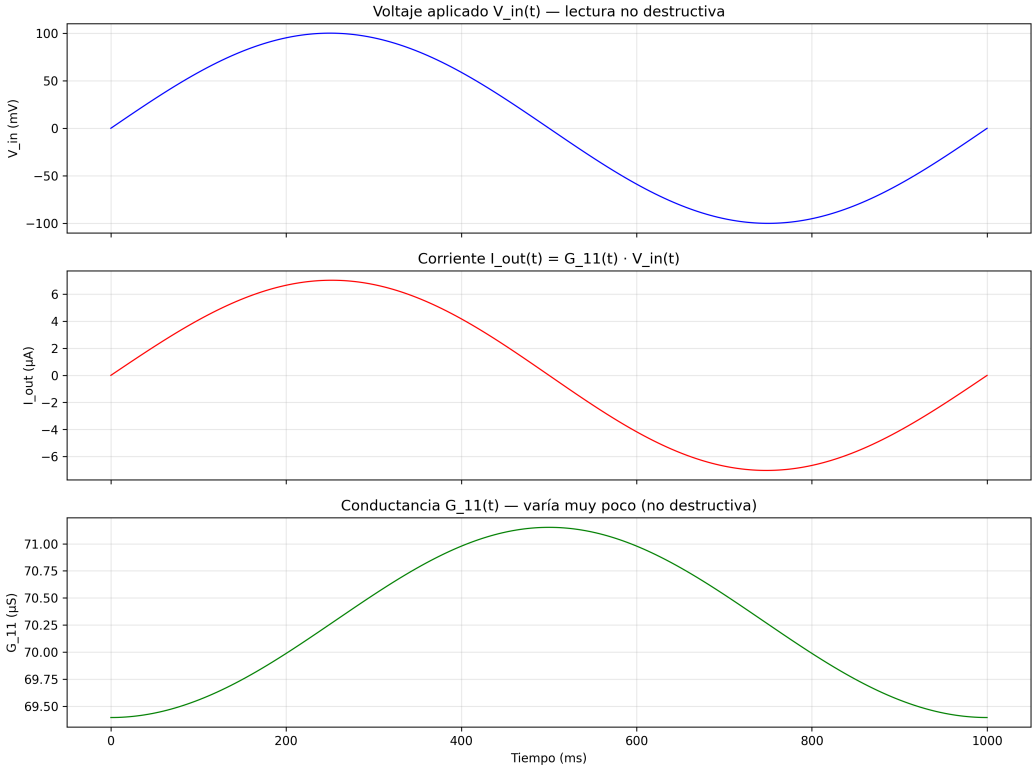


(a) Pulsos de excitación por línea de Palabra $V_i(t)$.

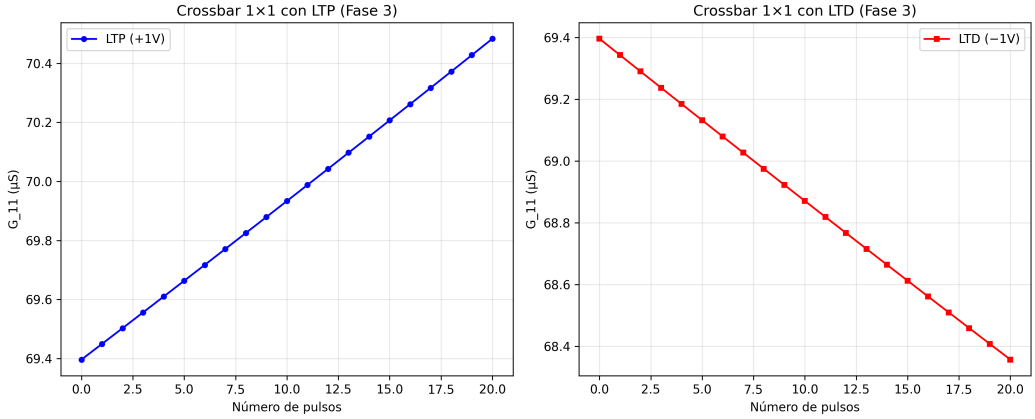


(b) Matriz espacial de conductancia transversal $G_{ij}(t)$.

Figura **D-8**: Verificación cualitativa y funcional de computación In-Memory en el bloque elemental Crossbar analógico (paneles a y b).



(c) Corrientes inferenciales recolectadas $I_j(t)$.



(d) Cinemática del oscilador memristivo de intersección.

Figura **D-8**: (Continuación; paneles c y d).

Propósito y Relevancia: Los cuatro paneles ilustran señales y estados calculados por el modelo de simulación. No demuestran una implementación de hardware ni una ventaja temporal sobre un sistema digital.

Comparación cualitativa con Prezioso et al. (2015) La Figura **D-9** compara cualitativamente la forma de las curvas I–V generadas por el modelo local con las curvas representativas de dispositivos del panel S3a de Prezioso et al. (2015) [25], correspondiente a la caracterización pre-forming de una porción de un crossbar 10×8. La comparación es de comportamiento y forma: el generador usa constantes locales y no carga ni digitaliza los puntos experimentales, por lo que no demuestra ajuste numérico ni concordancia punto por punto. Las Figuras **D-10** muestran una matriz y un histograma sintéticos motivados por los paneles S3b–S3c; no reproducen las mediciones experimentales.
La curva de D-21 es paramétrica. D-22 usa una matriz sintética 4×4 de conductancias, generada con semilla fija a partir de una media y desviación estándar configuradas en el código. Ninguna de estas dos salidas representa mediciones de celdas reales ni valida ruido térmico o variabilidad medida.

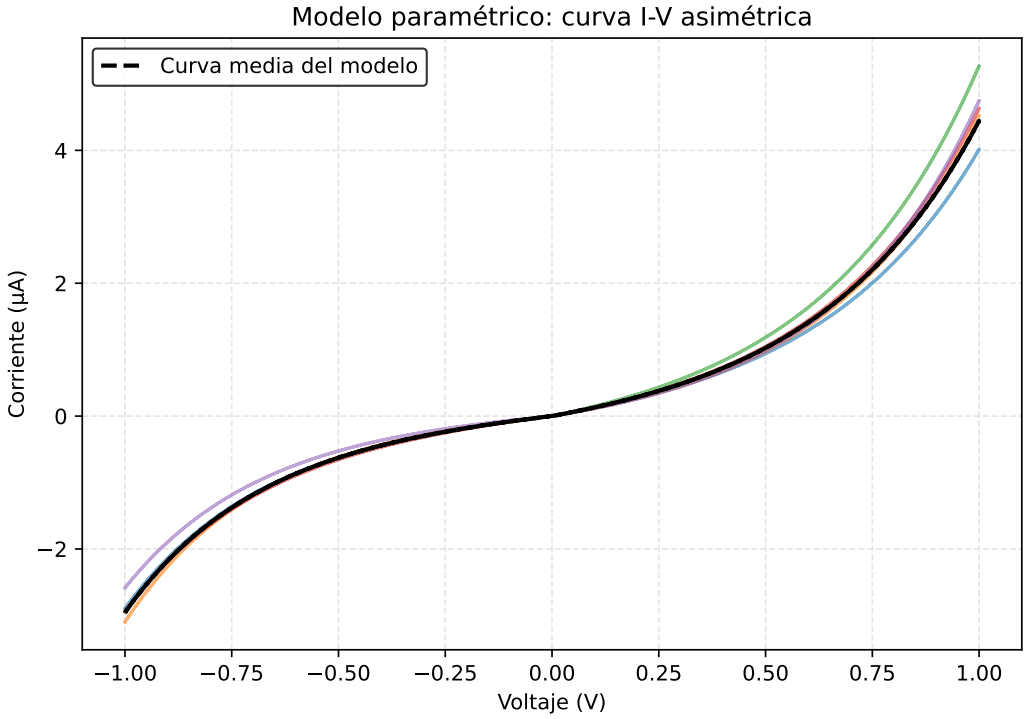
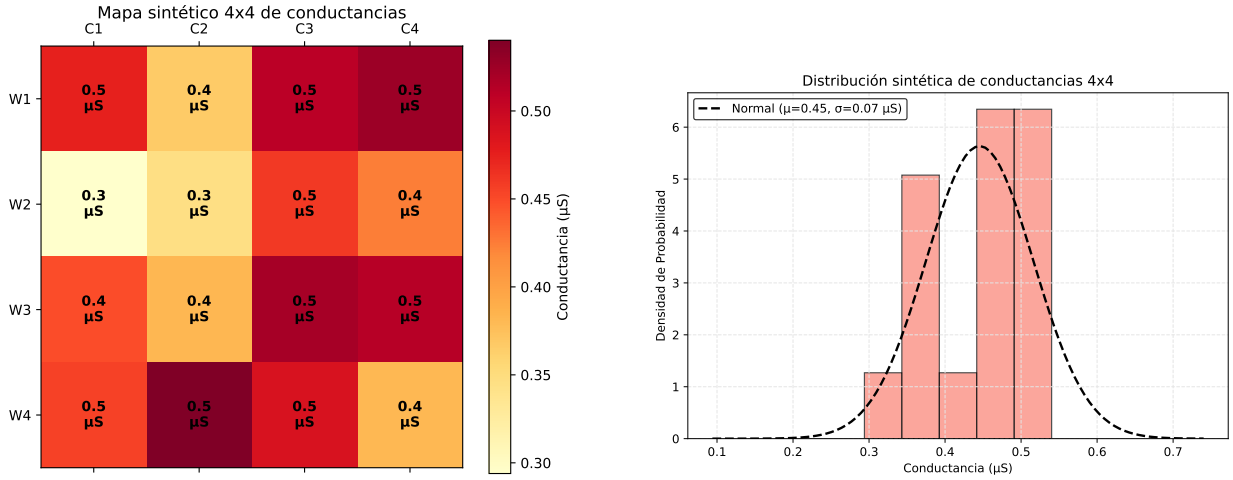


Figura **D-9**: Comparación cualitativa de la forma I–V pre-forming con el panel S3a de Prezioso et al. (2015) [25]. Las curvas se generan con un modelo paramétrico local; no se cargan puntos digitalizados ni se afirma un ajuste cuantitativo.

Propósito y Relevancia: Permite comparar visualmente el comportamiento asimétrico no lineal generado localmente con la forma de las curvas representativas del panel S3a. No acredita fidelidad de fabricación, ajuste numérico ni corrientes de fuga medidas.



(a) Matriz sintética de conductancias 4×4.

(b) Histograma de la matriz sintética.

Figura **D-10**: Ejemplos sintéticos de conductancias en una matriz 4×4 y su distribución. Los valores se generan en el simulador con semilla fija; no son mediciones ni una reproducción de las Figuras S3b–S3c de Prezioso et al. (2015) [25].

Propósito y Relevancia: Ilustran la generación de una matriz virtual y un resumen estadístico de sus conductancias. No demuestran que la dispersión del simulador coincida con la variabilidad Device-to-Device del hardware reportado.

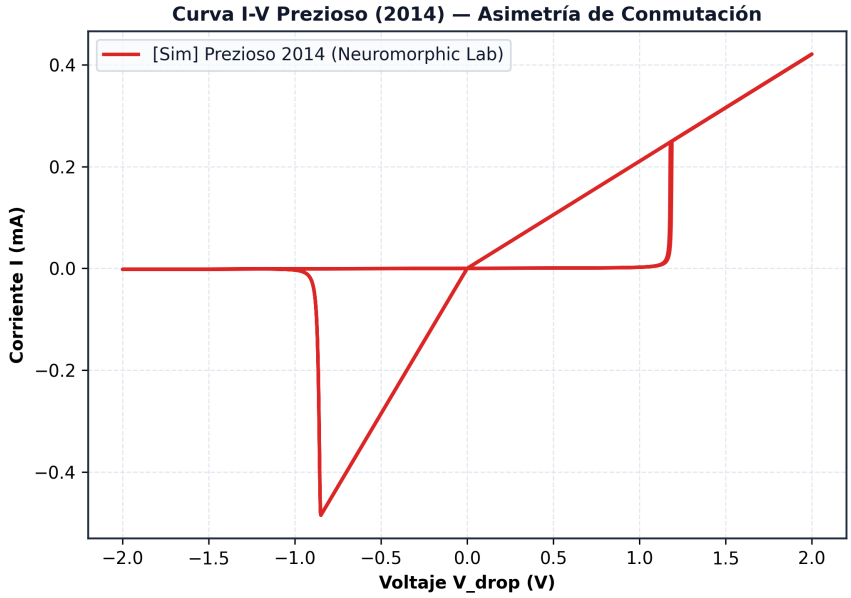


Figura **D-11**: Respuesta I–V simulada con el modelo local inspirado en Prezioso et al. (2014) [6]. La curva se genera con parámetros configurados en el código; no incluye puntos experimentales ni constituye una validación cuantitativa del artículo.

Propósito y Relevancia: La figura muestra la respuesta I–V de una simulación local de un dispositivo bajo un barrido de voltaje. No muestra una matriz Crossbar ni prueba integración de sistema, escalamiento, operación SET/RESET en hardware o correspondencia cuantitativa con mediciones del artículo.

Aplicación del crossbar al agente NeuroBot 24×12

En esta sección se describe una arquitectura de aprendizaje por refuerzo (*Reinforcement Learning*, RL) simulada: una red virtual de 24×12 acoplada al entorno CartPole. Los resultados son de simulación y no representan una validación experimental en hardware robótico o memristivo.

Arquitectura del Sistema (24×12) El cerebro neuromórfico del agente consiste en un crossbar memristivo de $N = 24$ filas de entrada y $M = 12$ columnas de salida.

Codificador de Estados (Encoder Top-K) El vector de observación continuo $\mathbf{s} = [x, \dot{x}, \theta, \dot{\theta}] \in \mathbb{R}^4$ se proyecta mediante una batería de receptores gaussianos distribuidos a lo largo del dominio de cada variable. Se aplica un mecanismo Top-K ($K = 10$) que selecciona las 10 filas de mayor activación, generando un patrón binario disperso de voltajes de lectura $V_{\text{rows}} \in \{0, V_{\text{read}}\}$.

Crossbar Memristivo y Solución Nodal (MNA) La corriente integrada por columna $\mathbf{I}_{\text{col}} \in \mathbb{R}^{12}$ se calcula resolviendo la matriz MNA con resistencia de línea $R_{\text{wire}} = 10^{-3}\,\Omega$ y resistencia de sensado $R_{\text{sense}} = 100\,\Omega$:

$$\mathbf{I}_{\text{col}} = \text{MNA}(\mathbf{G}, V_{\text{rows}}, R_{\text{wire}}, R_{\text{sense}}) \tag{D-8}$$

donde $G_{ij} = \frac{1}{R_{\text{on}} + X_{ij}(R_{\text{off}} - R_{\text{on}})}$ representa la conductancia de la celda en el estado interno $X_{ij} \in [0, 1]$.

Decodificador y Acción (Softmax e Inercia de Motor) Las 12 columnas se agrupan en pares de conductancias asociadas a 6 niveles discretos de fuerza activa para giro hacia la izquierda ($1 - 30\,\text{N}$) y 6 para la derecha ($1 - 30\,\text{N}$). La probabilidad de selección de acción π_j sigue una distribución Softmax parametrizada por la temperatura ambiental T:

$$\pi_j = \frac{\exp(I_j/T)}{\sum_{k=1}^{M} \exp(I_k/T)} \tag{D-9}$$

Se aplica una tasa de cambio (slew rate) de 50/50 en la dinámica del actuador para simular la inercia del motor robótico real.

REINFORCE con baseline El aprendizaje del agente no es STDP: es REINFORCE con baseline y ocurre al final de cada episodio:

$$\Delta X_{ij} = -\eta \cdot E_{ij} \cdot (R_{\text{ep}} - b) \tag{D-10}$$

donde E_{ij} es la elegibilidad acumulada por paso a partir de $\text{pre}_i(t)(\mathbf{1}_{j=a(t)} - \pi_j(t))$, con decaimiento γ entre pasos. No se usa una ventana de STDP entre disparos pre y postsinápticos ni una traza de memristores volátiles; E se mantiene como acumulador matemático del agente. Con la convención $X = 1 - x$, una ventaja positiva produce $\Delta X < 0$ en la columna elegida cuando su elegibilidad es positiva; esto reduce su resistencia y aumenta su conductancia. La prueba unitaria del agente comprueba este signo en un caso controlado, sin demostrar convergencia general ni equivalencia con STDP.
La actualización ΔX utiliza recorte por norma de gradiente ($\max|\Delta X| \leq 0{,}05$) y se congela si $R_{\text{ep}} \geq 430$. Este umbral es un criterio operativo del protocolo evaluado y no demuestra que la política alcanzada sea óptima.

Resultados de simulación y discusión en control robótico Los valores de recompensa y ablación de esta sección proceden de registros de simulación previamente reportados. No se reejecutaron las cinco semillas ni las ablaciones durante esta revisión; por tanto, se presentan como resultados reportados, no como cifras reproducidas aquí ni como resultados experimentales de hardware.

Convergencia y Reproducibilidad Multi-Semilla Se ejecutaron cinco semillas (0–4), con 300 episodios cada una, $V_{\text{read}} = 0{,}05$ V, D2D ($\sigma = 0{,}08$), C2C ($\sigma = 0{,}05$, $\theta = 1\,\text{s}^{-1}$, $dt = 0{,}02\,\text{s}$) y congelamiento de la actualización de pesos para $R_{\text{ep}} \geq 430$. Las recompensas medias de los últimos 200 episodios fueron 369.66, 421.44, 474.31, 462.79 y 448.36, respectivamente; la media entre semillas fue 435.31 y su desviación estándar 37.28. Cuatro corridas alcanzaron al menos una recompensa episódica de 480; la semilla 3 tuvo máximo 479.14. Los registros se guardan como `run_24x12_A_d2d_c2c_seed0.json`–`seed4.json`; la figura y discusión se presentan en la Sección 3.4.1.
La media de la semilla 0 debe interpretarse considerando su trayectoria: las recompensas medias por bloques de 50 episodios fueron 13.90, 58.55, 99.39, 470.69, 461.27 y 447.29. Su primera media móvil de 20 episodios que alcanzó 430 apareció en el episodio 157. Así, el promedio de los episodios 101–300 (369.66) refleja en buena medida el tiempo que tardó esta corrida en alcanzar recompensas altas y no basta para concluir que la semilla sea intrínsecamente peor. Este cruce es un indicador operativo de adquisición de desempeño, no una demostración formal de convergencia; el análisis detallado se presenta en la Sección 3.5.1.

Estudio de Ablaciones Para validar la contribución de cada bloque del agente, se evaluaron cuatro configuraciones (Figura **D-12**):

- **A (Control Base):** Sistema completo con decaimiento térmico y traza acumulativa ($\bar{R} \approx 491{,}9$ en semillas exitosas, 60 % de tasa de convergencia en 5 semillas aleatorias).
- **B (Sin Adaptación Térmica, $T = 0{,}10$ Fijo):** Colapso por congelamiento estocástico ($\bar{R} = -39{,}0$). Demuestra que sin alta exploración inicial, el agente congela la búsqueda de políticas.
- **C (Sin Traza de Elegibilidad, $\varepsilon = 0$):** Colapso total ($\bar{R} = -33{,}3$, $\Delta X = 0$), evidenciando que la elegibilidad acumulada es necesaria en la configuración REINFORCE evaluada.
- **D (Hebbiano Sin Modulación $R_{\mathbf{ep}} - b$):** Divergencia y colapso ($\bar{R} = -39{,}0$) por falta del escalar de refuerzo global.

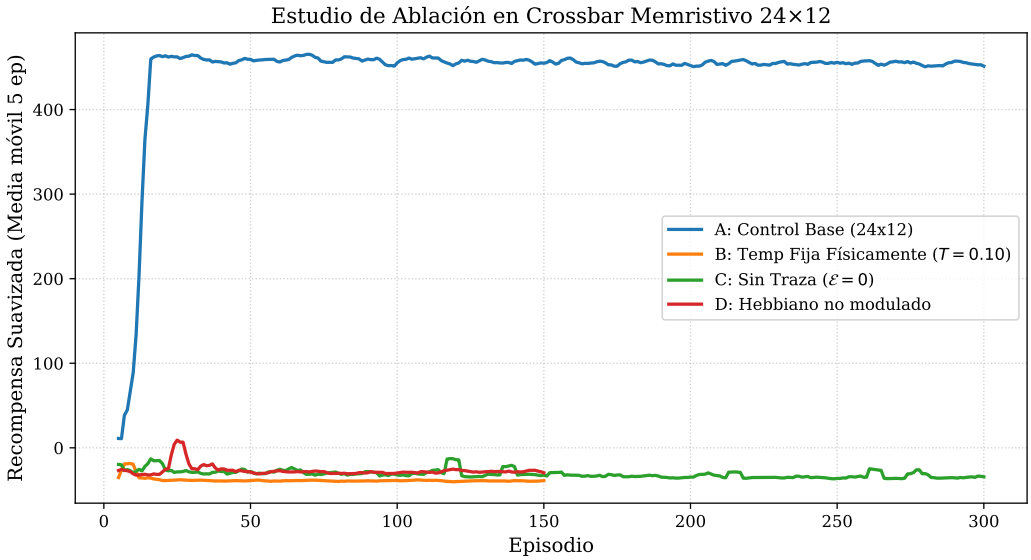


Figura **D-12**: Estudio de ablación comparativo (A, B, C, D) en el crossbar 24×12.

Heatmap de la Matriz Memristiva Consolidada (X) La Figura **D-13** muestra la distribución final del estado memristivo $X \in [0, 1]$. Se observa una estructura bimodal no saturada ($\mu = 0{,}165, \sigma = 0{,}118$) con bandas claras de activación sináptica diferenciadas por la velocidad y el ángulo del poste.

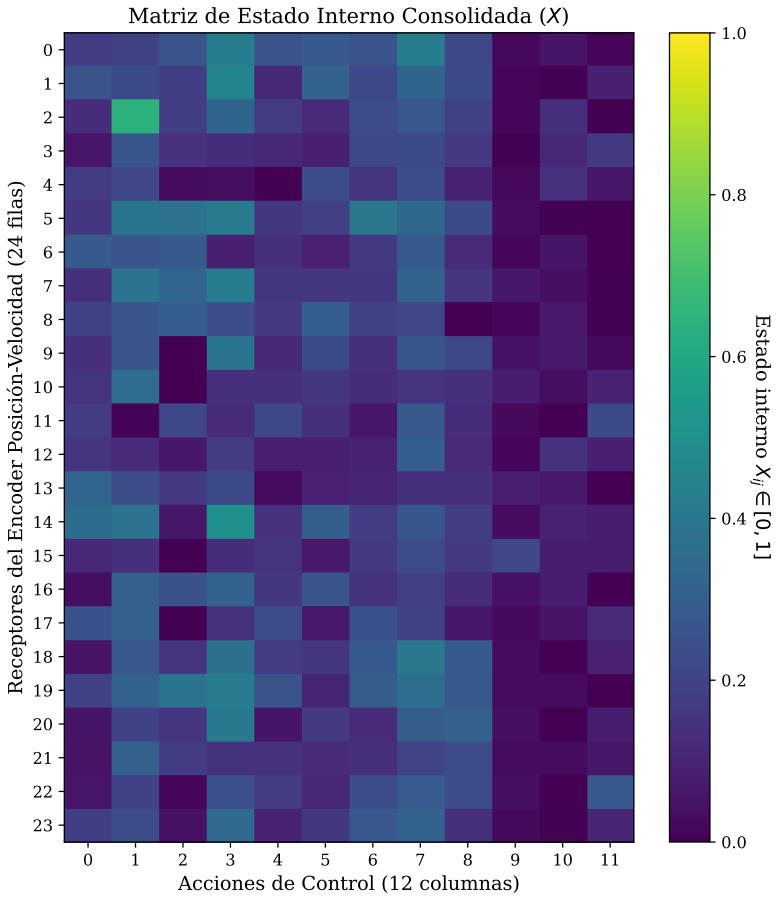


Figura **D-13**: Heatmap de la variable de estado interno X tras el entrenamiento del agente; la conductancia G se deriva de X y no se muestra en esta figura.

Validación del Modelo Físico del Crossbar (Suite de 10 Experimentos)

La suite consta de 10 experimentos de simulación autónomos del modelo crossbar. Las cifras que siguen son las consignadas en sus informes y figuras existentes; no se reejecutaron ni se verificaron de nuevo sus registros de entrada en esta revisión. Por ello describen resultados reportados del modelo, no una certificación de fidelidad respecto a hardware real. Las condiciones de excitación y las impedancias de sensado ($R_{\text{sense}} = 100\,\Omega$) se atribuyen a la configuración declarada para el agente.

Exp. 1 — Error de lectura del crossbar vs. tamaño La Figura **D-14** cuantifica el error relativo de lectura del arreglo al escalar de $N = 4$ a $N = 32$ filas con $M = 12$ columnas fijas. Se define el error relativo como $\varepsilon = \|I_{\text{col}} - I_{\text{ideal}}\| / \|I_{\text{ideal}}\|$, donde I_{col} es la solución nodal del arreglo (MNA con R_{wire} y R_{sense}) e $I_{\text{ideal}} = \mathbf{G}^T\mathbf{V}$ es la proyección lineal ideal.
Para $N = 4$ el error es $\varepsilon \approx 6{,}5\,\%$ y escala monótonamente hasta $\varepsilon \approx 41\,\%$ para $N = 32$. Este crecimiento refleja la contribución conjunta de la caída resistiva en las líneas (R_{wire}) y el

divisor formado por R_{sense} con la conductancia agregada del arreglo. El efecto de corrientes parásitas estrictas (*sneak paths* en el sentido clásico: fila única activa con filas flotantes) requiere una configuración de excitación distinta y se considera una extensión natural para trabajos futuros.

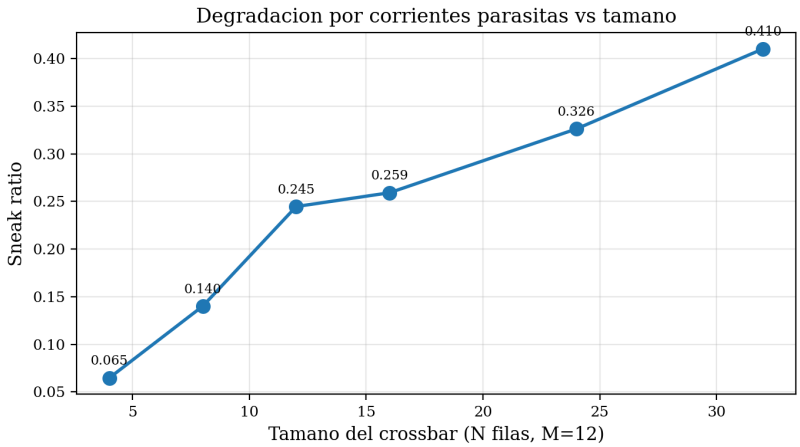


Figura **D-14**: Error relativo de lectura vs. tamaño del arreglo. Crecimiento monótono desde $N = 4$ (6,5 %) hasta $N = 32$ (41 %).

Exp. 2 — MNA real vs. modelo ideal La Figura **D-15** compara corrientes MNA e ideales para cuatro tamaños. En el caso $N = 24$, el error medio por componente alcanza 29,08 %, reflejando la contribución acumulada de R_{wire} y R_{sense} sobre las corrientes de columna.

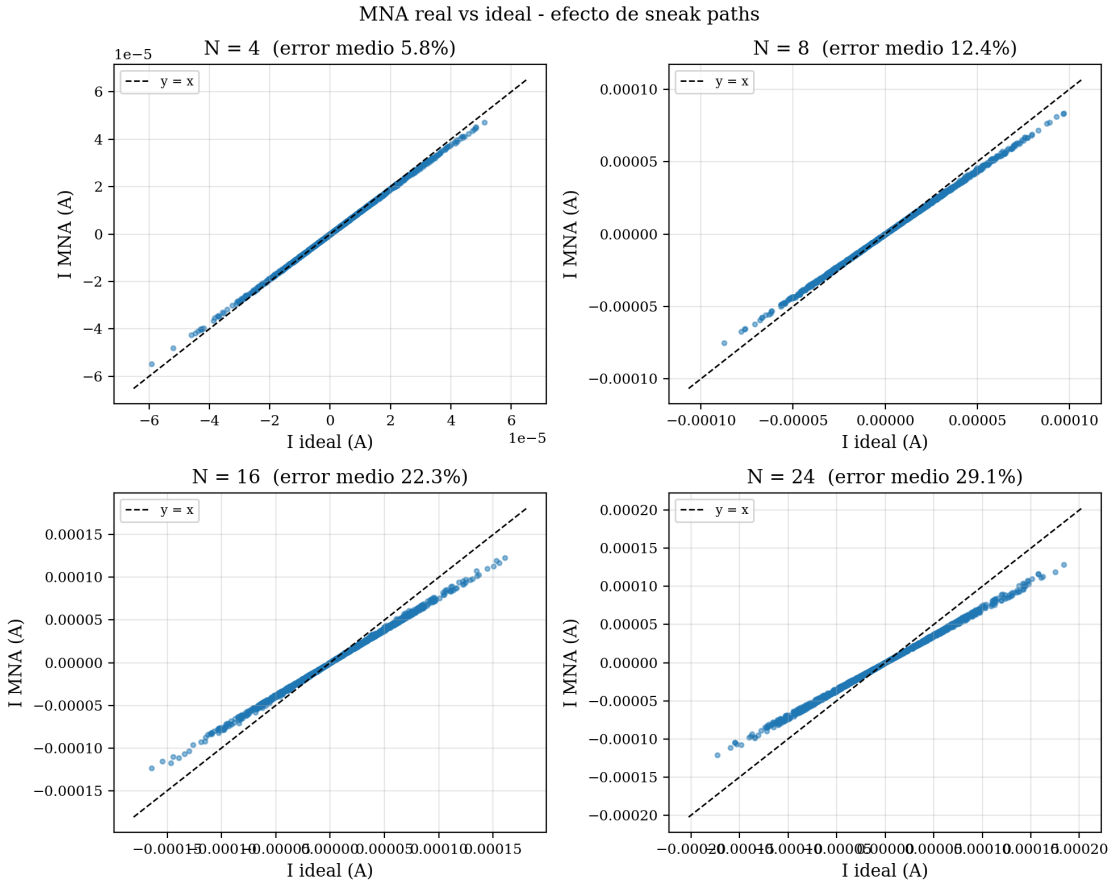


Figura **D-15**: Diagrama de dispersión MNA vs. Ideal. El error crece de forma no lineal con N.

Exp. 3 — Caída de tensión en línea (R_{wire}) La Figura **D-16** ilustra cómo R_{wire} distorsiona el perfil de corriente. El efecto sobre el perfil es doble: atenúa la corriente media ($-13\,\%$ entre $R_{\text{wire}} = 1\,\text{m}\Omega$ y $R_{\text{wire}} = 10\,\Omega$) y mantiene prácticamente invariante la no-uniformidad relativa entre columnas ($\sim 27\,\%$), dominada por la dispersión dispositivo-a-dispositivo de **G**.

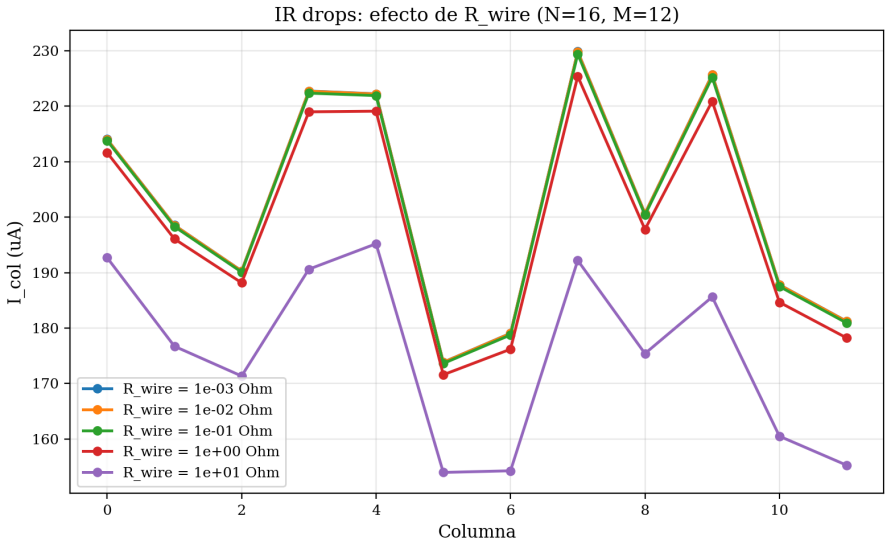


Figura **D-16**: Perfil de I_{col} para distintos R_{wire}. La atenuación de la corriente media aumenta con la resistencia de línea, mientras que la dispersión entre columnas se mantiene dominada por la variabilidad de **G**.

Exp. 4 — Precisión en VMM La Figura **D-17** muestra la distribución del error relativo en VMM para 1000 vectores aleatorios. El error medio crece de 5,82 % ($N = 4$) a 29,05 % ($N = 24$).

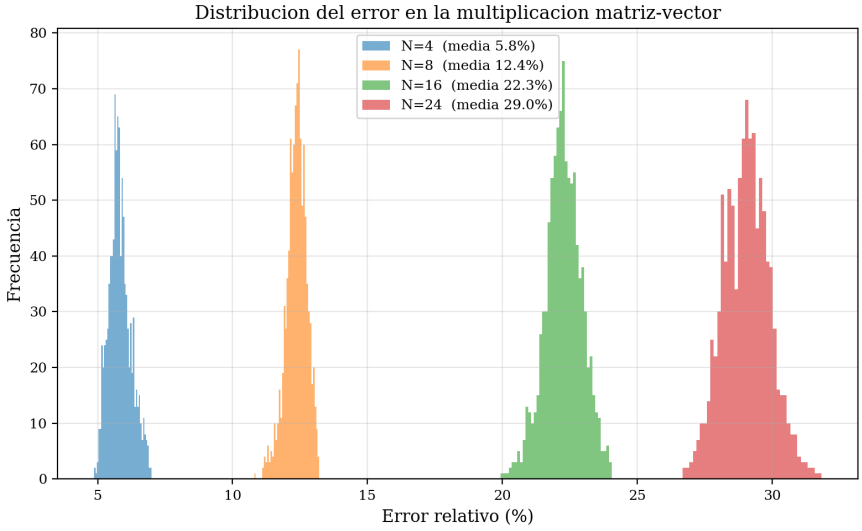


Figura **D-17**: Histograma del error relativo en VMM por tamaño de crossbar.

Exp. 5 — Half-select disturb simulado: V/2 vs. V/3 La Figura **D-18** compara la perturbación calculada por el modelo para celdas *half-selected*. Los valores reportados son

50.0 % para V/2 y 33.3 % para V/3; son razones de excitación del protocolo simulado, no medidas de disturb en celdas fabricadas.

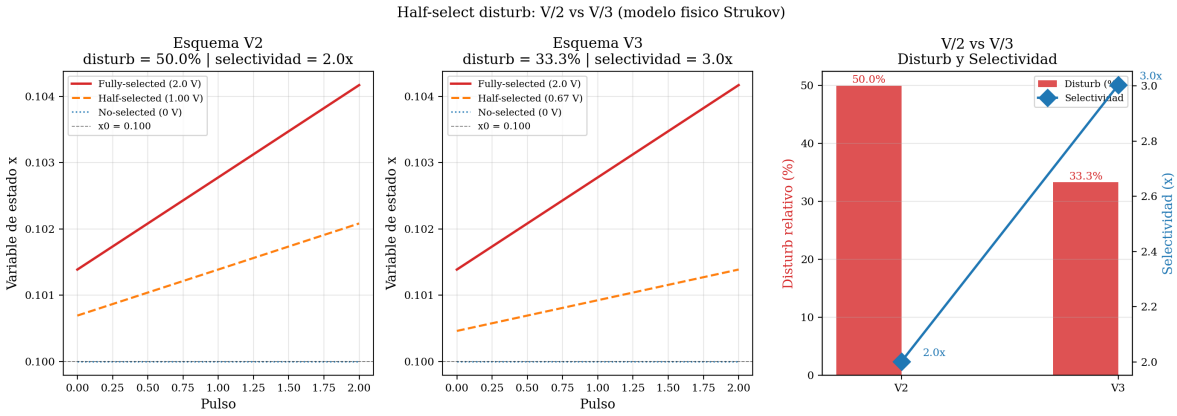


Figura **D-18**: Evolución de X para celda fully-, half- y no-selected bajo 3 pulsos.

Exp. 6 — Estado en reposo del modelo El resultado reportado tras 1000 pasos con $V_{\text{rows}} = 0\,\text{V}$ es un cambio máximo en X de $\Delta_{\text{máx}} = 0$ bajo las condiciones del experimento. Esto describe el modelo y protocolo evaluados, no certifica la retención de un dispositivo físico.

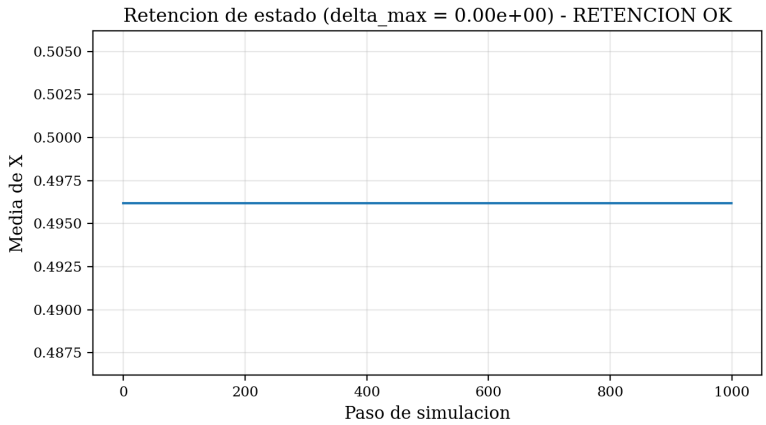


Figura **D-19**: Media de X durante 1000 pasos en reposo para la simulación reportada.

Exp. 7 — Respuesta simulada a pulsos LTP/LTD La Figura **D-20** muestra la respuesta reportada del modelo bajo pulsos de potenciación (LTP, $+1\,\text{V}$) y depresión (LTD, $-1\,\text{V}$). Para 50 pulsos desde $x_0 = 0{,}1$, se reportan $x = 0{,}135$ para LTP y $x = 0{,}066$ para LTD. Estos valores describen la ejecución simulada y no se presentan como una respuesta experimental de un dispositivo real.

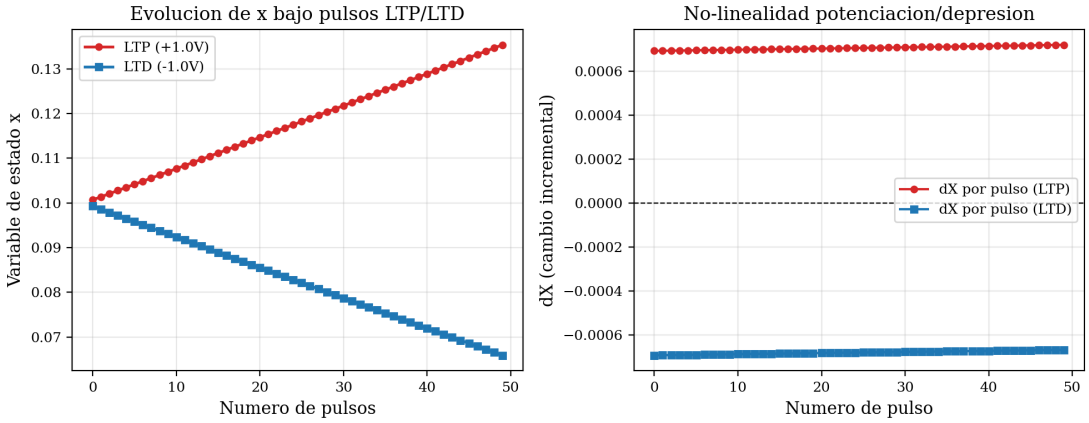


Figura **D-20**: LTP/LTD y derivada discreta. La no-linealidad satura suavemente los extremos.

Exp. 8 — Variabilidad dispositivo a dispositivo (D2D) La Figura **D-21** cuantifica la dispersión de I_{col} entre 20 crossbars virtuales. La variabilidad dispositivo-a-dispositivo se caracteriza por $CV(I_{\text{col}}) = 40{,}3\,\%$ en términos de la corriente integrada, equivalente al $4{,}0\,\%$ del rango dinámico de **G**.

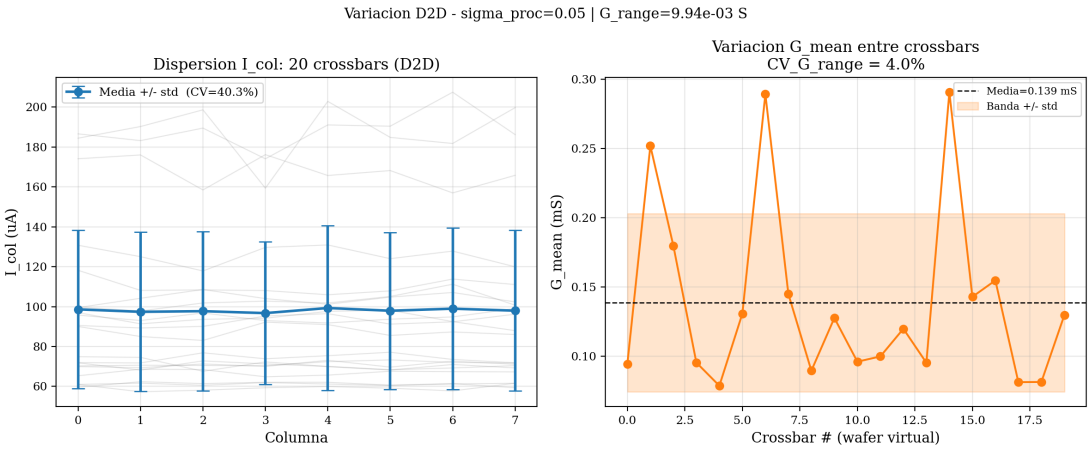


Figura **D-21**: Dispersión de I_{col} y variación de $\bar{G}_{\text{mean}}$ entre 20 crossbars virtuales.

Exp. 9 — Costo computacional del solver MNA La Figura **D-22** presenta un ajuste empírico reportado de $\mathcal{O}(N^{0{,}85})$ y un tiempo aproximado de ~ 3 ms por paso para $N = 24$ en el equipo usado entonces. El exponente es propio de esa muestra y configuración: no establece la complejidad asintótica del solver ni permite generalizar la comparación con $\mathcal{O}(N^3)$ sin conocer el rango medido, el hardware y el procedimiento de temporización.

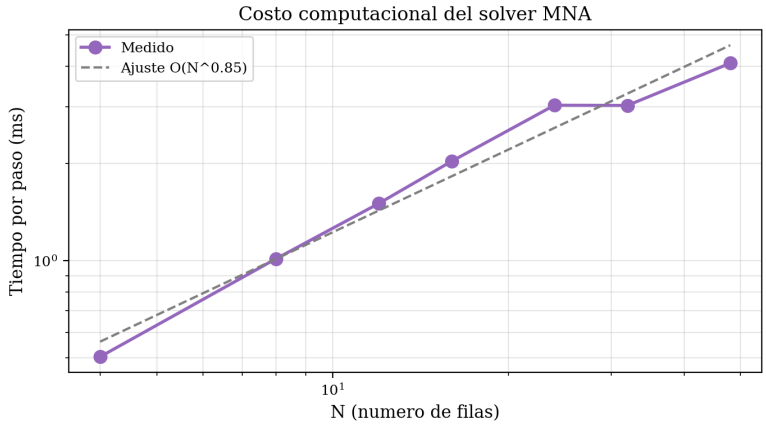


Figura **D-22**: Costo computacional reportado del solver MNA en función de N. El ajuste de las muestras archivadas es $\mathcal{O}(N^{0,85})$.

Exp. 10 — Crosstalk entre columnas La Figura **D-23** expone el efecto de crosstalk: al excitar únicamente la fila 0, las otras columnas muestran una desviación media del 13,32 % por interacción parásita MNA.

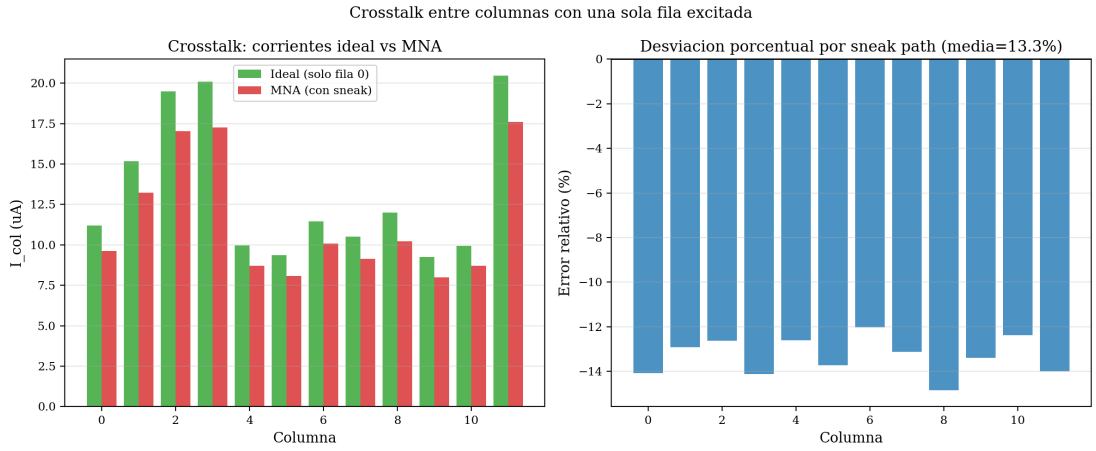


Figura **D-23**: I_{col} real vs. ideal al excitar únicamente la fila 0.

Alcance de los resultados de la Etapa 4 Los diez experimentos anteriores y las figuras del agente documentan resultados guardados de simulaciones, no mediciones de hardware. Para evitar repetir sus cifras en un segundo resumen, esta sección deja cada resultado junto a su configuración y figura; la tabla maestra del Apéndice C resume únicamente las verificaciones V01–V28.

APÉNDICE E

Evidencia visual de los entrenamientos

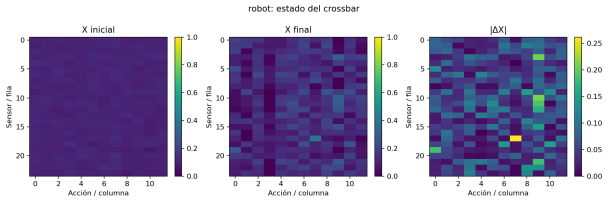
Este apéndice reúne las figuras ya generadas y archivadas para las condiciones de entrenamiento seleccionadas. Cada condición muestra el estado final del crossbar y la curva de éxito de su propio registro; las figuras no son corridas adicionales. El primer lote contiene 28 condiciones (CartPole, visión, obstáculos y Car R-STDP; semillas 0 y 1; $X_0 = 0{,}1$ y $X_0 = 0{,}9$). El segundo contiene cuatro condiciones adicionales de Car R-STDP (semillas 2 y 4), con evaluación cada 100 episodios. Los protocolos, registros JSON, matrices y hashes están en las carpetas citadas en cada bloque.

La curva corresponde a la tasa de éxito durante el entrenamiento. El campo de criterio indica si el registro cumplió la regla de parada de su protocolo; la evaluación greedy y la evaluación en mapas apartados, cuando existen, se informan por separado en las tablas de resultados. El umbral de ventana móvil es 271 éxitos de 300 episodios (90.33 %), no 300 éxitos consecutivos.

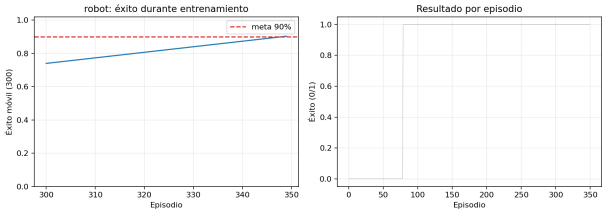
CartPole | REINFORCE | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 349; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



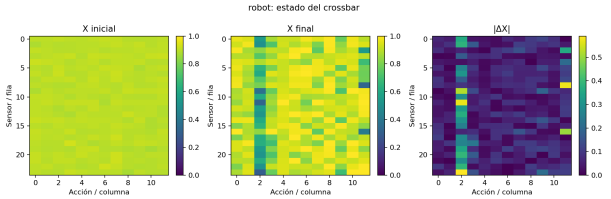
Curva de éxito



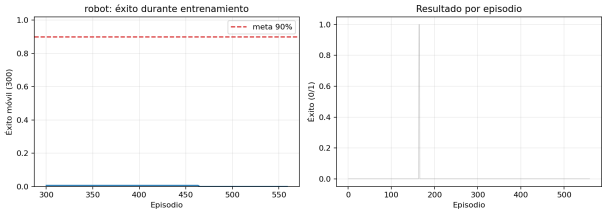
CartPole | REINFORCE | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 559; tasa de éxito en la ventana final: 0.00 %; criterio alcanzado: no.

Crossbar X



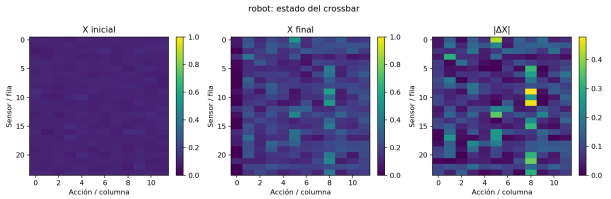
Curva de éxito



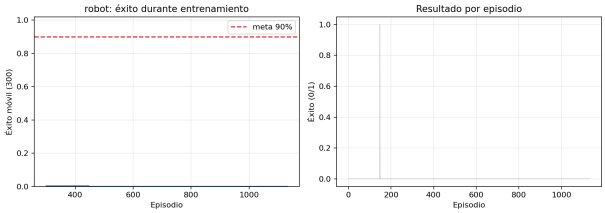
CartPole | REINFORCE | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 1132; tasa de éxito en la ventana final: 0.00 %; criterio alcanzado: no.

Crossbar X



Curva de éxito



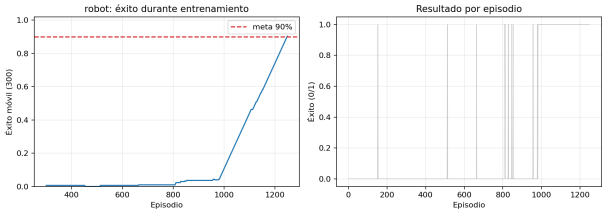
CartPole | REINFORCE | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 1249; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



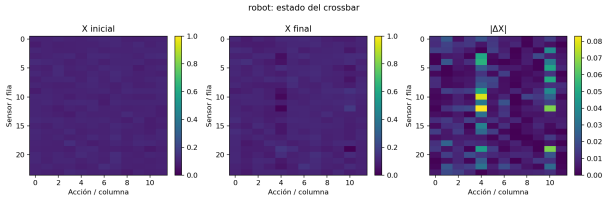
Curva de éxito



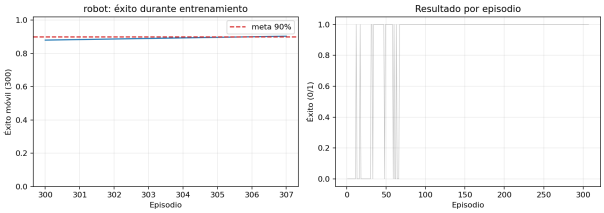
CartPole | R-STDP | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 307; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



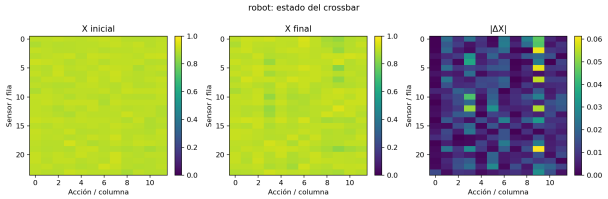
Curva de éxito



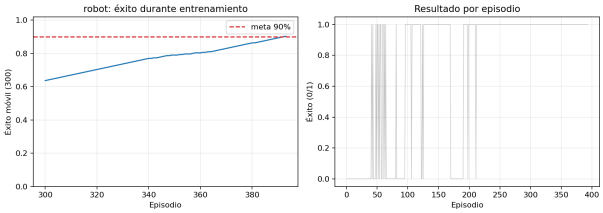
CartPole | R-STDP | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 393; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



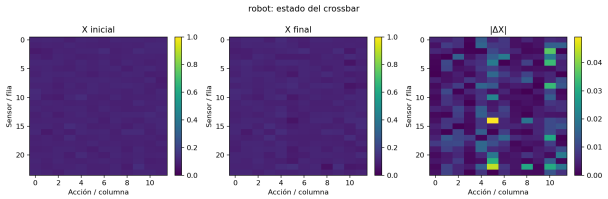
Curva de éxito



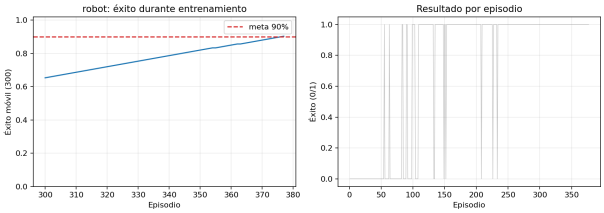
CartPole | R-STDP | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 377; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



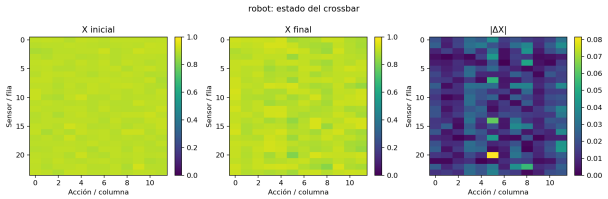
Curva de éxito



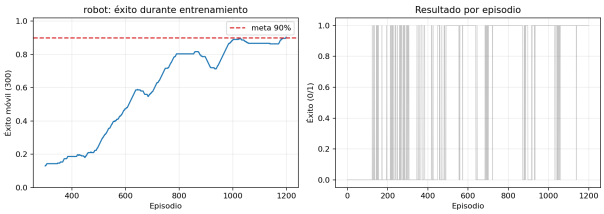
CartPole | R-STDP | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 1199; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



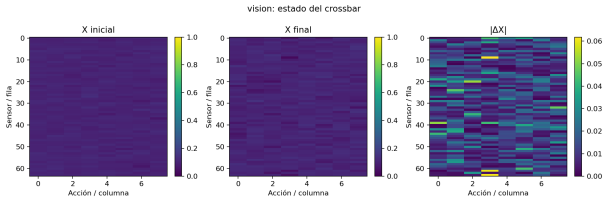
Curva de éxito



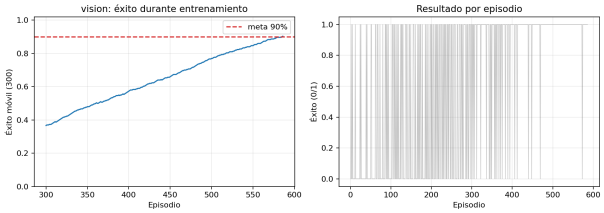
Visión | REINFORCE | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 586; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



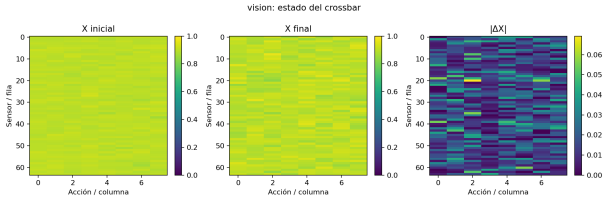
Curva de éxito



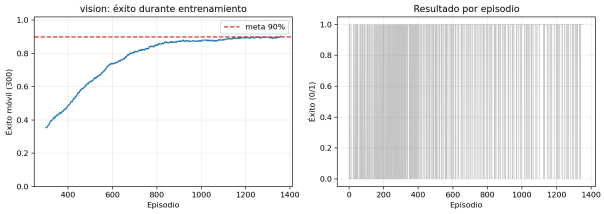
Visión | REINFORCE | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 1357; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



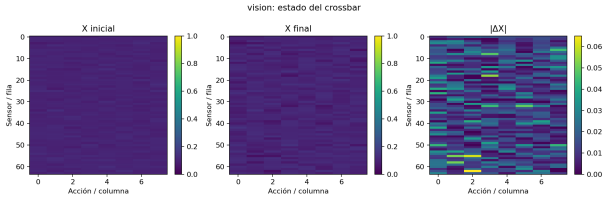
Curva de éxito



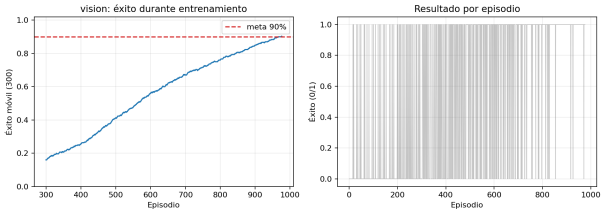
Visión | REINFORCE | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 976; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



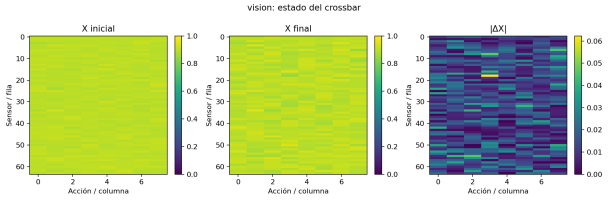
Curva de éxito



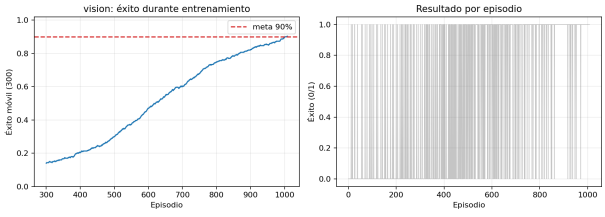
Visión | REINFORCE | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 1008; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



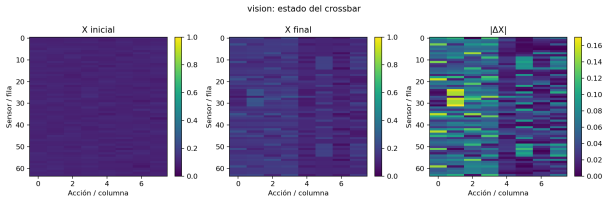
Curva de éxito



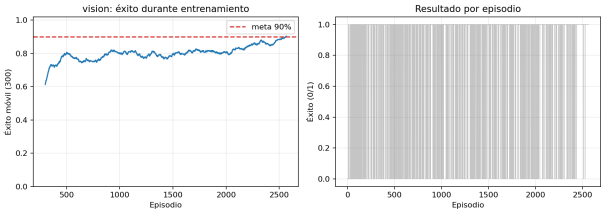
Visión | R-STDP | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 2566; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



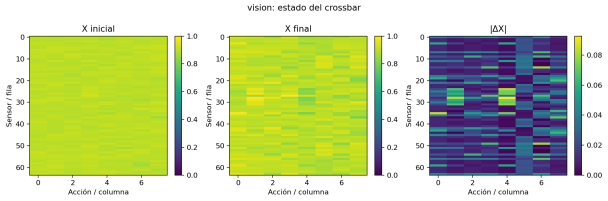
Curva de éxito



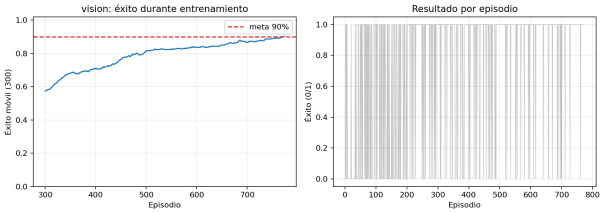
Visión | R-STDP | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 773; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



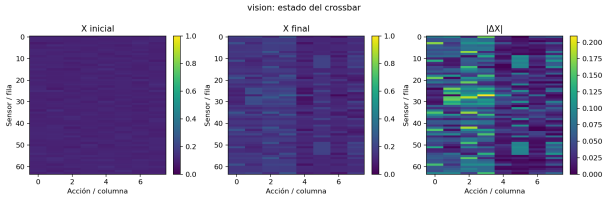
Curva de éxito



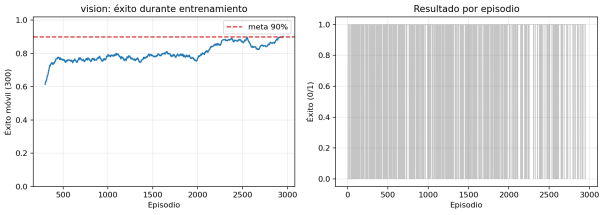
Visión | R-STDP | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 2951; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



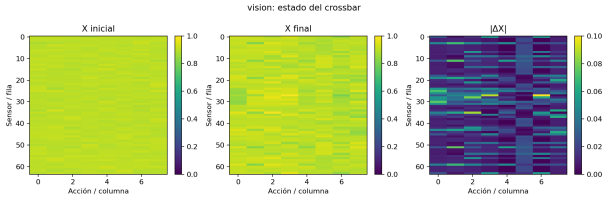
Curva de éxito



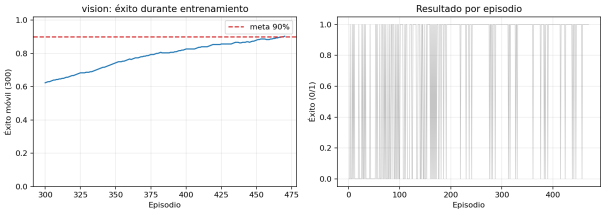
Visión | R-STDP | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 470; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



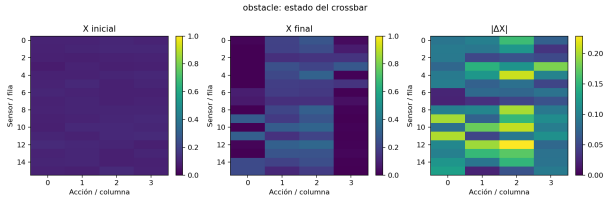
Curva de éxito



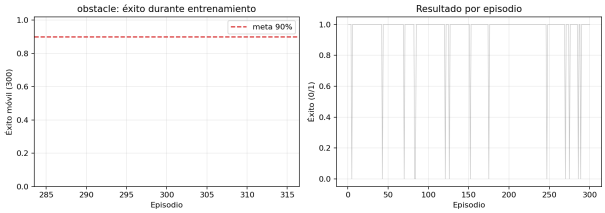
Obstáculos | REINFORCE | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 300; tasa de éxito en la ventana final: 95.33 %; criterio alcanzado: sí.

Crossbar X



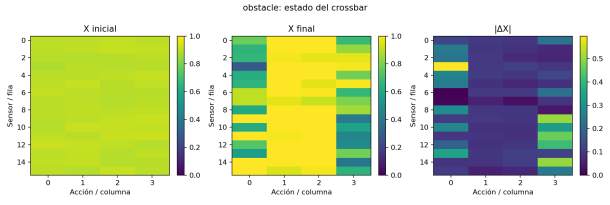
Curva de éxito



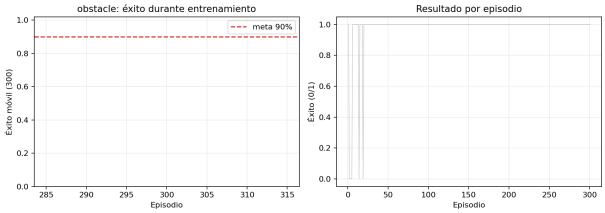
Obstáculos | REINFORCE | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 300; tasa de éxito en la ventana final: 98.00 %; criterio alcanzado: sí.

Crossbar X



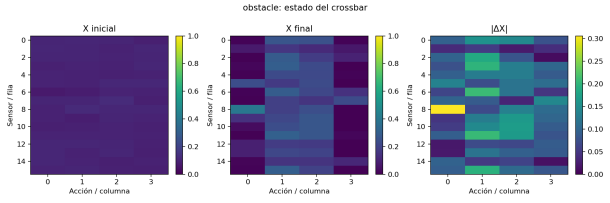
Curva de éxito



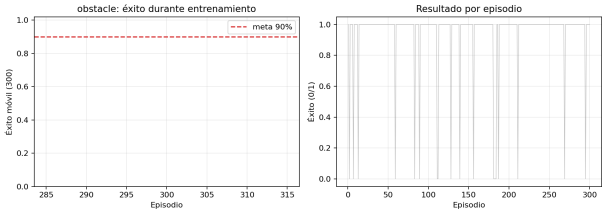
Obstáculos | REINFORCE | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 300; tasa de éxito en la ventana final: 93.67 %; criterio alcanzado: sí.

Crossbar X



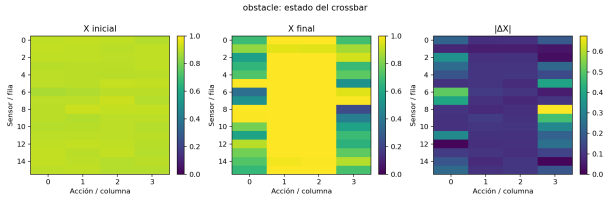
Curva de éxito



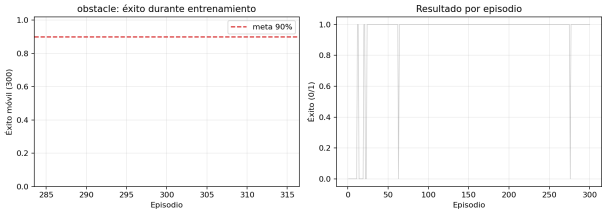
Obstáculos | REINFORCE | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 300; tasa de éxito en la ventana final: 93.00 %; criterio alcanzado: sí.

Crossbar X



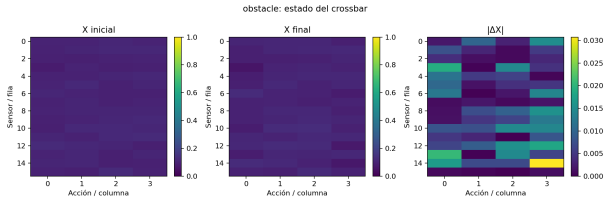
Curva de éxito



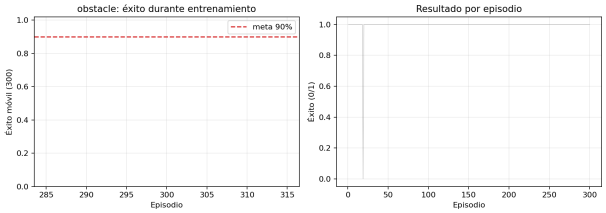
Obstáculos | R-STDP | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 300; tasa de éxito en la ventana final: 99.67 %; criterio alcanzado: sí.

Crossbar X



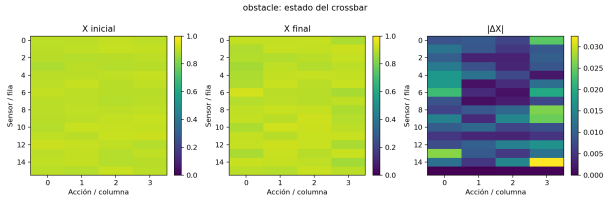
Curva de éxito



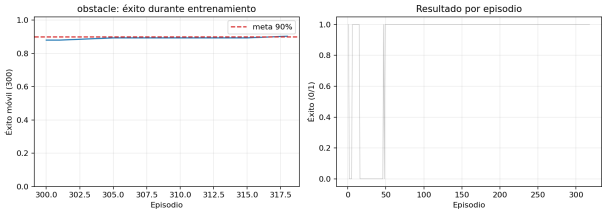
Obstáculos | R-STDP | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 318; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



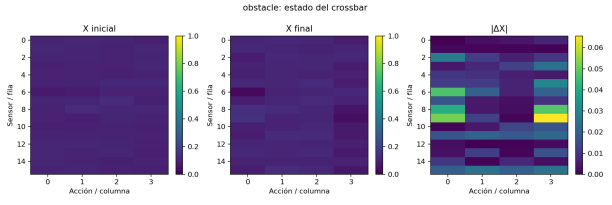
Curva de éxito



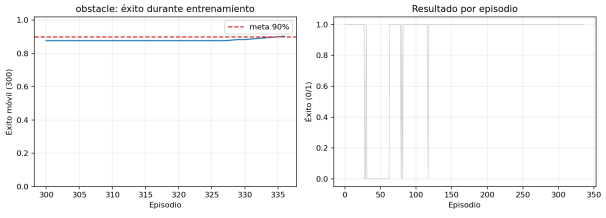
Obstáculos | R-STDP | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 336; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



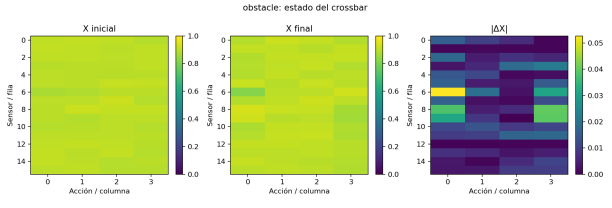
Curva de éxito



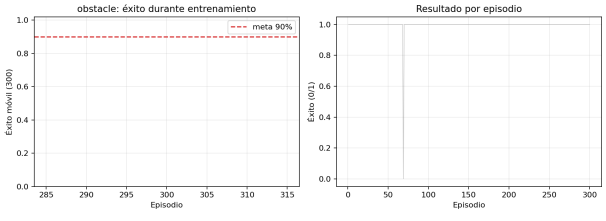
Obstáculos | R-STDP | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 300; tasa de éxito en la ventana final: 99.67 %; criterio alcanzado: sí.

Crossbar X



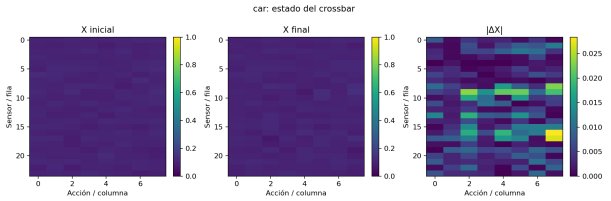
Curva de éxito



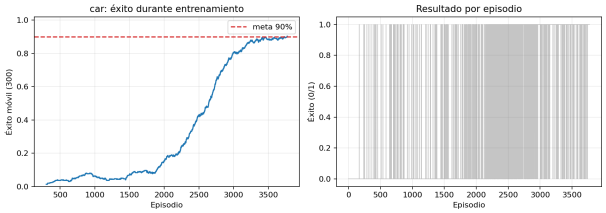
Car | R-STDP | semilla 0 | $X_0 = 0{,}1$

lote principal; episodios completados: 3775; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



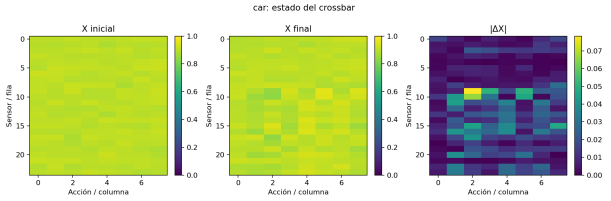
Curva de éxito



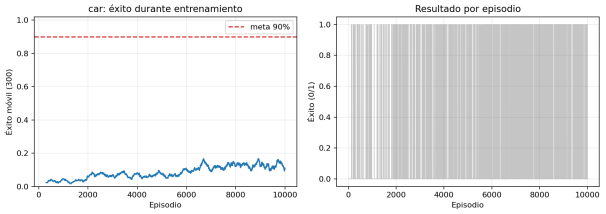
Car | R-STDP | semilla 0 | $X_0 = 0{,}9$

lote principal; episodios completados: 10000; tasa de éxito en la ventana final: 11.00 %; criterio alcanzado: no.

Crossbar X



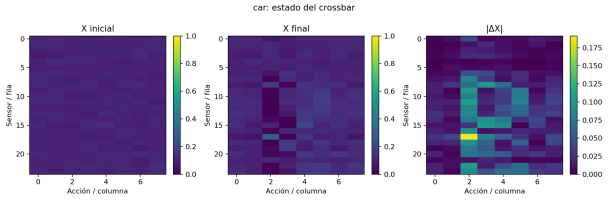
Curva de éxito



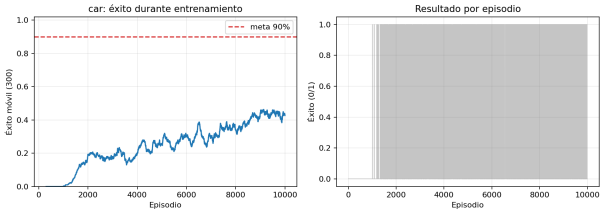
Car | R-STDP | semilla 1 | $X_0 = 0{,}1$

lote principal; episodios completados: 10000; tasa de éxito en la ventana final: 42.67 %; criterio alcanzado: no.

Crossbar X



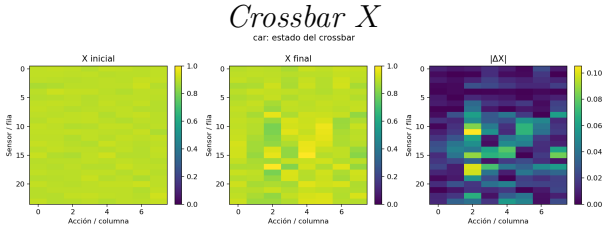
Curva de éxito



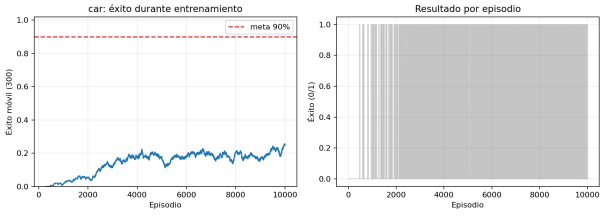
Car | R-STDP | semilla 1 | $X_0 = 0{,}9$

lote principal; episodios completados: 10000; tasa de éxito en la ventana final: 25.33 %; criterio alcanzado: no.

Crossbar X



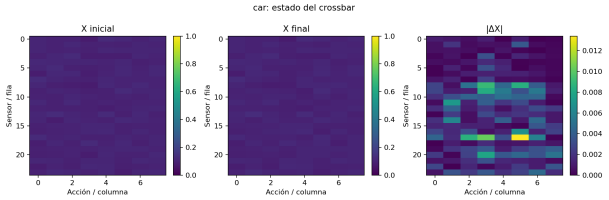
Curva de éxito



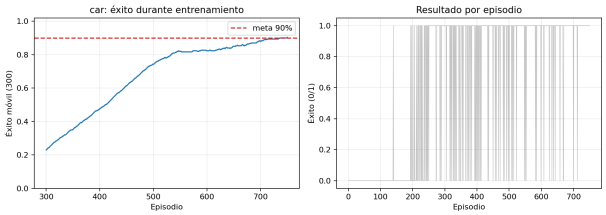
Car | R-STDP | semilla 2 | $X_0 = 0{,}1$

Car R-STDP, evaluación cada 100 episodios; episodios completados: 750; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



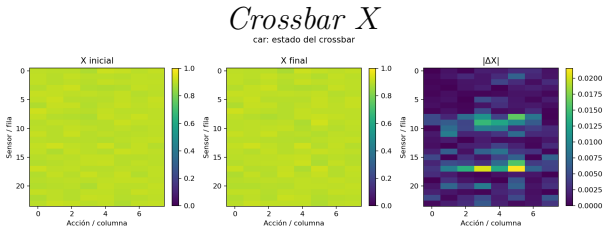
Curva de éxito



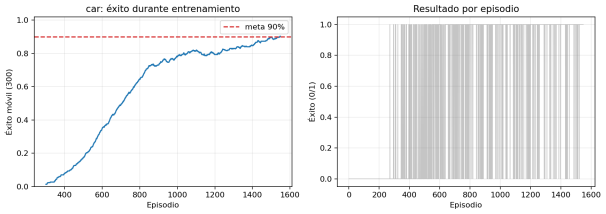
Car | R-STDP | semilla 2 | $X_0 = 0{,}9$

Car R-STDP, evaluación cada 100 episodios; episodios completados: 1548; tasa de éxito en la ventana final: 90.33 %; criterio alcanzado: sí.

Crossbar X



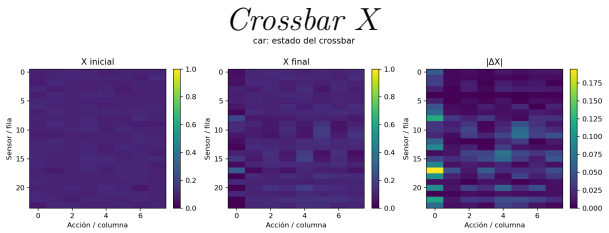
Curva de éxito



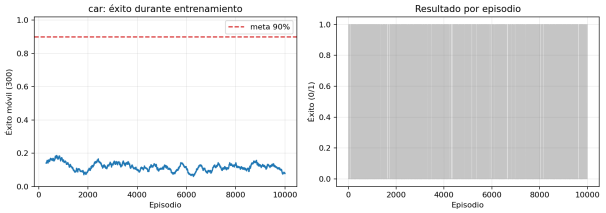
Car | R-STDP | semilla 4 | $X_0 = 0{,}1$

Car R-STDP, evaluación cada 100 episodios; episodios completados: 10000; tasa de éxito en la ventana final: 8.00 %; criterio alcanzado: no.

Crossbar X



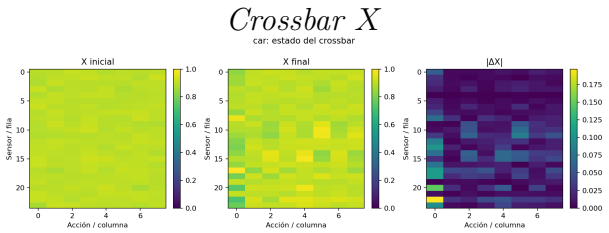
Curva de éxito



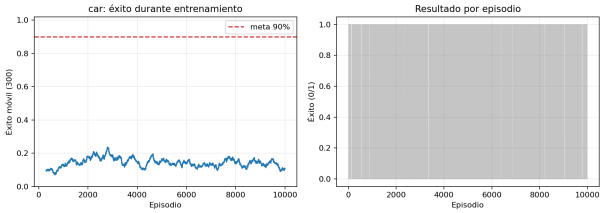
Car | R-STDP | semilla 4 | $X_0 = 0{,}9$

Car R-STDP, evaluación cada 100 episodios; episodios completados: 10000; tasa de éxito en la ventana final: 10.67 %; criterio alcanzado: no.

Crossbar X



Curva de éxito



Bibliografía

[1] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14. [Online]. Available: https://doi.org/10.1109/ISSCC.2014.6757323

[2] K. Berggren, Q. Xia, K. K. Likharev, D. B. Strukov *et al.*, "Roadmap on emerging hardware and technology for machine learning," *Nanotechnology*, vol. 32, no. 1, p. 012002, 2021. [Online]. Available: https://doi.org/10.1088/1361-6528/aba70f

[3] X. Wang, Y. Zhu, Z. Zhou, X. Chen, and X. Jia, "Memristor-based spiking neuromorphic systems toward brain-inspired perception and computing," *Nanomaterials*, vol. 15, no. 14, p. 1130, 2025. [Online]. Available: https://doi.org/10.3390/nano15141130

[4] L. O. Chua, "Memristor—the missing circuit element," *IEEE Transactions on Circuit Theory*, vol. CT-18, no. 5, pp. 507–519, Sep 1971. [Online]. Available: https://ieeexplore.ieee.org/document/1083337

[5] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, no. 7191, pp. 80–83, May 2008. [Online]. Available: https://www.nature.com/articles/nature06932

[6] M. Prezioso, F. Merrikh-Bayat, B. D. Hoskins, G. C. Adam, K. K. Likharev, and D. B. Strukov, "Training and operation of an integrated neuromorphic network based on metal-oxide memristors," *Nature Communications*, vol. 5, p. 4801, 2014. [Online]. Available: https://www.nature.com/articles/ncomms5801

[7] T. F. Schranghamer, A. Pannone, J. M. Kumar, D. K. Thiyyadi Baiju, C. Chen, T. McKnight, S. Tadekawa, E. Haines, R. Ordonez, C. Hayashi, J. M. Redwing, and S. Das, "Large-scale crossbar arrays based on three-terminal mos2 memtransistors," *Nature Communications*, vol. 16, no. 1, 2025. [Online]. Available: https://doi.org/10.1038/s41467-025-64536-2

[8] H. Li, S. Wang, X. Zhang *et al.*, "Memristive crossbar arrays for storage and computing applications," *Advanced Intelligent Systems*, vol. 3, no. 9, 2021. [Online]. Available: https://doi.org/10.1002/aisy.202170065

[9] S. Yarragolla, T. Hemke, J. Trieschmann, F. Zahari, H. Kohlstedt, and T. Mussenbrock, "Stochastic behavior of an interface-based memristive device," *Journal of Applied Physics*, vol. 131, no. 13, p. 134304, 2022. [Online]. Available: https://pubs.aip.org/aip/jap/article/131/13/134304/2836700

[10] J. Song, Y. Wang, L. Wang, Z. Liu, Y. Lei, M. Cheng, Y. Zhang, W. Zhou, Z. Xu, X. Li, M. S. Saleem, L. Chen, B. Huang, W. Wang, and C. Li, "Ultralow cyclic and device variability memristors via deterministic resistive switching in phase segregated nickel nanofilaments," *Applied Physics Reviews*, vol. 12, no. 3, p. 031403, 2025. [Online]. Available: https://pubs.aip.org/aip/apr/article/12/3/031403/3351904

[11] V. Ostrovskii, P. Fedoseev, Y. Bobrova, and D. Butusov, "Structural and parametric identification of knowm memristors," *Nanomaterials*, vol. 12, no. 1, p. 63, 2022. [Online]. Available: https://www.mdpi.com/2079-4991/12/1/63

[12] R. Naous, M. A. Zidan, A. Sultan-Salem, and K. N. Salama, "Memristor based crossbar memory array sneak path estimation," in *2014 14th International Workshop on Cellular Nanoscale Networks and Their Applications (CNNA)*, 2014. [Online]. Available: https://doi.org/10.1109/CNNA.2014.6888656

[13] C. Lammie, W. Xiang, B. Linares-Barranco, and M. Rahimi Azghadi, "Memtorch: An open-source simulation framework for memristive deep learning systems," GitHub Repository, 2023. [Online]. Available: https://github.com/coreylammie/MemTorch

[14] Sandia National Laboratories, "Crosssim: Gpu-accelerated simulator for memristive crossbar arrays," Sandia Corporation, 2023. [Online]. Available: https://cross-sim.sandia.gov/

[15] Tsinghua University, "Mnsim 2.0: A behavior-level modeling tool for memristor-based neuromorphic computing system," GitHub Repository, 2023. [Online]. Available: https://github.com/thu-nics/MNSIM-2.0

[16] S. Gaba, P. Sheridan, J. Zhou, S. Choi, and W. Lu, "Stochastic memristive devices for computing and neuromorphic applications," *Nanoscale*, vol. 5, no. 13, p. 5872, 2013. [Online]. Available: https://doi.org/10.1039/C3NR01176C

[17] W. Wang, L. Danial, Y. Li, E. Herbelin, E. Pikhay, Y. Roizin, B. Hoffer, Z. Wang, and S. Kvatinsky, "A memristive deep belief neural network based on silicon synapses," *Nature Electronics*, vol. 5, no. 12, pp. 870–880, 2022. [Online]. Available: https://doi.org/10.1038/s41928-022-00878-9

[18] Z. Wang, T. Luo, C. Liu, W. Liu, R. S. M. Goh, and W.-F. Wong, "Enabling energy-efficient deployment of large language models on memristor crossbar: A synergy of large and small," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 2, pp. 916–933, 2025. [Online]. Available: https://doi.org/10.1109/TPAMI.2024.3483654

[19] T. F. Tiotto, A. S. Goossens, A. E. Dima, C. Yakopcic, T. Banerjee, J. P. Borst, and N. A. Taatgen, "A compact model of interface-type memristors linking physical and device properties," *arXiv preprint*, 2022. [Online]. Available: https://arxiv.org/abs/2210.01455

[20] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge, UK: Cambridge University Press, 2002.

[21] G.-q. Bi and M.-m. Poo, "Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and post-synaptic cell type," *Journal of Neuroscience*, vol. 18, no. 24, pp. 10 464–10 472, 1998.

[22] S. H. Jo, T. Chang, I. Ebong, B. Bhadviya, P. Mazumder, and W. Lu, "Nanoscale memristor device as synapse in neuromorphic systems," *Nano Letters*, vol. 10, no. 4, pp. 1297–1301, 2010.

[23] Y. N. Joglekar and S. J. Wolf, "The elusive memristor: signatures in basic electrical circuits," *European Journal of Physics*, vol. 30, no. 4, pp. 661–675, 2009.

[24] D. Biolek, Z. Biolek, and V. Biolková, "Spice model of memristor with nonlinear dopant drift," *Radioengineering*, vol. 18, no. 2, pp. 210–214, 2009.

[25] M. Prezioso, F. Merrikh-Bayat, B. D. Hoskins, G. C. Adam, K. K. Likharev, and D. B. Strukov, "Training and operation of an integrated neuromorphic network based on metal-oxide memristors," *Nature*, vol. 521, pp. 61–64, 2015.